Handwriting robot

Final Report

D.R. Braune
23539195

Submitted as partial fulfilment of the requirements of Project EPR400
in the Department of Electrical, Electronic and Computer Engineering
University of Pretoria

November 2026

Study leader: Prof. H.C. Myburgh

Part 1. Preamble

This report describes work I completed in my final year project, designing and implementing a handwriting robot that recognises handwritten text and its writer and reproduces text on paper in a chosen writer's handwriting style.

1. Project proposal and technical documentation
This main report contains an unaltered copy of the approved Project Proposal (as Part 2 of the report).

All the code that I developed appears as a separate submission on the AMS.

2. Project history
This project does not continue the work of a previous student project, and no part of a previous project's design or code was reused. The following parts of the work are not my own and are identified as such wherever they appear in the report. The physical components of the gantry and the capture rig (the two stepper motors, the two driver modules, the camera, the single-board computer, the switches, the pen, the power supply and the standard belts, pulleys, rods, bearings and fasteners) were bought, as the project proposal stated they would be. The layer arrangement of the text recogniser follows a published network described in the literature study, and is cited where it is used in Section 3.4; the implementation of that arrangement is my own. The public handwriting database used for training, and its transcriptions, are the work of its compilers and are cited. The software libraries listed under declaration 6 below are the work of their authors. Everything else reported on here, including all of the image processing, the training and inference code, the style extraction and synthesis, the G-code generation and the motion control, I designed and implemented myself. Section 6.3 states this in full.

MUST STILL BE COMPLETED BY ME

Confirm the first sentence above before submission. Appendix 4 requires this paragraph to state explicitly whether the project builds on an earlier project, and to cite that project by author and year if it does. I could not establish this from the project files, so I have written the statement that matches my own understanding — if any part of this project continues a previous student's work, name that work here by author and year and say exactly what was taken from it.

3. Language editing
This document has been language edited by a knowledgeable person. By submitting this document in its present form, I declare that this is the written material that I wish to be examined on.

My language editor was _____.

_____ _____

Language editor signature *Date*

MUST STILL BE COMPLETED BY ME

Fill in the language editor's name and obtain their signature and date, or attach their written confirmation at the end of the report and write "proof of language editing is attached" in place of the signature. The report is rejected without this.

4. Plagiarism declaration

I, D.R. Braune, understand what plagiarism is and have carefully studied the plagiarism policy of the University. I hereby declare that all the work described in this report is my own, except where explicitly indicated otherwise. Although I may have discussed the design and investigation with my study leader, fellow students or consulted various books, articles or the internet, the design/investigative work is my own. I have mastered the design and I have made all the required calculations in my lab book (and/or they are reflected in this report) to authenticate this. I am not presenting a complete solution of someone else or that I found somewhere.

Wherever I have used information from other sources, I have given credit by proper and complete referencing of the source material so that it can be clearly discerned what is my own work and what was quoted from other sources. I acknowledge that failure to comply with the instructions regarding referencing will be regarded as plagiarism. If there is any doubt about the authenticity of my work, I understand that I may have to attend an oral ancillary examination/evaluation about the work.

5. AI usage declaration

Artificial intelligence tools, including large language models and code-generation tools, were used during this project. They were used to draft and edit the text of this report, to assist with writing, refactoring and reviewing source code, and to help build scripts that audited the code and checked the reported measurements. They were not used to produce any of the measured results reported in Section 4: every number in this report comes from running the project's own code on the project's own data, and the raw output files are part of the code submission. The choice of methods, the design of the system, the decisions about what to build and what to abandon, and the interpretation of the results in Section 5 are my own.

MUST STILL BE COMPLETED BY ME

Read declaration 5 above and make it exactly true before submission, adding or removing items as needed. The template's default wording, which denies any AI use, was deleted because it is not true of this project. A declaration that does not match the facts is itself an integrity violation, independently of the quality of the work, so this paragraph must be checked word by word and signed off by me.

6. Software library usage declaration

Unless explicitly stated otherwise in the next paragraph, the author confirms that no software library functions, packages, or toolboxes were used in the design, implementation, or analysis presented in this report. All code and computations were developed entirely by the author.

The exceptions to the above are the following. NumPy (`numpy.org`) was used for array storage and elementary array arithmetic, and Pillow (`python-pillow.org`) for reading and writing image files; no image-processing, machine-learning or scientific routine from either was used in the signal path, and the convolution, normalisation, pooling, recurrent, loss and optimisation routines were written from first principles as Section 3.4 describes. SciPy (`scipy.org`) was used in two places that are not part of the deployed system: `scipy.ndimage.gaussian_filter` and `scipy.ndimage.map_coordinates` in the training-time image augmentation, and `scipy.signal.find_peaks` in the script that cuts the public database pages into lines. Flask and Flask-CORS (`flask.palletsprojects.com`) provide the web server. OpenCV (`opencv.org`) is used only to read frames from the USB camera, and libgpiod (`libgpiod` Python bindings) only to drive and read the general-purpose input and output pins of the single-board computer; both are hardware-interfacing libraries and are imported only inside the single function that needs each. The Hugging Face `datasets` package (`huggingface.co/docs/datasets`) was used once, in a standalone script, to download the public line-level handwriting database; nothing in the running system imports it. Finally, the weights of the writer-identification network were trained with PyTorch (`pytorch.org`) and then converted into this project's own weight format; the network that runs on the machine, and the from-scratch training program described in Section 3.4, contain no PyTorch code and do not import it.

MUST STILL BE COMPLETED BY ME

Two things to settle in declaration 6. First, confirm whether the deployed text recogniser's weights (`paper_cnn_bilstm_ctc_scratch_best.npz`) were trained from random initialisation by my own training program, or were warm-started from the PyTorch-trained checkpoint using the `--warm-start` option. I could not determine this from the saved file, and the answer changes what Section 3.4 and this declaration may claim. Second, check this list against the code submission and add anything I have missed.

7. Unaltered Project Proposal

I certify that the Project Proposal appearing as the Introduction section of the report is an unaltered copy of the approved Project Proposal.

Signature

With my signature below, I hereby certify that all statements, declarations, and information presented above are complete.

D.R. Braune Date

MUST STILL BE COMPLETED BY ME

Sign and date the report here. The report is rejected without this signature.

TABLE OF CONTENTS

Part 2. Project definition: approved Project Proposal

1. Project description
2. Technical challenges in this project
3. Functional analysis
4. System requirements and specifications
5. Field conditions
6. Student tasks

Part 3. Main Report

1. Literature study 1
    - 1.1. Background and context 1
        - 1.1.1. Handwriting recognition: problem, data and early solutions 1
        - 1.1.2. Sequence recognition with neural networks 1
        - 1.1.3. Image pre-processing and page segmentation 3
        - 1.1.4. Writer identification and style 4
        - 1.1.5. Handwriting synthesis and pen-path generation 4
        - 1.1.6. Robotic writing and linear motion 4
        - 1.1.7. Embedded deployment and shortcomings of the literature 5
    - 1.2. Application of the background to this project 5
        - 1.2.1. Reading and identification (R2, R4) 5
        - 1.2.2. Reproduction (R1, R3, R5, R6) 6
        - 1.2.3. Embedded execution and the contribution 6

2. Approach 8
    - 2.1. From functional analysis to concept 8
    - 2.2. Design alternatives and trade-offs 8
    - 2.3. Off-the-shelf components and own design 9
    - 2.4. Verification strategy and roadmap 9

3. Design and implementation 11
    - 3.1. Design summary 11
    - 3.2. System block diagram 11
        - 3.2.1. Hardware block diagram 14
        - 3.2.2. Software block diagram and systems-level description 15
    - 3.3. Page conditioning and line segmentation 16
        - 3.3.1. Motivation and design goals 16
        - 3.3.2. Stage 1 and 3: size normalisation, grey conversion and illumination correction 16
        - 3.3.3. Stage 2: paper localisation 18
        - 3.3.4. Stage 4: binarisation, red-ink exclusion and speckle removal 18

3.3.5. Stage 7: connected components and the glyph-height scale 19
3.3.6. Stage 5: skew estimation and rotation 19
3.3.7. Stage 6: removal of printed structure and faint marks 19
3.3.8. Stage 8a: text versus non-text 19
3.3.9. Stage 8b: grouping components into lines 20
3.3.10. Stage 9: pixel ownership and rendering of the line crops 20
3.3.11. Data sets, complexity, performance and limitations 21
3.4. Text recognition 22
3.4.1. Problem statement and design approach 22
3.4.2. Input pre-processing 22
3.4.3. Network architecture 23
3.4.4. Connectionist temporal classification 25
3.4.5. Training data and splits 25
3.4.6. Training procedure 26
3.4.7. From-scratch forward-and-backward implementation 27
3.4.8. Evaluation metrics 28
3.4.9. Inference implementation for the single-board computer 28
3.4.10. Design alternatives and justification 29
3.5. Writer identification 29
3.5.1. Problem statement and the need for two classifiers 29
3.5.2. Architecture 29
3.5.3. Stroke normalisation 30
3.5.4. Training data, protocol and leakage 31
3.5.5. From line decisions to a page decision 32
3.5.6. Design-stage evidence 32
3.5.7. Alternatives and justification 33
3.5.8. Inference on the single-board computer 33
3.6. Style extraction and glyph library 33
3.6.1. Forced alignment: where is each character? 34
3.6.2. From ink to ordered pen paths 35
3.6.3. Variants, quality gates and the cross-writer letter prior 35
3.6.4. Style statistics and permanent storage 36
3.6.5. Design alternatives and justification 36
3.7. Handwriting synthesis 36
3.7.1. Global quantities and glyph choice 37
3.7.2. Placement, joins and randomness 38
3.7.3. Best-of-*N* search with joint scoring 38
3.7.4. Design alternatives and justification 39
3.8. G-code generation 39
3.8.1. Coordinate frames, placement and ordering 40
3.8.2. Pen state, feed rates and program structure 40
3.8.3. Time model 40
3.8.4. Verification of the program 41
3.9. Gantry control and hardware 41
3.9.1. Components, and why each was chosen 41

3.9.2. Mechanical arrangement 42
3.9.3. Stepper driver and the coil-current setting 43
3.9.4. Switch inputs 43
3.9.5. Pen-lift motor and its switching stage 44
3.9.6. Supply and decoupling 45
3.9.7. Step generation, resolution and speed 45
3.9.8. Bresenham line interpolation 47
3.9.9. Direction-agnostic calibration 47
3.9.10. End-stop monitoring, emergency stop and pen mechanism 48
3.10. Software architecture and system integration 48
3.10.1. Modules and data flow 49
3.10.2. Start-up, caching, job storage and deployment 49
3.10.3. Timing chains of the two modes 50

4. Results 52
4.1. Summary of results achieved 52
4.2. Qualification tests 52
Qualification test 1: Reproduction fidelity of the synthesised handwriting (R1)
Objectives of the test or experiment 52
Equipment used 52
Test setup and experimental parameters 53
Steps followed in the test or experiment 53
Results or measurements 53
Observations 53
Statistical analysis 54
Qualification test 2: Classification of text and writer from real photographed pages (R2)
Objectives of the test or experiment 54
Equipment used 55
Test setup and experimental parameters 55
Steps followed in the test or experiment 55
Results or measurements 55
Observations 55
Statistical analysis 56
Qualification test 3: Reproduction time per character (R3)
Objectives of the test or experiment 56
Equipment used 57
Test setup and experimental parameters 57
Steps followed in the test or experiment 57
Results or measurements 57
Observations 57
Statistical analysis 58
Qualification test 4: Classification time (R4)
Objectives of the test or experiment 58
Equipment used 58
Test setup and experimental parameters 58
Steps followed in the test or experiment 59
Results or measurements 59

Observations 59
Statistical analysis 59
Qualification test 5: Absolute positional accuracy of the gantry (R5)
Objectives of the test or experiment 59
Equipment used 60
Test setup and experimental parameters 60
Steps followed in the test or experiment 60
Results or measurements 60
Observations 61
Statistical analysis 61
Qualification test 6: Multi-writer storage (R6)
Objectives of the test or experiment 61
Equipment used 61
Test setup and experimental parameters 61
Steps followed in the test or experiment 61
Results or measurements 61
Observations 61
Statistical analysis 61

5. Discussion 66
5.1. Critical evaluation of the design 66
5.1.1. Interpretation of results 66
5.1.2. Critical evaluation 68
5.1.3. Unsolved problems 68
5.1.4. Strong points of the design 70
5.1.5. Expected failure conditions 71
5.2. Considerations in the design 71
5.2.1. Ergonomics 72
5.2.2. Health and safety 72
5.2.3. Environmental impact 72
5.2.4. Social and legal impact 72
5.2.5. Ethics clearance 73

6. Conclusion 74
6.1. Summary of the work completed 74
6.2. Summary of observations and findings 74
6.3. Contribution 75
6.4. Suggestions for future work 76

7. References 77

Appendix

Appendix: Page conditioning and line segmentation (supporting material) 82
Appendix: Text recogniser (supporting material) 87
Appendix: Writer identification (supporting material) 89
Appendix: Relation of the implemented blocks to the proposal 89
Appendix: thresholding, skew and crop-rendering detail 89

Appendix: weak-ink recovery, size normalisation and evaluation metrics 90
Evaluation metrics 92
Appendix: optimiser settings and text-versus-diagram clustering 93
Appendix: component labelling, skew estimation and blur construction 94
Stage 7: connected components and the glyph-height scale 94
Stage 5: skew estimation and rotation 94
Appendix: label generation for the database pages 96
Style-profile schema and per-writer values 96
Constants of the synthesiser 97
G-code generator configuration 97
Routes of the server application 97
Supplementary figures and table for Sections 3.6 to 3.8 97
Hardware information and timing measurements still to be supplied 98
Design alternatives 98
Further figures and tables for Sections 3.8 to 3.10 98
Style calibration of the synthesiser 98
Extended qualification-test protocols 99
Metric definitions and detailed steps (qualification tests 1 and 5) 99
Human legibility panel (qualification test 1) 103
Style-feature definitions (qualification test 1) 103
Hardware overlay measurement sheet (qualification test 5) 103
Appendix: design-stage measurements of the text recogniser 103
Appendix: design-stage measurements of the synthesiser 103
Appendix: design decisions, alternatives and trade-offs 104
Appendix: supporting figures for qualification test 1 104
Appendix: supporting material for the text recogniser 104
Appendix: supporting material for writer identification 106
Appendix: supporting material for the style-profile builder 107
Appendix: supporting material for the synthesiser 108
Appendix: supporting material for the gantry and motion control 109
Appendix: supporting material for the software architecture 109

LIST OF ABBREVIATIONS

AdamW	Adam optimiser with decoupled weight decay
AMS	assignment management system (code submission)
API	application programming interface
ARM	processor architecture of the ODROID N2+ (Advanced RISC Machine)
AUC	area under the receiver operating characteristic curve
BiLSTM	bidirectional long short-term memory
BN	batch normalisation
CAD	computer-aided design
CER	character error rate
CNN	convolutional neural network
CRNN	convolutional recurrent neural network
CTC	connectionist temporal classification
EMNIST	Extended Modified National Institute of Standards and Technology (data set)
FC	field condition (FC1, FC2); also fully connected layer where stated
FU	functional unit
G-code	line-oriented machine-control instruction language
GPIO	general-purpose input/output
HSV	hue-saturation-value
HTR	handwritten text recognition
HTTP	hypertext transfer protocol
HWR	handwriting recognition
IAM	Institute of Computer Science and Applied Mathematics (handwriting database)
ID	identification (writer identification)
IoU	intersection over union
JPEG	Joint Photographic Experts Group (image format)
JSON	JavaScript Object Notation
LCS	longest common subsequence
LSTM	long short-term memory
MAC	multiply-accumulate operation
PC	personal computer
POPIA	Protection of Personal Information Act (South Africa)
PSU	power supply unit
R1–R6	requirements 1 to 6 of the project proposal
ReLU	rectified linear unit
RMSprop	root-mean-square propagation (optimiser)
RNN	recurrent neural network
SBC	single-board computer
STEP/DIR	step and direction control signals of a stepper-motor driver
USB	Universal Serial Bus
WER	word error rate

NOTATION

x_{h}	x-height of the handwriting (height of lower-case letters without ascenders or descenders)
x	number of characters of a sentence or page (in the time requirements R3 and R4)
N	number of candidate syntheses tried in the best-of-N search (“tries”)
$A_{\mathrm{c}}, A_{\mathrm{w}}$	character accuracy and writer accuracy

Part 2. Project definition: approved Project Proposal

This section contains the problem identification in the form of the complete approved Project Proposal, unaltered from the final approved version that appears on the AMS.

MUST STILL BE COMPLETED BY ME

The approved Project Proposal included on the following pages is a PDF form whose fields have not been flattened, so none of its filled-in content is rendered: the project description, the technical challenges, the functional analysis, the requirement and specification table, the field conditions and the student tasks all appear blank, and so do the title, student number, study leader and the Feedback block. The fields are present in the file, but `includepdf` does not draw form-field appearances. Appendix 4 requires Part 2 to be the complete approved proposal, and an incomplete submission fails the project, so this must be fixed before submission. To fix it: open `2_Project_Definition/project_proposal.pdf` in a PDF reader, use "Print to PDF" (which flattens the fields into the page content), and replace that file with the printed copy. Then rebuild and check that page 1 shows the title, the student number, the study leader and the Feedback block, and that the later pages show the answers rather than empty boxes.

For use by the Project lecturer	Approved		Revision required	

Feedback

For use by the Project module lecturer only

To be completed by the student

PROJECT PROPOSAL 2026

Project no		Revision no	

Title	Surname	Initials	Student no	Study leader (title, initials, surname)

Project title (this is the title on the project concept note)

Language editor details	Language editor signature
Student declaration I understand what plagiarism is and that I have to complete my project on my own.	**Study leader declaration** This is a clear and unambiguous description of what is required in this project. Approved for submission (Yes/No)
Student signature	Study leader signature and date

1. Project description. What is the problem to be solved with your project? Describe in non-technical language what your project is about. What does your system have to do? Refer explictly to Figure 1 that you inserted on the right.

Figure 1. Insert an image below that captures the idea with your project. Explain in the text on the left.

2. Technical challenges in this project

Describe the technical challenges that are *beyond* those encountered up to the end of third year and in other final year modules.

2.1 Primary *design* challenges

What is the intended core technical challenge in the project? Which aspect or aspects of the design of the system do you expect to be the most challenging?

2.2 Primary *implementation* challenges

Which aspect or aspects of the implementation do you expect to be the most challenging?

All processing will be done on an embedded platform, and not on a PC platform. Confirmed by having ticked this box.

Library functions will be used for hardware interfacing only. Confirmed by having ticked this box.

3. Functional analysis

3.1 Functional description

Describe the design in terms of system functions as shown on the functional block diagram in section 3.2. This description should be in *narrative format*. **DO NOT** use a bullet list.

3.2 Functional block diagram (this should not be a flow diagram)

4. System requirements and specifications

These are the core requirements of the system or product (the mission-critical requirements) in table format **IN ORDER OF IMPORTANCE**. Requirement 1 is the most fundamental requirement.

	Requirement 1: the fundamental functional and performance requirement of your project	**Requirement 2** (Second in order of importance)	**Requirement 3** (Third in order of importance)
1. Core mission requirements of the system or product. Focus on requirements that are **core** to solving the engineering problem. These will reflect the solution to the problem.			
2. What is the target specification (this must be in *measurable* terms) to be met in order to achieve the requirement in 1. above?			
3. Explain where this specification originated : i.e., **explain** the value that you selected. I.e., *how* will meeting the specification given in point 2 above *solve the problem*?			
4. How will you demonstrate at the examination that this requirement and specification (points 1 and 2 above) have been met? Be explicit about how you will ***prove*** these were met.			
5. Your own design contribution: what are the aspects that *you will design and implement yourself* to meet the requirement in point 2? If none, *remove this requirement.*			
6. What are the aspects to be taken off the shelf to meet this requirement? If none, indicate "none". Clearly specify for what tasks **library functions** will be used (if relevant to the project).			

System requirements and specifications page 2

	Requirement 4 (Fourth in order of importance)	**Requirement 5**	**Requirement 6**
1. Core mission requirements of the system or product. Focus on requirements that are core to solving the engineering problem. These will reflect the solution to the problem.			
2. What is the target specification (this must be in *measurable* terms) to be met in order to achieve the requirement in 1. above?			
3. Explain where this specification originated: i.e., explain the value that you selected. I.e., *how* will meeting the specification given in point 2 above solve the problem?			
4. How will you demonstrate at the examination that this requirement and specification (points 1 and 2 above) have been met? Be explicit about how you will *prove* these were met.			
5. Your own design contribution: what are the aspects that *you will design and implement yourself* to meet the requirement in point 2? If none, *remove this requirement.*			
6. What are the aspects to be taken off the shelf to meet this requirement? If none, indicate "none". Clearly specify for what tasks **library functions** will be used (if relevant to the project).			

System requirements and specifications page 3

	Requirement 7 (Seventh in the order of importance)	**Requirement 8**	**Requirement 9**
1. Core mission requirements of the system or product. Focus on requirements that are core to solving the engineering problem. These will reflect the solution to the problem.			
2. What is the target specification (this must be in *measurable* terms) to be met in order to achieve the requirement in 1. above?			
3. **Explain where this specification originated**: i.e., explain the value that you selected. I.e., *how* will meeting the specification given in point 2 above solve the problem?			
4. **How will you demonstrate at the examination** that this requirement and specification (points 1 and 2 above) have been met? Be explicit about how you will *prove* these were met.			
5. Your own design contribution: what are the aspects that *you will design and implement yourself* to meet the requirement in point 2? If none, *remove this requirement.*			
6. What are the aspects to be taken off the shelf to meet this requirement? If none, indicate "none". Clearly specify for what tasks l**ibrary functions** will be used (if relevant to the project).			

5. Real world field conditions

These are the ACTUAL, REAL WORLD CONDITIONS under which your project has to work *and will be demonstrated.*

	Real world field condition 1 (required to include)	**Real world field condition 2** (optional to include)	**Real world field condition 3** (optional to include)
Field condition requirement. In which real world conditions does the system have to operate?E.g., "lab conditions" or "outside in uncontrolled conditions". Describe the one, two or three most important field conditions.			

6. Student tasks

6.1 Design and implementation tasks of the student

List your primary design and implementation tasks in bullet list format (5-10 bullets). These are *NOT* product requirements, but *YOUR* tasks.

6.2 New knowledge to be acquired by the student

Describe what the theoretical foundation to the project is, and which *new* knowledge you will acquire (*beyond* that covered in any other undergraduate modules).

7. References

Format your reference list either in IEEE reference format or in APA reference format. Check what your study leader prefers and use consistently.

Part 3. Main Report

1. Literature study

This section summarises the literature required to complete the project and extends the first-semester literature study. Section 1.1 follows the signal path of the system, from a photograph of a handwritten page, through recognition and writer identification, to the synthesis of new handwriting and its execution by a two-axis robot. Section 1.2 states which technique was adopted for each subsystem, why, and which requirement (R1 to R6) it serves.

1.1 Background and context

A handwriting-reproduction robot has to solve three problems usually studied separately: *reading* (converting an image of handwriting into text), *writer identification* (deciding who wrote it) and *writing* (producing new text in a person's style and drawing it with a machine). Each is treated in turn, then combining them in one embedded system.

1.1.1 Handwriting recognition: problem, data and early solutions

Handwriting recognition (HWR) or handwritten text recognition (HTR) converts handwriting into machine-readable text. Plamondon and Srihari [1] distinguish *online* recognition, where the pen position is recorded over time so that stroke order and pen lifts are known, from *offline* recognition, where only an image of the finished writing exists. This project reads a photographed page, so it is an offline problem, the harder of the two [1]. The distinction also matters for writing, because most pen-path generators are trained on online data (Section 1.1.5).

The data set fixes the *unit of recognition*. The Extended Modified National Institute of Standards and Technology (EMNIST) data set holds roughly 814 000 isolated 28×28 characters [2]; models trained on it classify characters already cut out but cannot read connected writing. The Institute of Computer Science and Applied Mathematics (IAM) database of Marti and Bunke [3] holds scanned pages of English sentences from many writers (5 685 sentences, 115 320 words, 8-bit greyscale at 300 dots per inch) labelled per line, suiting line recognisers and writer experiments.

Early offline systems followed a *segment-then-classify* paradigm. LeCun *et al.* [4] showed that a convolutional neural network (CNN) can classify isolated characters directly from pixels, and their LeNet-5 became the baseline for later character classifiers, refined by many first-semester sources [2, 5, 6, 7]. The weakness is segmentation: in cursive writing letters touch, a wrong cut yields a fragment no classifier can label, and the classifier never sees the neighbouring letters, so it cannot repair the error. System accuracy is therefore bounded by the segmenter.

1.1.2 Sequence recognition with neural networks

A *convolutional layer* slides a kernel of learned weights, typically 3×3 pixels, over the image and outputs a weighted sum followed by a non-linearity; because the weights are shared, a stroke at a given angle is detected wherever it occurs [4]. *Max-pooling* keeps the largest value in each small block, halving the resolution, and *batch normalisation* [8] rescales activations so that deeper networks can be trained. Back-propagation gives the gradient of the loss with respect to every weight, and an optimiser such as Adam [9] updates them against it.

A CNN gives a fixed-size output, whereas a line of text has an unknown number of characters. Recurrent networks process a sequence step by step while carrying a hidden state; the long short-term memory (LSTM) cell [10] adds input, forget and output gates that decide what to store, discard and expose, so that information survives many steps. A *bidirectional* LSTM (BiLSTM) runs one LSTM in each direction, so each position uses the writing on both sides of it and a stroke that could be a “u” or an “n” is resolved.

The remaining difficulty is the training signal. The network turns a line image into T time steps, but the label is only the transcription, so the alignment between steps and characters is unknown. Connectionist temporal classification (CTC) [11] solves this. At every step the network outputs a distribution over the characters plus a “blank” meaning “no new character”. A path π assigns one symbol to each step, and a collapse function B merges consecutive repeats then deletes blanks, so “–hh-e-ll-ll-o” becomes “hello” (the blank between the two groups of “l” allows a genuine double letter). The probability of a target text Y given the image X is the sum over all paths that collapse to it,

$$p(Y \mid X) = \sum_{\pi : B(\pi) = Y} \prod_{t=1}^{T} p_t(\pi_t \mid X), \tag{1}$$

where $p_t(\pi_t \mid X)$ is the probability the network assigns at step t to symbol π_t. The loss is $-\ln p(Y \mid X)$, and although the paths are very many, the sum in Equation 1 and its gradient are computed by dynamic programming [11]. Thus *no character segmentation and no alignment labels are needed*. At recognition time the most probable symbol at every step is taken and collapsed (greedy decoding), or a lexicon or language model guides a search. Viterbi’s algorithm [12] instead finds the most probable alignment between a *known* transcription and the network output (forced alignment), used in Section 1.1.5.

Figure 1 shows these two operations, on which the rest of the project depends: together they provide both the reading of a page and the raw material from which a writer’s letter shapes are harvested.

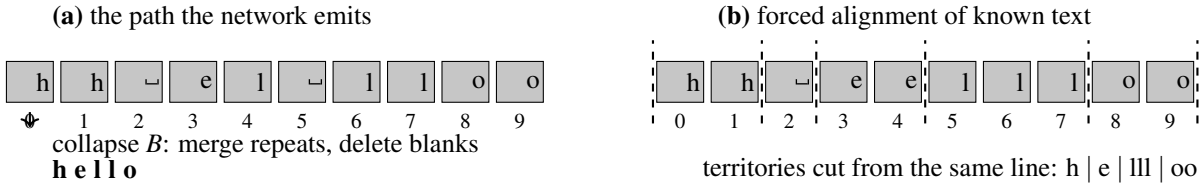


Figure 1. The two CTC operations used in this project. (a) Collapsing the emitted path gives the transcription, so a line can be trained from its text alone with no character boundaries. (b) Given the text, forced alignment finds the most probable block of time steps for each character; the dashed lines are the resulting territories, from which individual letter shapes are cut. Blank symbols are shaded.

Graves *et al.* [13] combined BiLSTM and CTC and reported word accuracies of 74.1 % (offline) and 79.7 % (online) on IAM-derived data. Shi *et al.* [14] replaced the raw-pixel input by CNN features, naming the arrangement the convolutional recurrent neural network (CRNN): a CNN turns the image into feature columns, a bidirectional recurrent network adds context, and CTC supplies the loss. Puigcerver [15] showed on IAM that a CNN followed by BiLSTM layers matches costlier multidimensional recurrence, and the first-semester sources [16, 17, 18] agree on CNN–RNN hybrids.

Reported accuracies. Figures are comparable only for the same metric, decoding and split. Quality is expressed as the character error rate (CER), the Levenshtein distance [19] between the recognised text and the reference divided by the reference length, or as the word error rate (WER); this report uses character accuracy = 1 − CER. Kizilirmak and Yanikoglu [20], whose CNN–BiLSTM–CTC network is closest to this one, report a best CER of 3.59 % and WER of 9.44 % on IAM, but only with lexicon-constrained decoding and test-time augmentation. Greedy decoding without a lexicon is therefore worse, and word error rates run several times the character error rate because errors cluster inside words. Accuracies above 95 % in the first-semester sources [6, 5, 2] concern isolated characters on clean data, a far easier task. Benchmark splits may also put the same writer, or neighbouring lines of one page, in both partitions, and published work is mostly evaluated on scanner-quality images, whereas an office photograph has uneven lighting, perspective, shadows and neighbouring objects. Pre-processing therefore cannot be omitted.

1.1.3 Image pre-processing and page segmentation

A photographed page must first be converted to clean, separated line images; the first-semester sources [2, 18] list normalisation, binarisation and line separation by bounding boxes around ink regions as standard.

A photograph records the paper's reflectance multiplied by the lighting, so a lamp gradient or shadow changes the grey value of the paper itself. *Binarisation* decides for each pixel whether it is ink or paper. Otsu's global method [21] picks the threshold t^* maximising the between-class variance of the two groups of pixels,

$$t^* = \arg\max_t \; \omega_B(t)\,\omega_F(t)\left[\mu_B(t) - \mu_F(t)\right]^2, \tag{2}$$

In Equation 2, $\omega_B(t)$ and $\omega_F(t)$ are the fractions of pixels at or below, and above, t, with mean grey values $\mu_B(t)$ and $\mu_F(t)$. It needs no tuning but fails under uneven lighting, because one threshold then drops strokes in dim areas or turns shadows into ink. *Adaptive* methods use a threshold per pixel: Sauvola and Pietikäinen [22] derive it from the local mean and standard deviation, and Bradley and Roth [23] compare each pixel with the local mean from an integral image. Dividing the image by a heavily blurred copy of itself (flat-field correction) turns a multiplicative lighting gradient into a nearly constant white paper level. Large local means are cheap with the summed-area table of Crow [24], which stores at each position the sum of all pixels above and to the left, so any rectangle sum needs four look-ups; three repeated box averages also approximate a Gaussian blur.

Ink pixels are grouped into *connected components* by the two-pass labelling of Rosenfeld and Pfaltz [25]. Component heights give a typical character height as a scale reference, so thresholds in character heights do not depend on camera resolution or handwriting size. Skew is found by a projection-profile search: the image is rotated through trial angles and the angle whose row-wise ink histogram is most sharply peaked is taken as the skew. Lines are formed from gaps in the row profile, which assumes straight horizontal lines, or by chaining neighbouring components from left to right, which tolerates drifting baselines and diagrams. Printed ruling lines glue letters together and must be removed without deleting the pen strokes crossing them. These steps are mature but mostly described for scanned documents, rarely for a camera, desk background, ruled paper and diagrams together.

1.1.4 Writer identification and style

Writer identification asks which member of a known set wrote a sample. The proposal grounded its target in signature classification: Agduk and Aydemir [26] compared transfer-learning CNNs for classifying signatures by person and gender. Signatures are short, practised and not free text, whereas this system must identify the writer of running text while reading it. Bulacu and Schomaker's statistical approach [27] is *text independent*: it compares histograms of contour direction and curvature (texture level) and of characteristic letter shapes (allograph level). Style is also described by slant, x-height (the height of the lower-case body), stroke width, spacing and joined-letter frequency. A CNN can instead be trained directly on handwriting images; He and Schomaker's FragNet [28] classifies small fragments and pools the evidence, since style is present in every part of a text.

Two cautions affect the project. A classifier trained on real ink can rely on cues a machine cannot reproduce: stroke thickness and ink density depend on the writer's pen and pen pressure, whereas a robot draws with one pen at a constant width. And most of this work uses hundreds of writers, whereas this closed set of thirteen writers has about one and a half thousand text lines, and test partitions that share pages with training data inflate accuracy in such small experiments.

1.1.5 Handwriting synthesis and pen-path generation

Neural sequence generators are trained on recorded pen trajectories. Graves [29] used an LSTM conditioned on the text through a soft attention window, its output at each step a mixture density over the next pen offset and a pen-up probability; it produces convincing cursive but needs hundreds to thousands of recorded lines from the writer, which photographs of a few pages cannot supply. Image generators such as the generative adversarial network of Kang *et al.* [30] need less data per writer after pre-training on a large corpus, but produce rasters that must still be converted to pen paths and give no control of slant, size or spacing. *Exemplar* (glyph-based) methods are the alternative: Haines *et al.* [31] synthesise an author's hand from an annotated sample by assembling extracted glyphs with learned spacing, thickness and pressure. They need only a few pages per writer, have interpretable parameters and stay faithful to the real letterforms. Glyphs can be extracted without hand-drawn boundaries by forced alignment of the known transcription with a trained recogniser's output (Section 1.1.2).

Turning a glyph into a pen path needs classical steps. *Thinning* reduces a stroke to a one-pixel centre line without breaking connectivity; the parallel algorithm of Zhang and Suen [32] deletes boundary pixels in two alternating sub-iterations according to their eight neighbours. The centre line is traced into ordered point sequences, which the Douglas–Peucker algorithm [33] simplifies by keeping the end points and recursively keeping the point farthest from the chord whenever its distance exceeds a tolerance ε; the first-semester report noted it is not guaranteed optimal.

1.1.6 Robotic writing and linear motion

The first-semester study identified three robot topologies [34]. A *Cartesian* (gantry) system moves along two perpendicular linear axes: it is mechanically simple, its kinematics are the identity and it covers a flat page naturally. *Serial-chain* arms reach a larger volume but need

inverse kinematics and joint errors add up at the pen. *Parallel* robots are stiff and precise but have a small workspace and complex kinematics. Belt-drive, rack-and-pinion and ball-screw actuators are used in handwriting systems for their durability, reliability and low cost [35].

Most such systems use stepper motors, which turn by a fixed angle for every pulse. A driver converts a logic-level pulse (STEP) and direction level (DIR) into winding currents, and microstepping gives finer positions; Acarnley [36] treats open-loop stepping, torque–speed limits and controlled acceleration, such as a trapezoidal velocity profile, to avoid lost steps. Because position follows from counting pulses, error does not accumulate if commanded positions become integer step counts from a known reference. To draw a straight line the two axes must be interleaved; Bresenham's algorithm [37], developed for digital plotters, does this in integer arithmetic: the faster axis steps every cycle and an error accumulator decides when the slower axis also steps, so the deviation from the ideal line never exceeds one step.

Published robots show what accuracy to expect. The proposal used Teikari's analysis [34] to set about four seconds per character for zero to one per cent error, the 0.3 mm absolute gantry error of Huang and Xiong [38] as the basis of the 0.5 mm limit of R5, and the roughly 85 % accuracy derived from Babushkin *et al.* [39] for acceptable character accuracy (R1). None shows how a robot would reproduce an *identified* person's style from a photograph of that person's page.

1.1.7 Embedded deployment and shortcomings of the literature

Executing all processing on a single-board computer (SBC) makes every algorithm's cost matter. Sze *et al.* [40] stress that multiply–accumulate operations and the memory for weights and activations determine execution time and energy on a constrained processor, and that convolutions usually dominate. A line recogniser with a convolutional front end costs of the order of 10^9 operations per image, whereas the classical image-processing steps need a few operations per pixel. Most research also relies on large off-the-shelf frameworks, so the algorithms themselves are never exposed [2].

Four shortcomings follow. First, recognition, writer identification, synthesis and robotics are studied in isolation, with no complete system from a photographed page to a drawn one. Second, embedded deployment is rarely demonstrated, and then through frameworks. Third, reported accuracies depend on benchmark conditions (lexicon, scanner images, shared writers or pages) unlike a photographed page. Fourth, the generation literature needs recorded pen trajectories or large pre-training and evaluates output as an image, whereas this project must evaluate output a machine can draw with a constant-width pen. These define the project's contribution.

1.2 Application of the background to this project

This section records how each subsystem applied what was learnt.

1.2.1 Reading and identification (R2, R4)

Cursive writing cannot be segmented reliably, a segmentation error cannot be repaired by the classifier, and the labels are line transcriptions, so a line-level recogniser trained with CTC [11] was adopted and FU 2.2 (character separation) was realised as separation into *lines*.

The architecture follows the CNN–BiLSTM–CTC pattern [14, 15, 20] with about 3.36 million parameters, and decoding is greedy because lexicon decoding needs corpora the project does not have (Sections 3.4 and 4). For writer identification a learned CNN on text lines was chosen over hand-crafted statistics [27] or fragment pooling [28], since the recogniser's features could be reused and accuracy measured directly. Following the caution of Section 1.1.4, two classifiers of the same architecture were built: one sees real photographed ink, the other lines thinned and redrawn at a constant width proportional to the x-height, which removes thickness and pressure cues and matches what the gantry can do. The second was motivated by the first: an ink-based classifier that was almost perfect on real handwriting identified only about a quarter of the machine's drawn lines (Section 3.5). Both are closed-set decisions among the stored writers; they address R2 and the storage requirement of R6 but cannot report an unknown writer (Section 5.1.5).

1.2.2 Reproduction (R1, R3, R5, R6)

Neural trajectory generators need pen recordings that the project does not have, and image generators lack parametric control and produce rasters, so an exemplar approach after Haines *et al.* [31] was adopted. For each writer, glyphs are harvested from the pages: the recogniser aligns the known transcription with the line (Figure 1b), the ink is cut into letter territories, thinned [32], traced into strokes, simplified [33], normalised to the x-height and de-slanted; the glyphs and measured style statistics form a *style profile* stored as a data file. Synthesis chooses glyphs, shears them to the writer's slant, spaces them by the writer's statistics and adds small random variation; because each run is random, several candidates are generated and the best kept by a combined score of legibility and writer-classifier confidence. The measured outcome, including that the scoring networks also perform the selection and so cannot be an independent verdict, is reported in Sections 3.7 and 4.

A Cartesian gantry with stepper motors was adopted, since position follows from step counts, the only practical route to 0.5 mm without position sensors. Pen paths in millimetres become G-code, which a program converts to absolute integer step counts, interleaving the axes with Bresenham's algorithm [37] and reading the limit switches before every step; the step resolution is 12.5 μm against the 0.5 mm limit of R5 (Section 3.9). Resolution is not accuracy: belt stretch, backlash, home-switch repeatability and pen compliance determine the true error, and none was measured, so R5 remains unassessed in Section 4.

1.2.3 Embedded execution and the contribution

Following Sze *et al.* [40], the networks' cost was analysed: about 4.2×10^9 multiply–accumulate operations and 13.4 MB of weights per recogniser pass over a line, 91 % of the arithmetic being convolution. Inference of both networks was therefore written from first principles in Python as matrix multiplications and element-wise operations, without a neural-network framework; it agrees with the training implementation to about 10^{-5} in log-probability, re-verified on the final trained weights (Section 3.4). Recognition measured 8.86 ms per character and 353 ms per line, meeting the 25 ms allowance of R4; page segmentation, however, measured 35 to 57 s against the roughly 2 s the same requirement leaves for capture and conditioning, so R4 fails on segmentation rather than on the network (Section 4). The contribution following from Section 1.1.7 is a complete pipeline from page photograph to pen strokes, implemented from

first principles in one language.

2. Approach

This section explains how the problem of the approved proposal was approached: how the functional analysis became a first concept, which alternatives were weighed, and why Section 3 is ordered as it is. Mathematics and parameter values are deferred to Section 3.

2.1 From functional analysis to concept

The proposal defined three modes (training, classification and reproduction) and four functional units: FU 1, page scanning; FU 2, the single-board computer (SBC), with 2.1 image conditioning, 2.2 character separation, 2.3 classification, 2.4 back-propagation, 2.5 characters to trajectory maps, 2.6 storage, 2.7 loading and 2.8 stepper-pulse generation; FU 3, the gantry (3.1 voltage shifter, 3.2 X/Y motor control, 3.3 pen and paper adjustment, 3.4 pen movement); and FU 4, the power supply. Requirements R1 to R6 and two field conditions, FC1 (a well-lit room, 400 to 1000 lux) and FC2 (a stable, level, vibration-free surface), were attached to this structure.

The first concept kept this structure and made one decision per block. Figure 2 shows it with the three modes, and Table 3 gives each unit's realisation and where it departs from the proposal; the departures are justified in Section 2.2.

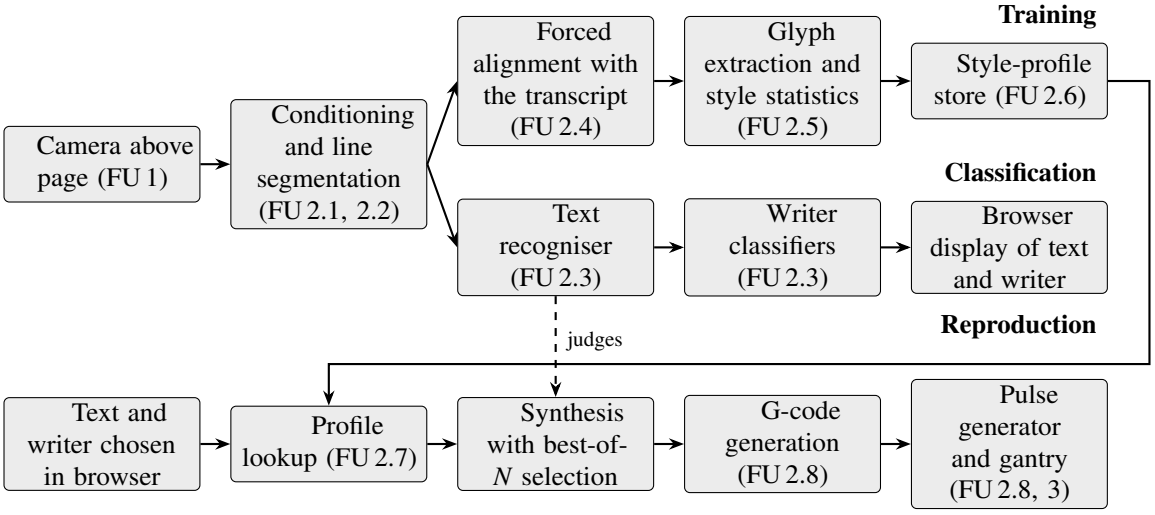


Figure 2. Block diagram of the chosen concept. Solid arrows show the flow of data; the dashed arrow shows that the recogniser and writer classifier also judge the candidates in the synthesis block. Functional-unit numbers of the proposal are in brackets. Training is performed offline and is not part of the browser interface.

2.2 Design alternatives and trade-offs

The software and hardware design decisions that follow from these criteria, each with the alternatives that were considered and the reason the preferred option was chosen, are listed in Tables 42 and 43 of the Appendix. Each of those decisions is argued in full in the "Design alternatives and justification" subsection of the corresponding part of Section 3.

Unit	Realisation in the final concept	Departure from the proposal
FU 1, 2.1	Camera on a fixed rig over an A4 page; photograph conditioned for scale, lighting, binarisation, skew and ruled lines.	None; conditioning is far more extensive than anticipated.
FU 2.2	Separation of the page into *text lines*.	Lines instead of characters (Section 2.2).
FU 2.3, 2.4	Line recogniser for text, two classifiers for the writer; gradient-based training offline.	Text and writer by separate networks; training not in the interface. **[MUST STILL BE COMPLETED BY ME: state on which computer the networks were trained and the profiles built; the proposal requires embedded-only processing.]**
FU 2.5–2.7	A style profile per writer (pen-path glyphs and style statistics) stored as a data file and loaded at start-up.	Stored per glyph variant with statistics, not one map per character.
FU 2.8, 3.1, 3.2	G-code, then step/direction pulses from the SBC to two stepper-driver modules; limit switches bound the travel.	Two stages; limit switches added as a safety function.
FU 3.3, 3.4	Pen and paper holders; pen lifted by a one-way toggle gear motor with a position switch.	Pen lift not controlled through the stepper driver. **[MUST STILL BE COMPLETED BY ME: reason for this change.]**
FU 4	Power supply.	**[MUST STILL BE COMPLETED BY ME: supply voltages, current ratings and distribution.]**

Table 3. Translation of the functional units of the proposal into the first concept.

Each major decision was weighed against criteria from the requirements: accuracy (R1, R2), processing time on the SBC (R3, R4), the data available for thirteen writers (R6), first-principles implementation, and mechanical and electrical simplicity and safety. The arguments are mostly qualitative; design-stage measurements are quoted with their caveats. Table 42 covers the processing chain and Table 43 the physical system.

Two consequences follow. Training from line transcriptions with connectionist temporal classification [11] costs a larger network and a heavy dependence on robust *line* segmentation. The glyph library's weak case, recognised from the outset, is the heavily cursive hand, whose letter links are lost when letters are cut apart.

2.3 Off-the-shelf components and own design

Consistent with rows 5 and 6 of the proposal's requirement tables, the following were bought: the camera and its driver, the SBC (an ODROID N2+), the stepper motors and driver modules, the limit switches, the pen, standard gantry parts (belts or screws, bearings, fasteners), storage and the power supply. Camera and general-purpose input/output access use standard system drivers for hardware interfacing only. The student designed the image conditioning and page segmentation, the recogniser and writer classifiers with their training, the style-profile builder, synthesiser and candidate selection, the conversion of pen paths to G-code and of G-code to step pulses with safety checks, the gantry calibration, the browser interface, and the gantry's mechanical parts drawn in computer-aided design software. **[MUST STILL BE COMPLETED BY ME: list of the mechanical parts designed and manufactured, as opposed to purchased.]**

2.4 Verification strategy and roadmap

The design was verified against two kinds of evidence. The first is performance on data *held out* from training: a public line-level partition for the recogniser, and one whole page per

public-data writer for the writer classifiers. The partition for the two personal writers is weaker, since held-out lines come from the training pages, and this is stated wherever the figures are quoted. The second is the proposal's *measurable specifications*: character and style accuracy (R1), text and writer accuracy (R2), time per character (R3, R4), positional error (R5) and stored styles (R6); Section 4 reports what was and was not measured. Two limitations were clear at the outset. Synthesis was tuned and judged by the same two networks, so its figures measure agreement with those judges; the recogniser's accuracy on the same writers' real handwriting is the non-circular reference. R3, R4 and R5 depend on the built hardware and the SBC, so only timing and positional measurements on it can give them.

Section 3 follows the path of the data, because each stage uses the output of the preceding one: the architecture (Section 3.2); page conditioning and line segmentation (Section 3.3; R2, R4), where FC1 and FC2 act and on which every later stage rests; the text recogniser (Section 3.4; R2, R4); the writer classifier (Section 3.5; R2, R6), which starts from its features; the style profile (Section 3.6) and synthesis with candidate selection (Section 3.7; R1, R6); G-code generation (Section 3.8) and the gantry with its control (Section 3.9; R3, R5); and integration with the interface and timing chains (Section 3.10; R3, R4). R1 comes after the networks because it rests on everything upstream.

3. Design and implementation

3.1 Design summary

This section summarises the project design tasks and how they were implemented (see Tables 4 and 5). The rows follow the deliverables of line 5 of the table of Section 4 of the approved project proposal (requirements R1 to R6, “own design contribution”) and the nine design and implementation tasks of Section 6.1 of that proposal. The summary is given as two tables only so that neither of them runs across a page break; together they are the single design summary that the report requires, and the column headings are the same in both. A task is marked *Completed* only when the deliverable itself works; where any part of it is missing, unbuilt or unmeasured it is marked *Incomplete* and the second column says exactly what is missing. R1 is reproduction of text in the style of any writer of the training set, R2 decoding of the text and identification of the writer, R3 reproduction speed, R4 classification time, R5 gantry positional accuracy and R6 at least ten stored writing styles. Only work designed and implemented by the student is claimed; the physical parts that were bought as finished items (camera, single-board computer, stepper motors and driver modules, pen, end-stop switches, power supply, standard mechanical parts) carry no design credit. All image-processing, machine-learning, trajectory and motion-control algorithms were written by the student in Python, and the only external functions used interface with the camera, the file system and the network.

Two deviations from the proposal shape several rows. First, the proposal planned an explicit separation of the page into characters (functional unit (FU) 2.2) before classification; the implemented design separates the page into *text lines* only and reads each line with a sequence recogniser, because the characters of connected handwriting have no reliable boundaries and a segmentation error would pass directly into a wrong reading (Sections 3.3 and 3.4). Second, the proposal placed training and the building of trajectory maps on the single-board computer (FU 2.4 to 2.6); in the implemented system the networks and the per-writer style profiles are produced by offline programs on a general-purpose computer, and only inference, synthesis and motion control run on the single-board computer, so the training mode is not offered by the user interface. **[MUST STILL BE COMPLETED BY ME: state the computer used for network training (the project notes mention a laptop processor and a cloud notebook) and confirm that the single-board computer was not used for training.]**

3.2 System block diagram

This subsection gives a block-diagram-level description of the whole implemented design. Two diagrams are used: Figure 3 shows the hardware blocks and the way in which they are connected to the single-board computer (SBC), and Figure 4 shows the software data flow of the three modes of operation (reproduction, classification and offline training). Both diagrams are related to the functional block diagram of the approved proposal, which divided the system into the page-scanning set-up (FU 1), the single-board computer (FU 2), the text-reproduction gantry (FU 3) and the power supply (FU 4); the differences are explained in Table 23.

Deliverable or task	Implementation	Completion of deliverable or task, and section in the report
Research of network types, structure, layers and feature extraction (task 6.1.1)	A literature study of convolutional, recurrent and sequence-to-sequence recognisers, of writer-identification methods and of the alternatives to each, used to choose the methods of Sections 3.4 and 3.5.	Completed. Section 1.
Research of conditioning, noise removal, separation and trajectory mapping (task 6.1.2)	A literature study of thresholding, illumination correction, line and character separation, thinning and exemplar-based handwriting synthesis, used to choose the methods of Sections 3.3 and 3.6.	Completed. Section 1.
Image conditioning and noise removal (FU 2.1; R2, R4)	Designed and implemented from first principles using only array arithmetic: size normalisation, page localisation, flat-field illumination correction with summed-area tables, local adaptive thresholding, skew correction and removal of printed rules, faint print and neighbouring-page ink.	Completed. Section 3.3.
Separation into characters (FU 2.2; R2, R4)	Implemented as separation into text lines and diagram regions rather than into characters (connected components, glyph-scale estimation, chaining, baseline merging, text versus non-text scoring, one deskewed crop per line). Character territories are derived only offline, by forced alignment, when style profiles are built. The reason for the change from the proposal is given above and in Section 3.4.	Completed. Sections 3.3, 3.4.
Optimised and efficient character separation (R4; task 6.1.3)	The segmentation was profiled and its two dominant routines, component labelling and the box sums, were rewritten in C by the student at 84 to 88% faster each, 25 to 32% faster per page. The C routines are not called by the deployed program, and segmentation still exceeds the R4 allowance by over an order of magnitude.	Incomplete. Sections 3.3, 4.
Neural network that decodes the text (FU 2.3; R2)	Convolutional and bidirectional recurrent network with connectionist temporal classification (CTC) output, 3 358 763 trainable parameters. The layer pattern follows a published design [20]; the student chose the input geometry, data pipeline, label cleaning, augmentation and staged training, and wrote the from-scratch module (forward and backward passes, optimisers, no machine-learning library) used by training, validation and deployment alike.	Completed. Section 3.4; measured accuracy 98.61% character and 94.21% word on 669 real line crops (Section 4).
Neural network that identifies the writer (FU 2.3; R2)	Two thirteen-class convolutional classifiers sharing the recogniser's backbone and layer module: an ink-based one for photographs, which reached 99.78% over 1358 real lines, and one whose input is re-drawn at a constant pen width by the student's stroke normalisation, with page-level aggregation.	Completed. Section 3.5; measured accuracy 93.67% for the normalised classifier after a training fault was corrected (Section 4).
Training on datasets and choice of layer numbers (FU 2.4; task 6.1.4)	Training programs for the recogniser and both classifiers on a public database, a public line-level set and the photographed personal pages, all built on the from-scratch module of Section 3.4.7, whose every layer gradient was checked against numerical differentiation in 78 configurations. A sweep over widths, depths and learning rates was programmed but no result was kept, so the layer numbers are the published design's and are not justified by this project's own measurement. Training is offline.	Incomplete. Sections 3.4, 3.5.
Trajectory maps and permanent storage (FU 2.5 to 2.7; R1, R6)	Letter shapes are harvested by forced alignment with the recogniser, thinned to centre-line pen paths, normalised and stored as one JavaScript Object Notation (JSON) file per writer with measured style statistics; thirteen styles exist, for eight database writers and five personal writers. The builder is an offline program.	Completed. Section 3.6.
Pen trajectory mapping and G-code (FU 2.5 to 2.8; R1, R3)	A synthesiser assembles the writer's own letter paths into lines (slant, spacing, joins, wobble) with a best-of-N search judged by the recogniser and the writer classifier; a generator turns the trajectory into G-code, the numerical-control command language of plotters, with placement, stroke ordering and pen-state tracking, and a simulator replays it.	Completed. Sections 3.7, 3.8.
Optimisation of pen movement (R3)	Greedy nearest-endpoint stroke ordering reduces pen-up travel. The saving it achieves was not measured.	Completed. Section 3.8.

Table 4. Design summary, part 1 of 2: the research, image-processing, network and trajectory tasks.

Deliverable or task	Implementation	Completion of deliverable or task, and section in the report
Capture rig and deployment on the single-board computer (FU 1; R2, R4; task 6.1.5)	A generic USB camera in a printed housing that clamps the page flat, read with a live preview and a five-frame warm-up; the recognition, synthesis and motion programs run on the ODROID N2+ behind a web server. The rig geometry, camera resolution and lighting are not documented. **[MUST STILL BE COMPLETED BY ME: camera model, resolution, mounting height, field of view, lamp, lux readings, photograph of the rig.]**	Incomplete. Sections 3.2, 3.10.
Testing the classifier on the single-board computer with captured images (task 6.1.6)	The deployed programs were written to run on the board and do run on it, but no accuracy or timing figure in this report was obtained on the board; every measurement reported in Section 4 was taken on a laptop.	Incomplete. Section 4.
Gantry design and construction in CAD (FU 3.3, 3.4; R5; task 6.1.7)	A Cartesian X/Y gantry built by the student from bought parts: one motor turning an 8 mm cross-shaft that drives both ends of the moving bridge, and a second motor on that bridge driving the carriage, both on GT2 belts over linear bearings and chromed rods, with four end-stops. Hand-measured travel 204 mm (X) and 262 mm (Y). The machine exists but no CAD drawing or photograph was kept. **[MUST STILL BE COMPLETED BY ME: CAD files and photographs.]**	Incomplete. Section 3.9.
Pen holder, pen adjuster and paper holder (R5)	The pen is carried on the moving carriage in a holder with the toggle cam of the lift mechanism above it, and the page is clamped flat in the camera housing for capture. **[MUST STILL BE COMPLETED BY ME: confirm how the pen height is adjusted and how the paper is held down on the writing bed, and supply a photograph of each; neither is documented in the project files.]**	Completed. Section 3.9.
Power supply (FU 4)	A 24 V supply feeds both stepper drivers through a 100 µF bulk capacitor. The conversion to the voltages the single-board computer and the pen motor require is not documented, no power budget was drawn up and the supply's rating is unknown.	Incomplete. Section 3.9.6.
Stepper-motor control and pen lift (FU 2.8, 3.1 to 3.4; R5; task 6.1.8)	A G-code interpreter generates step and direction pulses directly on the general-purpose pins, with Bresenham interpolation, absolute step bookkeeping, calibration between end-stops, debounced end-stop monitoring and software limits. The pen is lifted by a toggle gear motor confirmed by a position switch, switched by a transistor stage designed by the student, rather than through the stepper driver as the proposal planned. During bring-up the X-axis direction pin was not observed to change level on reversal and that fault was not resolved.	Incomplete. Section 3.9.
Smooth motor control for efficient gantry movement (R3)	The interpreter steps at a constant interval within a segment and executes no acceleration ramp, although the generator's time model assumes one. Abrupt starts at the feeds used may lose steps, and no torque margin was computed because the carriage masses were not measured.	Incomplete. Sections 3.9.2, 3.9.7.
Display and user interface (R4, R6)	A web server exposes classification, generation, calibration and sending of jobs; browser pages show the text, the writer with a confidence, the stored writers and a G-code preview. The synthesis time is returned, but the classification time that R4 asks to be displayed is not shown.	Incomplete. Section 3.10.
Full-system testing against the requirements (task 6.1.9)	The accuracy, writer-identification and timing evaluations of Section 4 were designed and run by the student. The physical writing time of R3, the on-board timing of R4, the positional error of R5 and the random-writer reproduction of R6 were not measured, because the gantry bring-up was not completed.	Incomplete. Section 4.

Table 5. Design summary, part 2 of 2: the capture rig, the gantry hardware, the motion control and the verification tasks.

3.2.1 Hardware block diagram

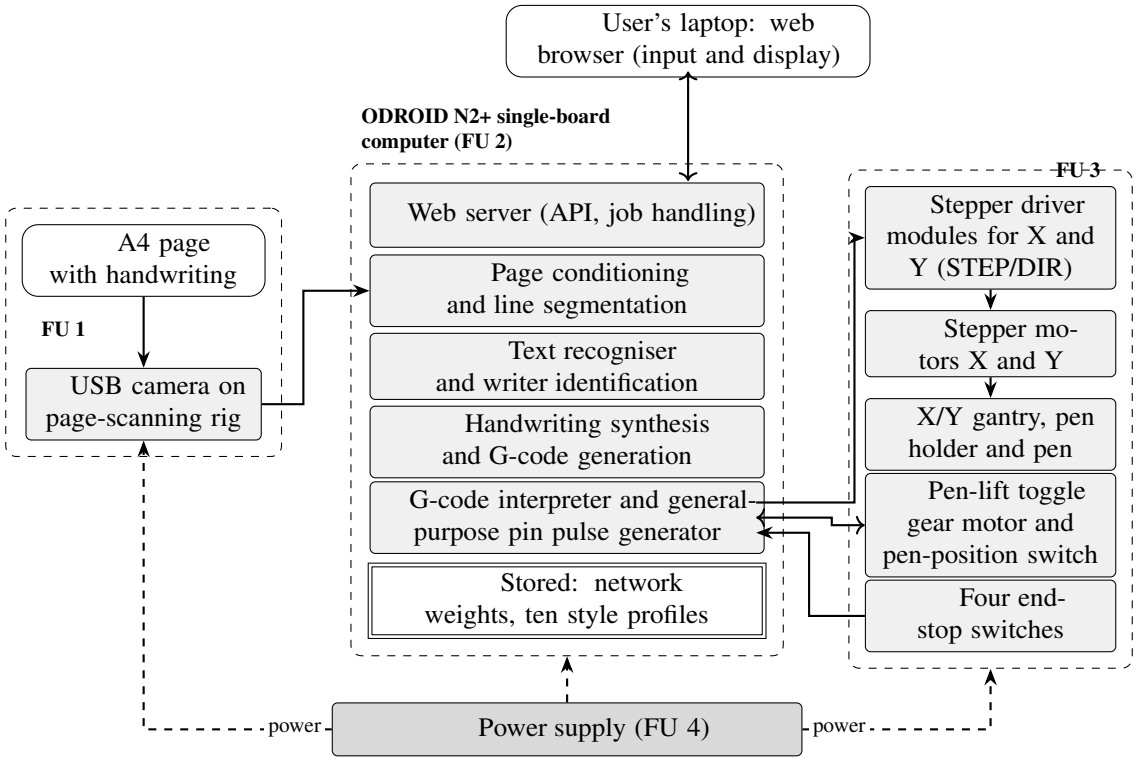


Figure 3. Hardware block diagram of the implemented system. Solid arrows show signals and data, dashed arrows show the power supply. FU numbers refer to the functional units of the project proposal.

The hardware of the implemented system (Figure 3) consists of four groups. The page-scanning rig (FU 1) holds a generic USB camera above an A4 page; the camera is read by the single-board computer through the standard driver of the operating system, so no camera driver was designed by the student. The single-board computer (FU 2) is an ODROID N2+; it runs every software block of the system, including the web server through which the user's laptop controls the robot. The text-reproduction gantry (FU 3) is a Cartesian X/Y gantry in which each axis is moved by one stepper motor; each motor is driven by one stepper driver module whose STEP and DIR inputs are connected directly to general-purpose pins of the single-board computer, so that no separate microcontroller is present. Four end-stop switches, two per axis, bound the travel and are read by the same program that generates the step pulses. The pen is raised and lowered by a toggle gear motor whose state is confirmed by a pen-position switch, and both are connected to general-purpose pins as well. The power supply (FU 4) feeds the camera, the single-board computer and the gantry electronics. **[MUST STILL BE COMPLETED BY ME: model numbers of the camera, stepper motors and driver modules, the supply voltages and current ratings of the power supply, the way in which the pen motor is switched, and the logic-level and grounding arrangement between the single-board computer and the drivers; these are not documented in the project files.]**

3.2.2 Software block diagram and systems-level description

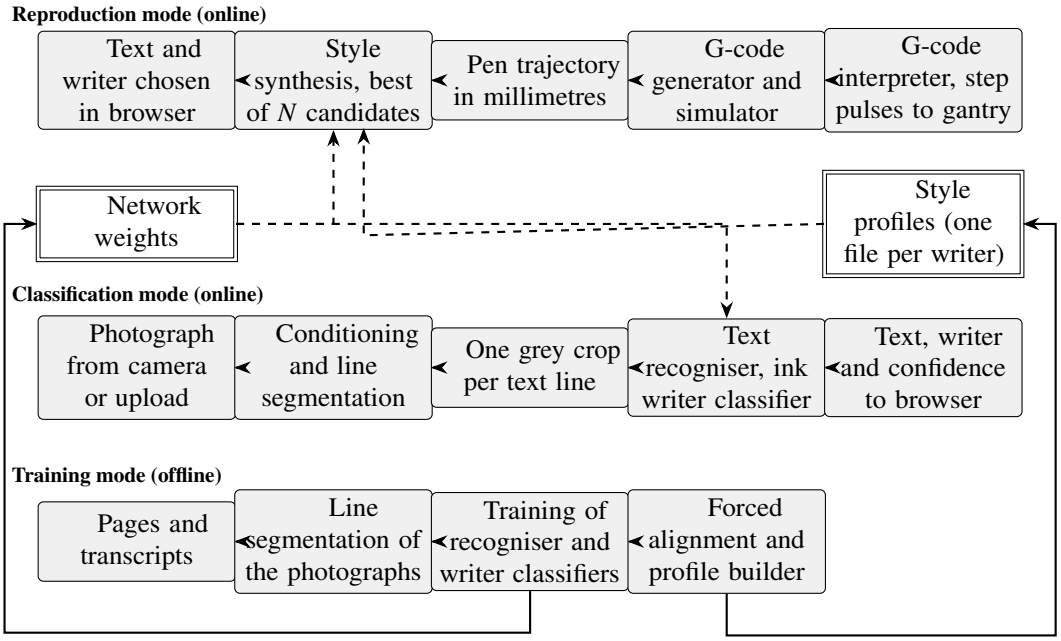


Figure 4. Software data flow of the three modes. Dashed arrows are reads from permanent storage, solid arrows between the training row and the two stores are writes. The same line-segmentation and recognition blocks serve classification, training and the judging of synthesised handwriting.

The intention of the system is to learn how a person writes from photographs of that person's pages and later to write new text in the same style; three chains of blocks share two permanent stores, the network weights and the per-writer style profiles (Figure 4). In the classification mode (R2, R4) the page is conditioned and separated into text lines (Section 3.3), each line is read by the text recogniser (Section 3.4) and classified by the ink-based writer classifier (Section 3.5), and the writer probabilities of the lines are averaged to a page decision. In the reproduction mode (R1, R3, R5, R6) the synthesiser (Section 3.7) assembles the writer's own stored letter paths (Section 3.6) into a line, repeats the random construction for N candidates and keeps the one that the recogniser and the stroke-normalised writer classifier jointly prefer; the trajectory is converted to G-code (Section 3.8) and executed by the interpreter after the gantry has been calibrated between its end-stops (Section 3.9). The training mode runs offline: the same segmentation block prepares the pages, the networks are trained and the profile builder writes the two stores, after which no image and no training code are needed to use a style.

Table 23 relates the implemented blocks to the functional units of the proposal. The structure of the proposal was kept for the physical units FU 1 to FU 4, whereas the software functional units FU 2.1 to FU 2.8 were reorganised for the reasons given in the last column.

3.3 Page conditioning and line segmentation

The first block of the classification chain (functional units FU 2.1 and FU 2.2 of the proposal) turns a photograph of a handwritten page into clean, deskewed, white-background line images for the text recogniser (Section 3.4) and the writer classifier (Section 3.5). Every algorithm was written by the student in Python using plain array arithmetic; no image-processing library was used, and the camera interface is the only external function.

3.3.1 Motivation and design goals

Requirement R2 demands that the content and the writer are decoded from a photographed page, and requirement R4 allows 2 s for capture and conditioning. Both depend on this block, because a badly cropped line cannot be repaired by the networks that follow. A photograph differs from a flat-bed scan by seven nuisances the block has to remove: uneven illumination (a lamp gradient or a shadow), surroundings (desk, hand, neighbouring page), skew, printed structure (ruled and margin lines touching the handwriting), faint marks (bleed-through, pale print, lightly pressed strokes), non-text content (diagrams) and touching lines (descenders meeting ascenders). The block shall therefore (a) work for any camera resolution and handwriting size without manual tuning, (b) never delete handwriting to remove a nuisance, (c) separate text from non-text and (d) use only operations cheap enough for a single-board computer.

Goal (a) is met by *scale invariance*: every photograph is normalised to a long side of 2400 pixels and the median glyph height h_{t} (in pixels) is estimated from the page, so almost every threshold is a multiple of h_{t} (for example a gap of at most $6h_{\mathrm{t}}$) or of the page width and holds for small and large handwriting. Goal (b) is met by a *two-tier ink description*: a strict ink mask decides the page structure (which components exist and which line they belong to), while a permissive "weak-ink" mask only paints pale strokes into the crop of a line already found. The strict mask can therefore be conservative without losing pale words from the crops.

Figure 5 shows the nine stages in execution order; about 60 functions take part. The output is an ordered list of items, each tagged `TEXT` (a line of handwriting) or `MESS` (a non-text region), with one grey-scale crop per item.

3.3.2 Stage 1 and 3: size normalisation, grey conversion and illumination correction

Size normalisation and grey conversion. Every photograph is resampled by bilinear interpolation (Equation 78, Appendix) to a longer side of 2400 pixels and converted to a grey image G with the luma weights of Equation 79, $0.299R + 0.587G_{\mathrm{c}} + 0.114B$. Smaller pages are enlarged and larger ones reduced without a pre-filter, which is cheap.

Summed-area tables and box sums. Most later operations (blurring, local means, dilation, erosion) are sums over rectangular windows, so the block uses the summed-area table (integral image) [24], whose cost does not depend on the window size. For an image I the table S has one extra zero row and column:

$$S(y,x) = \sum_{i<y} \sum_{j<x} I(i,j), \tag{3}$$

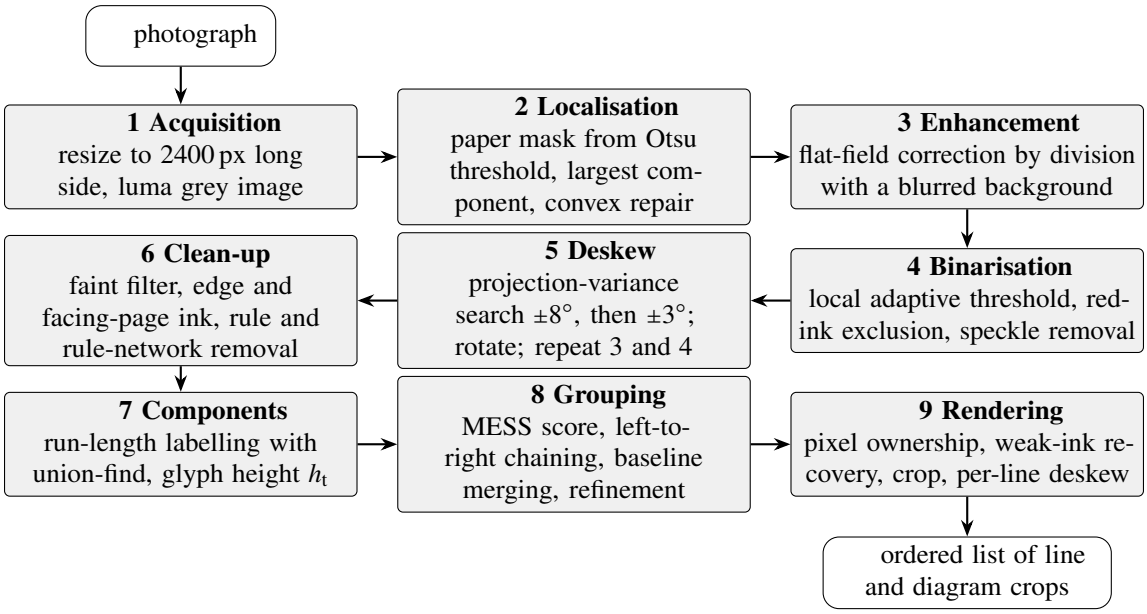


Figure 5. Flow diagram of the page-conditioning and line-segmentation block. The numbers correspond to the subsubsections of this subsection.

The table of Equation 3 is computed with two cumulative sums, one per axis. A window of half-sizes (r_y, r_x) centred on pixel (y,x) then needs four look-ups (Figure 19):

$$\mathrm{BoxSum}(y,x) = S(y_2,x_2) - S(y_1,x_2) - S(y_2,x_1) + S(y_1,x_1), \tag{4}$$

with clamped limits $y_1 = \max(0, y - r_y)$, $y_2 = \min(H, y + r_y + 1)$, $x_1 = \max(0, x - r_x)$ and $x_2 = \min(W, x + r_x + 1)$ for an image of height H and width W. The box mean divides by the pixels inside the clamped window, $\mathrm{BoxMean} = \mathrm{BoxSum}/\big((y_2 - y_1)(x_2 - x_1)\big)$, so border pixels are not darkened. The cost is one $O(HW)$ build plus $O(1)$ per pixel, against $O(r)$ per pixel per axis for a direct convolution; this matters because the illumination blur below has a radius of about 35 pixels.

Gaussian blur from three box blurs. A Gaussian blur is approximated by three successive box means of equal radius (central-limit argument), and the radius used gives a realised standard deviation $\sigma_{\mathrm{real}} \approx 0.58\,\sigma$, so the nominal σ of this report is a blur-scale parameter, not the realised deviation. The derivation and the worked values are in the Appendix, “Gaussian blur from three box blurs”.

Flat-field illumination correction. The observed intensity is approximately paper reflectance times illumination, so the background is estimated as a heavily blurred copy of the grey image and the image divided by it:

$$B = \mathcal{G}_{\sigma_b}(G), \quad \sigma_b = \frac{W}{30}, \qquad I_c(p) = \mathrm{clip}\left(255\,\frac{G(p)}{B(p) + 10^{-3}},\, 0,\, 255\right), \tag{5}$$

In Equation 5, $\mathcal{G}_\sigma$ is the three-box blur of Equation 89 with nominal scale σ, B the background estimate, W the image width, p a pixel position and 10^{-3} a guard against division by zero. Paper pixels end close to 255 and are clipped while ink stays darker than its neighbourhood, so a lamp gradient or a soft shadow becomes a nearly constant white level; division rather than subtraction was used because the lighting is multiplicative. The blur covers the whole frame, including the desk, which makes the paper margin slightly too bright, but the value is clipped and this

is harmless. A limitation is that large ink-heavy areas such as diagrams bias the background estimate (the blur radius is comparable with a few glyph heights), so big dark areas are partly flattened. Figure 20 shows the effect on a real photograph.

3.3.3 Stage 2: paper localisation

The desk, a hand or a facing page must not generate ink or influence the glyph-height estimate, so all later thresholds are confined to a boolean *page mask* that marks the sheet. A global Otsu threshold t^* [21] on the blurred grey image maximises the between-class variance of the intensity histogram,

$$t^* = \arg\max_t \; \omega_B(t)\,\omega_F(t)\big[\mu_B(t) - \mu_F(t)\big]^2, \tag{6}$$

where $\omega_B(t)$ and $\omega_F(t)$ are the numbers of pixels below and above t and $\mu_B(t)$, $\mu_F(t)$ the mean intensities of the two classes; a global threshold suffices because the paper–desk contrast is large. The ten steps that turn this threshold into the final mask, with all their threshold values, are in the Appendix, “Stage 2 details”: a bright low-saturation paper candidate, morphological cleaning, a threshold re-derived from the paper itself for shadowed areas, a convex-hull repair of shadow notches, and a final erosion so the paper edge is not binarised as ink. The method fails for strongly tinted paper (saturation above 90), when the desk is almost as bright as the paper, or when the sheet is not convex. The fallback is the whole frame, which is also used when the mask covers less than 15 % of it. Figure 21 is reserved for the mask construction.

3.3.4 Stage 4: binarisation, red-ink exclusion and speckle removal

After flat-field correction a local threshold separates ink from paper. For a window size $B_w = \max(15, 2\lfloor W/60 \rfloor + 1)$ (61 pixels for $W = 1800$) the local mean is computed with the three-box blur of Equation 89 using the derived blur scale

$$\sigma_a = 0.3\Big(\frac{B_w - 1}{2} - 1\Big) + 0.8 \quad (= 9.5 \text{ for } B_w = 61), \tag{7}$$

With the blur scale of Equation 7, a pixel is declared ink if it is darker than its local mean by more than a constant C:

$$\mathrm{ink}(p) = \mathbb{1}\big[I_{\mathrm{c}}(p) < M(p) - C\big] \wedge \mathrm{pageMask}(p), \qquad C = 12, \tag{8}$$

where $M = \mathcal{G}_{\sigma_a}(I_{\mathrm{c}})$ is the Gaussian-weighted local mean, $\mathbb{1}[\cdot]$ the indicator function and C is in grey levels out of 255. The offset $C = 12$ rejects paper grain and compression noise; a larger value would drop pale strokes and a smaller one admit texture. Pixels whose red channel dominates are excluded, so a red margin line or a marking pen does not become ink, while blue and black ink are kept, and the smallest 8-connected components are then removed as speckle; the colour conditions and the component-size limits are in the Appendix, “Stage 4 details”. A single global threshold fails because shading and pen contrast are of the same order; a local threshold using the standard deviation [22] was not needed, as the two-step normalisation is cheaper and sufficient.

Weak-ink recovery by hysteresis. The strict threshold $C = 12$ drops pale strokes; omitting them would truncate words in the crop, and adding them to the structural mask would create noise. A weak mask with a lower offset is therefore built, also from a colour-aware grey image in which blue ink stays dark, and a weak component is accepted only if it lies close to strong ink; Equation 77 and the Appendix, “Weak-ink recovery by hysteresis”, give the windows and the

area condition. The weak mask never influences which components form a line.

3.3.5 Stage 7: connected components and the glyph-height scale

A connected component is a maximal set of touching ink pixels. Components are found by run-length two-pass labelling with union-find [41, 25], at a cost linear in the number of runs rather than in pixels (Equation 85). The glyph height h_{t} is the median height of components of plausible size (Equation 86), recomputed three times because the cleaning stages change the component set. Both algorithms are given in full in the Appendix.

3.3.6 Stage 5: skew estimation and rotation

The skew is estimated from the projection profile: for horizontal lines the row-sum profile of the ink has sharp peaks and deep troughs. The ink mask, reduced by a factor 4, is rotated through a coarse and then a fine grid of candidate angles, the angle maximising the profile variance $J(\theta) = \mathrm{Var}_y[P_\theta(y)]$ is chosen and applied, stages 3 and 4 are recomputed, and a second residual pass follows. The grids, the rotation by inverse mapping and the thresholds are in the Appendix, with Equations 87 and 88. Only one global angle is estimated, so page curl and keystone are not corrected. This objective was preferred to a Hough transform because ruled lines and diagrams dominate edge directions, and a sign mismatch between rotation and scoring that doubled the skew in an early version was corrected.

3.3.7 Stage 6: removal of printed structure and faint marks

Ruled paper and other printed structure join words and lines into giant components, so it must be removed without removing handwriting that touches it. Four steps are applied, each rule-aware or scale-aware; their step lists, all threshold values and the equations are in the Appendix, “Stage 6 details”. *Rules:* long printed rules are found from the ink-run lengths of every pixel and chained into structures that must cross the page, so a long word or underline cannot be mistaken for a rule. *Faint components:* after a rule has been cut out of any word glued to it, components fainter than the handwriting are removed with a one-dimensional Otsu threshold on component darkness, so each page defines for itself what “faint” means. *Page edges and facing pages:* junk at the mask edges and ink of a neighbouring page are removed by edge and column-profile tests. *Sparse rule networks:* sloped or wavy rules are removed by a column-span test.

3.3.8 Stage 8a: text versus non-text

Diagrams would be read as nonsense by the text recogniser, so each component is scored for being a drawing. Letters such as o, a and e enclose only small holes, below $0.7h_{\mathrm{t}}^2$ in area, whereas diagrams contain large enclosed loops, and open line art is wide and sparse. The score is

$$\mathrm{holeScore} = \mathrm{clip}\Big(\frac{A_h/h_{\mathrm{t}}^2 - 0.7}{0.8}, 0, 1\Big), \qquad \mathrm{sparse} = \begin{cases} 0.7, & w > 6h_{\mathrm{t}} \wedge h > 1.8h_{\mathrm{t}} \wedge \mathrm{fill} < 0.05 \\ 0, & \text{otherwise} \end{cases} \tag{9}$$

$$\mathrm{mess} = \max(\mathrm{holeScore}, \mathrm{sparse}),$$

where A_h is the area of the largest enclosed hole (holes are found by filling the component and subtracting it from the filled version) and w, h and fill are the width, height and fill ratio of the component. Figure 24 shows the ramp: a hole of $0.7h_t^2$ gives the score 0 and a hole of $1.5h_t^2$ the score 1. A component with mess ≥ 0.5 becomes a candidate, unless at least three text-sized neighbours stand in the same row band and the candidate is not taller than their typical height (a veto that rescues large looped letters in headings); surviving candidates are clustered greedily into `MESS` blocks (thresholds in the Appendix, "Text-versus-diagram clustering").

3.3.9 Stage 8b: grouping components into lines

Grouping uses local chaining and a baseline curve instead of a global projection profile, because personal pages have drifting baselines, ascenders that touch the next line and diagrams between the lines. *Core text components* are not `MESS`, have a height between $0.3h_t$ and $1.8h_t$, an area of at least $1.2h_t$ and are not underline-like. They are sorted by centroid abscissa, and each component a is linked to at most one successor b on its right (Figure 29) for which the gap is at most $6h_t$ and $\Delta y = |c_{y,b} - c_{y,a}| \leq 0.75h_t$, choosing the minimum of

$$\text{cost}(a,b) = \max(0, \text{gap}) + 3\Delta y, \tag{10}$$

where gap is the horizontal distance between the bounding boxes (a negative gap larger than $0.6w_a$ is clamped to zero). The factor 3 prefers the same baseline over a nearer component on another row. Union-find collects the links into *chains*.

Baseline curves, merging and leftovers. Each chain receives a baseline curve whose ordinate is an area-weighted five-point moving average,

$$\tilde{y}_i = \frac{\sum_{k=i-2}^{i+2} A_k y_k}{\sum_{k=i-2}^{i+2} A_k}, \tag{11}$$

where y_k and A_k are the centroid row and area of the kth component (the window is truncated at the ends), and $y_{\text{curve}}(x)$ is interpolated linearly. Three passes then merge the chains of one physical row, by curve height, by the measured line pitch so that the thresholds scale with the writing, and by demoting weak isolated chains. Components not yet in a chain are assigned by shape, a tall component linking two rows being split pixel by pixel between the nearest chains so one stroke can be shared by two lines, and a refinement step rejoins rows split by large word gaps, moves underlines to the line above, re-attaches pale components removed as faint and absorbs text lines that lie inside a diagram block. All thresholds, the four criteria for rejoining rows and the exact rules are in the Appendix, "Grouping details", "Leftover components" and "Refinement".

3.3.10 Stage 9: pixel ownership and rendering of the line crops

Rendering decides which pixels the recogniser sees. A pixel-ownership map labels every ink pixel with its item, so a neighbouring row's descender or recovered halo cannot enter a crop. The crop is the padded bounding box of the item, filled from its own dilated mask and from weak ink within a short reach but never from pixels of another item, with grey values taken from the darkest evidence on a white background; a pale word without a strong ink anchor is attached to a row if it lies on its baseline curve. The padding, the dilations and the reach are in the Appendix, "Stage 9: crop rendering details".

A final *per-line deskew* removes the linear drift of the baseline. For `TEXT` items with at least three components and a crop wider than twice its height, a line $y = ax + b$ is fitted to the component centroids by weighted least squares,

$$\min_{a,b} \sum_i A_i \,(y_i - a x_i - b)^2, \qquad \phi = \arctan a, \tag{12}$$

In Equation 12, (x_i, y_i) is the centroid and A_i the area of component i (the area weights make large letters dominate) and ϕ is the drift angle. If $0.4° < |\phi| < 12°$ the crop is padded vertically by $\lfloor |\tan\phi| W_c/2 \rfloor + 4$ white rows (W_c is the crop width), rotated by ϕ with Equation 88 and trimmed again. Each result is a record with the order index, tag, bounding box in the coordinates of the deskewed page, the grey crop and the number of components. Crop heights vary; the scaling to the fixed recogniser canvas is in Section 3.4. Figure 6 is reserved for an example output.

FIGURE MUST STILL BE INSERTED BY ME

Result of the block on a personal page: the deskewed photograph with green boxes for TEXT items and orange boxes for MESS items with order labels (existing preview files `*_segpreview.png` in `Software/CNN/NOGIT/dylan` and `.../yeukita`), next to three example line crops, one of them before and after per-line deskew.

Figure 6. Output of the segmentation block (placeholder).

3.3.11 Data sets, complexity, performance and limitations

Use for the training data and complexity. The same block segments the 13 personal pages used for training; pages of the public database have a fixed printed form and are scanned flat, so a simpler row-projection method prepares them (it is not part of the deployed block). Every pixel pass is $O(N_p)$ for $N_p = HW \approx 4.3 \times 10^6$ pixels; a complete run makes 123 large box-sum calls and 1396 labelling calls (32 of them full-page). Chaining is $O(n_c^2)$ in the worst case (n_c components), the merge passes restart after each merge and dominate, and the leftover assignment evaluates a nearest-chain distance for every pixel of every tall component; peak memory is about a dozen full-page arrays plus integral images of about 35 MB each.

Measured performance. Profiling on the development laptop with the pure-Python implementation gave about 44 to 48 s for one 2400×1800 page in the cleanest run (29 to 142 s were observed over several runs), of which about 70 % was spent in grouping and about 10 s each in box sums and labelling. Optional compiled replacements for the box sum and the labelling make the whole block 25 to 32 % faster; they are not used by the deployed pipeline, and the estimated ceiling of a complete port is 12 to 16 s. **[MUST STILL BE COMPLETED BY ME: the processing time of the block on the ODROID N2+ was not recorded in the project files; measure it.]** These laptop figures are far above the 2 s that requirement R4 allows for image capture and conditioning, so the block as implemented does not satisfy that budget; Section 4 must report this.

Segmentation accuracy. A project note states that the segmenter reproduced the hand-checked line count on a set of six pages; the test set and the scoring rule are not documented, so this statement is not used as evidence here. In a sanity run on one personal page the block found 27

text lines and no diagram, which equals the 27 lines of the transcription file of that page. **[MUST STILL BE COMPLETED BY ME: a quantified segmentation accuracy over the test pages, if it is claimed in Section 4.]**

Design alternatives and justification. Table 13 in the Appendix compares the main design choices with their alternatives. Each chosen operation is the cheapest one robust against exactly one photographic nuisance.

Limitations. The block assumes left-to-right writing in roughly horizontal lines with a single dominant glyph height, so headings that differ strongly in size from the body text can be grouped wrongly (the neighbour veto and the pitch-aware merge mitigate this). A diagram of only open strokes and small extent may be read as text, and text inside a diagram is absorbed into the diagram. Page curl and perspective are not corrected, and tinted paper or a bright desk defeats the page mask. In the deployed classification path the photograph is converted to grey scale before the block, so the red-ink mask is empty and the colour-aware channel equals the luma; those colour features matter only for the colour photographs used to prepare the training data.

3.4 Text recognition

This subsection describes the neural network that decodes a handwritten line image into text (functional unit FU 2.3 of the proposal, requirement R2). The network, its training procedure and the layer library that trains it and runs it on the single-board computer were designed and written by the student. No machine-learning library is used in the recogniser, the writer classifiers of Section 3.5 or the forced alignment of Section 3.6: forward passes, backward (gradient) passes and optimiser updates are all evaluated by the student's own code (Section 3.4.7). Every building block below is explained first in words and then with its equations.

3.4.1 Problem statement and design approach

The input is one grey-scale image of a text line from the segmentation block (Section 3.3); the output is the string that was written. Three properties of handwriting determine the design. First, cursive characters are joined, so their boundaries cannot be located reliably; cutting the line into characters and classifying each piece, as the proposal planned, would pass every cutting error into a wrong reading. Second, the number of characters is unknown and has no fixed relation to image width, since the same letter can be written narrow or wide. Third, the available training material (public handwriting databases and the personal pages of the two writers) gives line images with transcriptions, but not the position of every character.

The design is therefore a *convolutional neural network (CNN)* followed by a *bidirectional long short-term memory (BiLSTM)* network and trained with *connectionist temporal classification (CTC)* [11, 14]. The convolutional layers turn the image into one feature vector per narrow vertical slice, the recurrent layers give every slice the context of the whole line, and the CTC loss allows training from the line transcription alone, with no alignment between characters and columns. The layer pattern follows a published design for the IAM handwriting database [20]; the student fixed the input geometry, built the data pipeline and the training recipe, and verified every layer shape and parameter count (Table 14).

3.4.2 Input pre-processing

The input size is fixed at 64×640 pixels (height × width), which allows batching and a fixed sequence length; it is smaller than the 100×960 of the published design, which needed about one hour per training epoch on a processor-only computer. A line crop of height h and width w

pixels is resized *uniformly* (the aspect ratio is preserved) with the scale

$$s = \min\Big(\frac{64}{h}, \frac{640}{w}\Big), \qquad w' = \max(1, \mathrm{round}(ws)), \qquad h' = \max(1, \mathrm{round}(hs)), \tag{13}$$

using bilinear interpolation (Equation 78), and pasted onto a white 640×64 canvas, left-aligned and vertically centred (Figure 25). Left alignment makes the conversion from a network time step back to a pixel position in the original crop an exact inverse, $x_{\mathrm{orig}} = x_{\mathrm{canvas}}/s$, which the style-extraction block (Section 3.6) needs when it cuts characters out of lines. A plain stretch to the canvas was used initially but distorted letter proportions, so the uniform scale of Equation 13 replaced it; the canvas aspect ratio, the rows a typical line occupies and the measured distortion of the rejected stretch are given in the Appendix. The grey values $p \in [0, 255]$ are then inverted and normalised with a *fixed* constant,

$$x = \frac{(1 - p/255) - 0.5}{0.5} = 1 - \frac{2p}{255}, \tag{14}$$

where $x \in [-1, 1]$ is the network input; white paper ($p = 255$) becomes -1 and black ink ($p = 0$) becomes $+1$. A fixed constant replaces a per-image mean and standard deviation, so the amount of white padding cannot change the contrast statistics.

3.4.3 Network architecture

The $1 \times 64 \times 640$ input passes through stage 1 (two units of convolution, batch normalisation and ReLU, 32 channels), a 2×2 max-pool, stage 2 (four such units, 64 channels), a second 2×2 max-pool and stage 3 (six such units, 128 channels), which leaves a $128 \times 16 \times 160$ feature map; a maximum over the height turns it into 160 vectors, two bidirectional LSTM layers of hidden size 256 per direction give 160×512, and a linear layer with log-softmax gives the 160×75 scores that greedy CTC decoding turns into text. The overview diagram (Figure 47) and the table of every layer with its output shape and parameter count (Table 14) are in the Appendix; the building blocks are defined first.

Two-dimensional convolution. A convolutional layer slides a small learned filter over the image, so the same pattern detector acts at every position. For an input feature map X with C_{in} channels, the output channel o at position (i, j) is

$$Y[o, i, j] = b_o + \sum_{c=1}^{C_{\mathrm{in}}} \sum_{u=0}^{k-1} \sum_{v=0}^{k-1} W[o, c, u, v]\, X_{\mathrm{pad}}[c, s\,i + u, s\,j + v], \tag{15}$$

with W the learned weights, b_o the bias of channel o, k the kernel size, s the stride and X_{pad} the input zero-padded by p pixels on every side. The output size is

$$H_{\mathrm{out}} = \Big\lfloor \frac{H + 2p - k}{s} \Big\rfloor + 1, \qquad W_{\mathrm{out}} = \Big\lfloor \frac{W + 2p - k}{s} \Big\rfloor + 1, \tag{16}$$

which equals the input size for the values used, $k = 3$, $p = 1$ and $s = 1$, so only pooling changes the size. A layer has $C_{\mathrm{out}}(C_{\mathrm{in}}k^2 + 1)$ parameters.

Batch normalisation. Every convolution is followed by batch normalisation [8]: during training each channel is normalised with the mean and variance of the mini-batch (Equation 91 in the Appendix) and then scaled and shifted by learned parameters. At test time running statistics replace them, reducing the layer to a fixed per-channel affine map (Equation 92 in the Appendix), the form the inference implementation uses.

Rectified linear unit and pooling. The non-linearity is the rectified linear unit $\mathrm{ReLU}(x) = \max(0, x)$. A 2×2 max-pooling with stride 2 outputs the maximum of each block, $Y[c, i, j] = \max_{u,v \in \{0,1\}} X[c, 2i + u, 2j + v]$, which halves height and width, tolerates small shifts and has no

parameters. After the last convolution the remaining 16 rows are collapsed by a maximum over the height,

$$z_t[c] = \max_i F[c, i, t], \qquad t = 1, \dots, T = 160, \tag{17}$$

where F is the output of the last convolutional stage and $z_t \in \mathbb{R}^{128}$ the feature vector of step t. The collapse keeps only whether a feature is present anywhere in the column, which suits ascenders and descenders, and turns the map into a sequence. The two pools halve the width twice, so $T = 640/4 = 160$ steps remain and one step covers a four-pixel-wide column group.

Receptive field. The recursion of Equation 80 (Appendix) gives each column feature a receptive field of 72×72 pixels, the full height of the line and about 18 time steps of context, before the recurrent layers add unbounded horizontal context.

Long short-term memory. The recurrent part reads the 160 feature vectors in order. A plain recurrent network forgets information over long sequences, so a long short-term memory (LSTM) cell [10] is used, whose memory c_t is protected by three learned gates (Figure 26). For input x_t and the previous hidden state h_{t-1} and memory c_{t-1} (both initially zero), one direction of one layer with hidden size $n = 256$ computes

$$\begin{aligned}
i_t &= \sigma(W_{ii}x_t + b_{ii} + W_{hi}h_{t-1} + b_{hi}), && \text{input gate,} \\
f_t &= \sigma(W_{if}x_t + b_{if} + W_{hf}h_{t-1} + b_{hf}), && \text{forget gate,} \\
g_t &= \tanh(W_{ig}x_t + b_{ig} + W_{hg}h_{t-1} + b_{hg}), && \text{candidate memory,} \\
o_t &= \sigma(W_{io}x_t + b_{io} + W_{ho}h_{t-1} + b_{ho}), && \text{output gate,} \\
c_t &= f_t \odot c_{t-1} + i_t \odot g_t, \qquad h_t = o_t \odot \tanh(c_t),
\end{aligned} \tag{18}$$

with $\sigma(a) = 1/(1 + e^{-a})$ the logistic function, $\odot$ the element-wise product, the W weight matrices (input $W_{i\cdot}$, recurrent $W_{h\cdot}$) and the b bias vectors. The forget gate sets how much old memory is kept, the input gate how much of the candidate is written and the output gate how much is revealed; how the four pre-activations are obtained from one matrix product in the implementation is given in the Appendix. A *bidirectional* layer runs one LSTM from step 1 to 160 and an independent second from 160 to 1, and concatenates their hidden states at every step,

$$h_t = \left[\overrightarrow{h}_t ; \overleftarrow{h}_t \right] \in \mathbb{R}^{512}, \tag{19}$$

so each step sees the letters before and after it, which is needed to distinguish the cursive strokes of u, n and m. Two such layers are stacked, with no dropout, following the published design.

Output layer and parameter count. A linear layer maps the 512-dimensional state of each step to one score ℓ_t^k for each of the 75 classes (74 characters and one blank, index 0), and a log-softmax turns the scores into log-probabilities,

$$\ell_t = W_o h_t + b_o, \qquad \log y_t^k = \ell_t^k - \log \sum_{j=0}^{74} e^{\ell_t^j}, \tag{20}$$

with $W_o \in \mathbb{R}^{75 \times 512}$ and $y_t^k = P(\pi_t = k \mid x)$ the probability of class k at step t. The 74 symbols are the letters, the digits and twelve punctuation marks, listed in the Appendix with the filter that removes every other character from the training targets. The counts in Table 14 follow from the shapes. The network holds 3 358 763 parameters, of which the LSTM layers hold 70.5 %, and one line costs about 4.17×10^9 multiply–accumulate operations, 91 % of them in the convolutions, while the LSTM is inherently sequential, with $160 \times 2 \times 2 = 640$ small matrix–vector products. The per-layer parameter formulae (Equation 101), the share of every stage, the operation count of Equation 102 and its breakdown are given in the Appendix.

3.4.4 Connectionist temporal classification

The network outputs a distribution over 75 symbols at each of 160 steps, but the transcription has only, for example, 40 characters and its alignment with the steps is unknown. CTC solves this by summing over all alignments. A *path* $\pi = (\pi_1, \dots, \pi_T)$ assigns one symbol, possibly the blank, to every step; the many-to-one map $\mathcal{B}$ merges consecutive repeats and then deletes the blanks, so that $\mathcal{B}(\text{–hh-e-ll-ll-o}) = \text{hello}$, the blank between the two groups of l being what allows a genuine double letter. With the steps conditionally independent, the probability of a transcription $y = (y_1, \dots, y_U)$ of length U is the sum over all paths that collapse to it, and the loss its negative logarithm:

$$P(\pi \mid x) = \prod_{t=1}^{T} y_t^{\pi_t}, \qquad p(y \mid x) = \sum_{\pi : \mathcal{B}(\pi) = y} P(\pi \mid x), \qquad \mathcal{L}_{\mathrm{CTC}} = -\ln p(y \mid x). \tag{21}$$

The number of paths is exponential in T, so the sum is evaluated by dynamic programming on the blank-augmented label $y' = (\varnothing, y_1, \varnothing, y_2, \dots, y_U, \varnothing)$ of length $S = 2U + 1$ (Figure 30). The forward variable $\alpha_t(s)$ is the total probability of all consistent path prefixes of length t ending at position s of y':

$$\begin{aligned} &\alpha_1(1) = y_1^{\varnothing}, \quad \alpha_1(2) = y_1^{y_1}, \quad \alpha_1(s) = 0 \ (s > 2), \\ &\alpha_t(s) = \Big[\alpha_{t-1}(s) + \alpha_{t-1}(s-1) + \mathbb{1}\big[y'_s \neq \varnothing \wedge y'_s \neq y'_{s-2} \big] \, \alpha_{t-1}(s-2) \Big] y_t^{y'_s}, \end{aligned} \tag{22}$$

$$p(y \mid x) = \alpha_T(S) + \alpha_T(S-1), \tag{23}$$

where $y_t^{\varnothing}$ is the network's blank probability at step t and $y_t^{y'_s}$ the probability of the symbol at position s. The three bracket terms are the allowed transitions into position s: staying on the same symbol, advancing one position, and skipping a blank to the next symbol. The skip is forbidden when y'_s is a blank or equals y'_{s-2}, since skipping the blank between two equal letters would merge them. Backward variables $\beta_t(s)$ follow symmetrically from $t = T$, and the gradient with respect to the pre-softmax score is

$$\frac{\partial \mathcal{L}}{\partial \ell_t^k} = y_t^k - \frac{1}{p(y \mid x)\, y_t^k} \sum_{s : y'_s = k} \alpha_t(s) \beta_t(s), \tag{24}$$

from which back-propagation proceeds through the log-softmax, both LSTM layers and the convolutional stack. The recursion runs in the logarithmic domain for numerical stability, and each line's loss is divided by its target length, so every loss quoted below is a per-character negative log-likelihood. The remaining conventions of the implementation, namely the treatment of lines with no valid path, the padding of short lines and the 120-character label cap that keeps every label far below T, are given in the Appendix.

Decoding. At inference the most probable symbol of each step is taken, repeats are merged and blanks deleted:

$$\hat{\pi}_t = \arg\max_k y_t^k, \qquad \hat{y} = \mathcal{B}(\hat{\pi}). \tag{25}$$

This best-path decoding approximates $\arg\max_y p(y \mid x)$ in 160 arg-max operations; no lexicon, language model or beam search is used. The published design reaches its best error rate with lexicon-constrained decoding, which was not implemented here, so that publication's headline figure is not a result of this project.

3.4.5 Training data and splits

Table 15 lists the data sets, the rule separating training from validation data, and the leakage each rule leaves.

In short, the first-stage hold-out is writer-dependent with noisy labels; the public set's validation split drives learning-rate decay and checkpoint choice, so its accuracy is mildly optimistic, while its test split is used for no decision, although the partitions are only believed, not verified, to be writer-independent; the personal validation lines are random lines of the same pages as the training lines (optimistic); and the ten-author hold-out may overlap the training data and has project-generated labels.
The labels of the database pages were generated by aligning the printed prompt words with the detected ink word runs by dynamic programming (Appendix). An earlier purely proportional labelling assigned wrong text to lines; correcting the evaluation labels, with no change of the weights, raised the measured accuracy on the ten-author hold-out from 42.7 % to 89.4 %. Label quality, not the network, had limited the earlier measurements, which is why the recogniser was later trained on a public line-level set with clean labels.

Data augmentation. Training images, and never validation images, are augmented after the resize with at most one of four geometric transformations: horizontal shear, rotation, elastic distortion [42] and perspective distortion. The enabling probability of each transformation, the fraction of images that stay unchanged and every transformation parameter are given in the Appendix.

3.4.6 Training procedure

Optimiser. The loss of Equation 21 is minimised by mini-batch gradient descent with the RMSprop update. For the gradient g_t at update t, learning rate η and weight decay $\lambda = 10^{-5}$ (an L2 term added to the gradient),

$$g_t \leftarrow g_t + \lambda\,\theta_{t-1}, \qquad v_t = \rho\,v_{t-1} + (1-\rho)\,g_t^2, \qquad \theta_t = \theta_{t-1} - \eta\,\frac{g_t}{\sqrt{v_t}+\varepsilon}, \tag{26}$$

with θ the parameters, v_t the running average of the squared gradient (smoothing $\rho = 0.99$) and $\varepsilon = 10^{-8}$; there is no momentum. Dividing by $\sqrt{v_t}$ gives every parameter its own step size, which suits the very different gradient magnitudes of convolutional and recurrent layers. In the joint stage the global gradient norm is clipped at 5,

$$g \leftarrow g\,\min\Big(1, \frac{5}{\|g\|_2}\Big), \tag{27}$$

which stops a rare exploding gradient of the recurrent layers from destroying the weights; batches with a non-finite loss are skipped. A plateau schedule halves the learning rate on a validation loss that stops improving, and validation uses no augmentation, the batch-normalisation running statistics and greedy decoding; the schedule and the patience are given in the Appendix.

Staged recipe. Training was carried out in four stages, each motivated by a limitation of the previous one: the published recipe on lines segmented from database pages by the student's own tools, which gave small and noisy data; a warm start on the public line-level IAM set with human-checked labels; fine-tuning on the personal lines alone, which was rejected because the general accuracy fell by about ten points (catastrophic forgetting, Table 17); and joint training from the second stage on both sets with a weighted sampler, so that 20 % of every epoch is personal data with fresh augmentation and the checkpoint with the largest $\min(\text{accuracy}_{\text{public}}, \text{accuracy}_{\text{personal}})$ is kept. The four stages with their losses and accuracies, and the sampler weights of Equation 90, are in the Appendix (Table 16 and Figure 32). A sweep over learning rate, LSTM width, depth, convolution width, dropout and optimiser (14 configurations) was programmed, but no result file was kept, so the deployed architecture is the published one and not an optimum found by the student. Training was done offline on a general-purpose computer; the log of the joint stage was not kept, and the stored joint checkpoint contains weights only.

3.4.7 From-scratch forward-and-backward implementation

Only work done from first principles counts for credit in this report, so a framework supplying ready-made layers and automatic differentiation would earn no credit for the recogniser's training, even though the student designed the architecture and data pipeline around it. The student therefore wrote one Python module that implements, from the defining equations and with no machine-learning library, the forward *and* the backward (gradient) pass of every layer used by the recogniser and the two writer classifiers of Section 3.5, together with the two optimiser updates, the learning-rate schedules and the gradient-norm clip, so that no part of training depends on an external library; the Appendix lists each implemented layer and update against the equation it was derived from. The convolutional stack, the pooling layers and the fully connected layer are also called in their inference (eval) form by the validation step at the end of every epoch and by the deployed program on the single-board computer, so training, validation and deployment are three uses of one implementation.

Verification by numerical differentiation. No automatic differentiation tool checks these hand-derived backward passes, so each was verified independently against a central-difference estimate obtained by perturbing one parameter at a time with the step $\varepsilon = 10^{-5}$ (Equation 103 in the Appendix), on a scalar loss whose own derivative is exact, and a configuration, meaning one (layer variant, tensor) pair, was accepted when the relative difference between the analytic and the numerical gradient stayed below 10^{-6} (Equation 104 in the Appendix). Of the 78 configurations, 77 passed. The relative differences of Equation 104 lie between 4.9×10^{-12} and 1.4×10^{-10}, four to six orders of magnitude inside the tolerance, the agreement expected of a correct derivative at this step size. Which gradients one configuration covers, the construction of the scalar loss, the standalone check program, the number of elements probed, a stricter per-element measure with its worst case, and the precautions taken for the layers with kinks and for dropout are given in the Appendix.

The one configuration that failed is a genuine defect, reported here rather than removed. The CTC loss does not survive an empty label sequence: for a transcription of length $L = 0$ the blank-interleaved target has $S = 2L + 1 = 1$ entry, but two of the shifted arrays that carry the recursion are built with two entries regardless of S, so the recursion writes a two-element array into a one-element row and raises a shape error instead of returning the loss. A single-character target ($S = 3$) and a target with a repeated character, the case that exercises the forbidden skip transition, both pass, so this is an edge-case indexing fault and not a wrong derivative. Its practical effect is small, because a line with no transcription never reaches the trainer, but the function is not safe against one and the fault is listed in Section 5. The check also confirmed that when no alignment exists, that is when $2L + 1 > T$, the loss is returned as infinite with undefined gradients; this is correct for an impossible alignment, but such samples must be removed before training rather than relied upon to contribute nothing.

Every layer was first written as an explicit loop over the batch for clarity and correctness, and the batch loop was then replaced, where it dominated the running time, by one batched matrix product; the gradient check runs against the batched version the trainer uses, so the faster form is the one verified. The forward and backward code of the convolution, which illustrates the technique used throughout, is Figure 48 in the Appendix.

Training speed and warm-starting. With no graphics processor and no vectorised automatic-differentiation engine behind it, one training step of a batch of 16 real lines took on the order of tens of seconds on a general-purpose laptop processor, substantially slower than the published-framework implementation used for the four-stage recipe above. Timing showed that an un-

vectorised convolution, not the recurrent layers or the CTC loss, was the dominant cost, hence the batched matrix product of Figure 48. Training is therefore not run to convergence from a random initial condition at this speed; the warm start from an existing checkpoint and the resumption of an interrupted run are described in the Appendix. Complete code for this module, the shared training program and the deployed inference program has been submitted separately on the AMS.

3.4.8 Evaluation metrics

Accuracy is measured with the Levenshtein edit distance [19], the minimum number of insertions, deletions and substitutions that turn one string into another (recursion in the Appendix). For training and validation the character error rate is micro-averaged over all lines n with prediction $\hat{y}_n$ and reference y_n,

$$\mathrm{CER} = \frac{\sum_n d_{\mathrm{lev}}(\hat{y}_n, y_n)}{\sum_n |y_n|}, \qquad \text{character accuracy} = 1 - \mathrm{CER}, \tag{28}$$

where d_{lev} is the edit distance; the measure is case-sensitive. The end-to-end evaluation of Section 4 instead averages per line the floored accuracy $\max(0, 1 - d_{\mathrm{lev}}/|y|)$ after lower-casing, which is slightly more lenient, so the two kinds of figure are not computed identically. The word measure (in-order recall of the reference words, Equation 82 in the Appendix) is not $1 - \mathrm{WER}$; a standard word error rate (WER) was computed only once, for the public sets.

The measurements taken while this block was being designed, together with the limits of what they support, are reported in the Appendix (Table 40).

3.4.9 Inference implementation for the single-board computer

The single-board computer runs the eval-mode form of the same module (Section 3.4.7), in Python, using only array arithmetic, so no re-implementation was needed between training and deployment. Each convolution becomes one matrix product (im2col): the padded input is cut into overlapping 3×3 patches that become the columns of a matrix cols, and

$$Y = \tilde{W}\,\mathrm{cols} + b\,\mathbf{1}^{\top}, \tag{29}$$

where $\tilde{W} \in \mathbb{R}^{C_{\mathrm{out}} \times C_{\mathrm{in}} k^2}$ is the weight tensor reshaped to a matrix and $b\,\mathbf{1}^{\top}$ adds the channel bias at all positions. One optimised matrix product replaces the six nested loops of Equation 15 and was about ten times faster than a general summation routine, at the cost of memory; the size of the largest matrix and of the working set is given in the Appendix. Batch normalisation uses the folded form of Equation 92, and the LSTM is an explicit loop over the 160 steps. The weights are read directly from the plain array checkpoint written by training, with no parsing of any third-party model format (Appendix, subsection “Text recogniser: weight file and inference details”). Before the eval-mode form was trusted, its forward pass was cross-checked, independently of the gradient check, against the training-mode forward pass on the same weights with the batch-normalisation statistics frozen: on the final trained weights the agreement over 1358 real lines was 100.00 % top-1 agreement with a maximum absolute logit difference of 7.6×10^{-6}, and the earlier single-input comparison is reported in the Appendix. Recognition measured 8.86 ms per character and 353 ms per line over 669 real crops, inside the 25 ms per character of requirement R4 by a margin of about 2.8 times, with the model already resident in memory on the development laptop. The earlier figure of 65 to 75 ms per character differs by measurement conditions, not by a change of model: it was a single isolated forward pass including image preparation, whereas the current figure is total inference time divided by total characters over a full run. Timing on the ODROID N2+ was not measured, and the ARM processor is expected

to be slower than the development laptop, so the margin is expected to shrink on the target hardware.

3.4.10 Design alternatives and justification

Table 18 compares the design with the alternatives that were considered. Only the chosen design was built and measured; the statements about the others are arguments, not measurements.
The chosen design fits the four facts of the problem: the output is a sequence of unknown length (CTC), cursive strokes need context from both sides (bidirectional recurrence), the available labels are line transcriptions (an alignment-free loss), and the model is small enough to re-implement with matrix products and element-wise gate equations on the single-board computer. The same convolutional backbone is reused for the writer classifiers of Section 3.5 and the forced alignment of Section 3.6, which saves design and training effort.

3.5 Writer identification

This subsection describes the neural networks that identify which of the thirteen stored writers wrote a line or a page (functional unit FU 2.3, requirement R2) and that judge how closely synthesised handwriting resembles a writer (requirement R1). Two classifiers with the same architecture were designed, differing in input pre-processing and in what was trained. The training programs, the stroke-normalisation algorithm, the aggregation rule, the shared from-scratch layer library and the inference code were written by the student; training and inference call the same Python functions (Section 3.4.7).

3.5.1 Problem statement and the need for two classifiers

The task is closed-set writer identification: given a line image, choose one of $K = 13$ known writers, eight from a public handwriting database and the five personal writers whose pages were photographed for this project. A line-level image classifier was chosen, trained with few lines per writer (63 to 184 training lines per writer, Table 20).
A first classifier, trained on the raw grey-scale line crops, classified real held-out lines almost perfectly (99.78 % over 1358 lines in the final system) but identified the writer of *synthesised* output in only about 28 % of trials, with the errors concentrated on a few classes rather than scattered. The explanation is the physics of the plotter: it draws every stroke with one pen at one width and one pressure, whereas real handwriting carries a stroke thickness, an ink density and a pressure modulation that a convolutional network readily exploits when training lines are few. A judge relying on such cues does not predict what the robot can achieve, nor match how a person judges a style, which is by geometry (slant, proportions, spacing, letter shapes and joins). A second classifier was therefore built whose input contains only geometry, the thickness having been removed by construction by the stroke-normalisation algorithm of Section 3.5.3.
The ink classifier takes the raw grey-scale line crop, is used for real photographed pages in classification mode, and has all 966 445 parameters trained from the recogniser backbone. The shape classifier takes the stroke-normalised crop, judges synthesised and G-code-rendered output inside the best-of-N search of Section 3.7, and trains only the 13 645 parameters of its head on the frozen recogniser backbone.

3.5.2 Architecture

Both classifiers use the same network, whose overview diagram and layer table are in the Appendix (Figure 49 and Table 19). The line crop is brought to the fixed 64×640 canvas and normalised exactly as for the text recogniser (Equations 13 and 14). The backbone is

the 12-layer convolutional stack of the recogniser (Table 14, stages 1 to 3, with convolution, batch normalisation and ReLU defined in Equations 15 to 92), which delivers a feature map $F \in \mathbb{R}^{128\times16\times160}$. The recurrent layers are not used, because writer identity does not depend on the letter sequence. The feature map is reduced to one 128-dimensional vector by a maximum over the height and a mean over the width:

$$m_{c,t} = \max_{r=1,\dots,16} F_{c,r,t}, \qquad f_c = \frac{1}{160}\sum_{t=1}^{160} m_{c,t}, \qquad f \in \mathbb{R}^{128}, \tag{30}$$

where c is the channel, r the row and t the column of F. The network is fully convolutional along the width, so the representation is a "bag of local stroke patterns" independent of position along the line. The mean runs over all 160 columns, including the white padding of short lines, so a short line's feature is diluted; this is the same in training and in use, but the absolute size of the scores, and therefore the confidence, depends on line length. A two-layer head converts f into scores for the $K = 13$ writers:

$$h = \mathrm{ReLU}(W_1 f + b_1) \in \mathbb{R}^{96}, \qquad z = W_2\,\mathrm{Drop}_{0.2}(h) + b_2 \in \mathbb{R}^{K}, \tag{31}$$

where $W_1 \in \mathbb{R}^{96\times128}$ and $W_2 \in \mathbb{R}^{K\times96}$ are learned and $\mathrm{Drop}_{0.2}$ is dropout [43] with probability 0.2: during training each element is replaced by $h_k\xi_k/(1-0.2)$ with $\xi_k \sim \mathrm{Bernoulli}(0.8)$, which prevents the head from relying on single units, and at inference it is the identity. The scores become probabilities by the softmax and the network is trained with the cross-entropy loss,

$$p_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}, \qquad \mathcal{L} = -\frac{1}{B}\sum_{n=1}^{B} \log p_{y_n}^{(n)} = \frac{1}{B}\sum_{n=1}^{B}\Big[\log\sum_{j} e^{z_j^{(n)}} - z_{y_n}^{(n)}\Big], \qquad \hat{k} = \arg\max_k p_k, \tag{32}$$

where B is the batch size, y_n the true writer of sample n, and p_k the probability that the line was written by writer k; the loss is unweighted, without class weights or label smoothing.

3.5.3 Stroke normalisation

Stroke normalisation re-draws every line with a constant pen width proportional to the line's own letter height, so the thickness cue is identical for all writers and for synthesised lines. It is applied to the full-resolution crop *before* the resize to the 64×640 canvas, in six steps (Figure 31).

The crop is first binarised with the global Otsu threshold (Equation 6), ruled lines and underlines are removed by the run-length test of Equation 105 in the Appendix, hairline breaks are closed, specks are removed, and a crop with no ink left is dropped. With the row ink profile $P(r) = \sum_c \mathrm{ink}(r,c)$, the core band then runs from the first to the last row with $P(r) \geq 0.45 \max P$, the dense band of the lower-case bodies, while ascenders and descenders are sparse, and the pixel x-height is

$$x_h = \mathrm{base} - \mathrm{top}, \qquad \text{or } x_h = \max(8, h/3) \text{ if no valid band exists or } x_h < 4, \tag{33}$$

where top and base are the first and last row of the band and h is the crop height. The target stroke width is proportional to the x-height, which makes it independent of the scan resolution:

$$w = \mathrm{clip}\big(\mathrm{round}(0.14\,x_h), 2, 9\big), \qquad k = 2\lfloor w/2 \rfloor + 1, \tag{34}$$

where k is the odd side of the structuring element that realises the width (for example $x_h = 20$ gives $w = 3$ and $k = 3$). The ink is then thinned to a one-pixel centre line with the iterative algorithm of Zhang and Suen [32], which deletes boundary pixels under neighbour-count and transition-count conditions (Equation 83 in the Appendix) and preserves stroke topology; the skeleton is dilated back to the constant width k (Equation 106 in the Appendix); and the result is inverted to black strokes on white and passed through the standard resize and normalisation. The thresholds and structuring-element sizes of the binarisation, rule-removal and speck-removal steps, and the effect of the square dilation kernel on diagonal strokes, are given in the Appendix.

Stroke normalisation removes the thickness and pressure cues but leaves slant, letter proportions, spacing and connection habits, the properties the style profiles and the synthesiser measure. Every input of the shape classifier, real or synthesised, passes through the same transform, so real training lines and synthesised test lines share an identical geometry-only distribution; the ink classifier on real pages does not use it. The synthesis renderer draws with a pen width of 0.085 times the x-height and the judge uses 0.14; because the judge re-draws every input itself, the two constants need not match.

3.5.4 Training data, protocol and leakage

The thirteen writers are the database writers 150, 151, 152, 153, 384, 551, 552 and 588 (ten pages of each, except where a page failed the crop-to-label check) and the personal writers, whose photographed pages were added as the project progressed. This writer count, against which the current system is measured, exceeds the ten styles required by the proposal. The stroke-normalised data set used for the current measurements holds the lines listed in Section 4: 689 database-writer lines and 669 personal-writer crops, 1358 in total.

The split differs between the two kinds of writer and the figures must be read accordingly. For the database writers the last labelled page of each writer is held out and all its lines are validation lines, so the split is at page level with the same writer in training, which is the intended closed-set task; each writer contributes 8 to 11 validation lines. For the personal writers the lines of every photographed page are shuffled with a fixed seed and 15 % (at least one line) become validation lines, so training and validation lines come from the same physical pages, pen, paper, lighting and session. A line classifier can exploit such page-level nuisance, and since each personal class is perfectly confounded with its collection session, the validation figure is inflated for 52 of the 123 validation lines (42 %). The checkpoint with the best validation accuracy is also the one reported, so the validation set serves both for selection and for reporting; with 123 lines a single line is 0.81 percentage points. There is no untouched test set for the writer classifiers, and the validation accuracies below must not be described as true held-out test accuracies. A defensible mitigation that was not carried out is to hold out whole pages of the personal writers.

Augmentation. The recogniser's augmentation (shear, rotation, elastic and perspective distortion, at most one per image) is applied to the training images. Shear and perspective change slant and proportions, which are themselves writer cues; it was inherited from the recogniser and not tuned. The shear magnitude is compared with the measured writer slants in the Appendix.

Optimisation. Both classifiers are trained with AdamW [9, 44], which applies the weight decay (here 10^{-4}) separately from the gradient step, with a cosine learning-rate schedule (Equation 84 and Table 21 in the Appendix). The AdamW update rule and the gradient-norm clip of Equation 27 were implemented by the student from the defining equations, inside the same from-scratch layer library used for the text recogniser (Section 3.4.7), not taken from an existing framework. The ink classifier is fully fine-tuned from the recogniser backbone, while for the shape classifier the frozen backbone computes the 128-dimensional feature of every training image once, each line appearing four times, and only the head is trained on these stored vectors; this is fast and cannot overfit the thickness cue in the backbone, but augmentation is not re-sampled between epochs. The number of stored feature vectors, the comparison that confirms which backbone tensors changed and which stayed bit-identical to the recogniser's, and the checkpoint format the two training programs share with the recogniser are given in the Appendix.

3.5.5 From line decisions to a page decision

On real photographs the ink classifier is applied to every text line that the segmentation block delivers, and the probabilities are averaged:

$$\bar{p}_k = \frac{1}{N}\sum_{n=1}^{N} p_k^{(n)}, \qquad \hat{k} = \arg\max_k \bar{p}_k, \qquad \text{confidence} = 100\,\bar{p}_{\hat{k}}\ \%, \tag{35}$$

where N is the number of detected text lines, $p_k^{(n)}$ the probability of writer k for line n and $\bar{p}_k$ the page probability (Figure 28). All lines carry equal weight, not weighted by length or by confidence. Averaging is less sensitive to a few confidently wrong lines than a majority vote and uses all the probability mass; no experiment comparing the two was carried out. The confidence is a softmax probability, not a calibrated one: no calibration step exists and the networks were trained to a near-perfect training fit, so over-confidence is expected. For synthesised output the shape classifier is used differently: each rendered sentence is classified by the arg-max of a single softmax, and the probability of the target writer becomes one score in the best-of-N search of Section 3.7.

3.5.6 Design-stage evidence

Table 6 lists the measurements that motivated the design and the status of each figure; the final qualification figures belong to Section 4.

Measurement	Result	Status of the figure
Ink classifier, 1358 real lines (current 13-writer system)	**99.78 %**	689 database-writer lines and 669 personal-writer crops. The strongest figure in the project and the classifier the live server uses on photographs.
Shape classifier, 1358 real lines (after the training fault was corrected)	**93.67 %**	Same lines. Raised from 83.36 % by training the network end-to-end instead of fitting only the classification head on frozen recogniser features.
Shape classifier, 1358 real lines, before the correction	83.36 %	118 of one writer's 148 lines were assigned to a single neighbour, a systematic rather than scattered error.
Ink classifier on synthesised output	28 % over 39 drawn lines	Chance for thirteen writers is 7.7 %. The ink classifier depends on stroke thickness and ink density, which a constant-width plotter cannot reproduce, so it cannot judge the machine's own output; this is why the normalised classifier exists.
Shape classifier on synthesised output, current system	89.7 % through G-code over 13 writers, 3 sentences, best-of-8; 80.1 % in the official run at best-of-30	The same classifier family selects the candidates, so these figures measure whether the pipeline satisfies its own judge and not an independent verdict.

Table 6. Evidence for the writer classifiers. Chance level for thirteen writers is 7.7 %. The two classifiers are complementary and neither is redundant: the ink classifier is the correct instrument for real photographs and the normalised classifier for drawn output.

The residual errors of the corrected shape classifier over the 1358 real lines are scattered, not concentrated: eleven of the thirteen writers score 98.9 % or above, and the remaining loss is 48 lines for owen, 19 for dylan, 6 for abhinav, 5 each for 552 and thiya, and 1 each for 150, 153 and yeukita. The concentration on owen is the only systematic pattern left and is listed as an unsolved problem in Section 5.1.3. The ink classifier is effectively perfect on the same lines,

which is why the two are reported side by side rather than merged: the same hand is identified at 99.78 % by one instrument and 93.67 % by the other, a property of the instruments and not of the handwriting.

3.5.7 Alternatives and justification

Table 22 lists the alternatives. None of them was built, so they are arguments and not measurements.

Hand-crafted features (slant, x-height, spacing, connectedness) with nearest-profile classification are interpretable and already measured by the style-profile builder, but separate two similar hands (writer 150 and a personal writer) poorly, and no such classifier was measured; contour-direction and allograph statistics [27] are text-independent and strong for large writer sets but hand-built and less direct as a judge of constant-width output; Siamese or metric learning is open-set but needs pair mining and threshold calibration, while only a closed set of ten is required; deep fragment networks [28] scale to hundreds of writers, which are not available here; a joint text-and-writer network was tried first, but writer accuracy reached 98.9 to 100 % by epoch 20 while text accuracy stalled at 28 to 29 %, so separate single-task networks were adopted; and an ink classifier with random pen-width augmentation would reduce but not remove the thickness cue, whereas normalisation removes it by construction (no ablation was run).

The known limits are a closed set with no "unknown writer" class (the softmax cannot answer "none of them"), small validation sets, geometry cues of the personal writers that may be confounded with the collection session, and a judge that is also an objective of the generator.

3.5.8 Inference on the single-board computer

The classifiers run with the same layer functions as the recogniser (Section 3.4): convolution as one matrix product per layer, batch normalisation in the folded form of Equation 92, windowed pooling, a matrix product for each linear layer, the dropout skipped and the softmax computed after subtracting the maximum score. The training pass and the deployed pass are the same Python functions, as the Appendix describes, so no numerical comparison between a "training" and a "deployment" implementation is needed here, only the one gradient check of Section 3.4.7. The weights are read from the plain array checkpoint written by the training script, with no parsing of any third-party model format. A forward pass needs about 3.79×10^9 multiply–accumulate operations, and the parameters occupy about 3.9 MB as 32-bit floating-point numbers. **[MUST STILL BE COMPLETED BY ME: execution time of one classifier pass on the ODROID N2+.]**

3.6 Style extraction and glyph library

This subsection describes how a writer's style is learned from photographs of that writer's pages and stored permanently (functional units FU 2.5 to FU 2.7; requirements R1 and R6). "Training" a writer trains no network. A *style profile* is built once per writer: a library of the writer's own letter shapes, each reduced to a thin centre-line pen path (a *glyph*), plus measured numbers such as slant, letter height, spacing and how often letters are joined. It is one text file; afterwards no page images or networks are needed to write in that style. The builder (Figure 7) is an offline program written by the student, which uses the recogniser of Section 3.4 for inference only. Its input is the line crops and transcriptions of Section 3.3: ten pages each of eight database writers (150, 151, 152, 153, 384, 551, 552, 588; the last labelled page is held out, so no evaluation text enters a profile) and the notebook pages of the two personal writers (15 % of their lines held out), aligned with the recogniser fine-tuned on their hands.

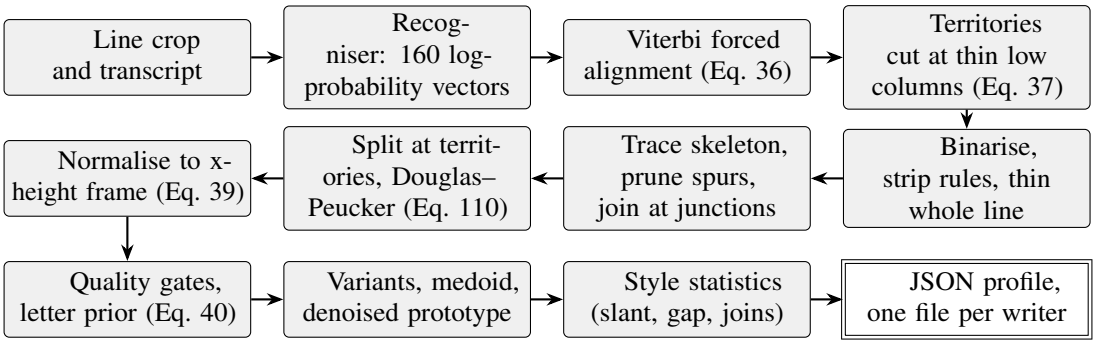


Figure 7. Flow chart of the offline style-profile builder: alignment (top), glyph extraction (middle) and profile assembly (bottom).

3.6.1 Forced alignment: where is each character?

A character in a joined line has no visible boundary and no per-character labels exist, so *forced alignment* asks which of the recogniser's $T = 160$ time steps (4-pixel slices of the canvas) belong to which character, given the known transcript. Let $y = (y_1, \dots, y_U)$ be the transcript, $y' = (\varnothing, y_1, \varnothing, \dots, y_U, \varnothing)$ the blank-augmented label of length $S = 2U + 1$ (Figure 30) and $\ell_t(k) = \ln y_t^k$ (Equation 20). Where the CTC loss (Equation 22) *sums* over all paths, the alignment keeps the single best path, found by the Viterbi recursion [12] with $\delta_t(s)$ the best log-probability of a path at position s at step t:

$$\begin{aligned} &\delta_1(1) = \ell_1(\varnothing), \quad \delta_1(2) = \ell_1(y'_2), \quad \delta_1(s) = -\infty \ (s > 2), \\ &\delta_t(s) = \ell_t(y'_s) + \max\left\{\delta_{t-1}(s),\ \delta_{t-1}(s-1),\ \delta_{t-1}^{\text{skip}}(s)\right\}, \\ &\delta_{t-1}^{\text{skip}}(s) = \begin{cases} \delta_{t-1}(s-2) & s \geq 3,\ y'_s \neq \varnothing,\ y'_s \neq y'_{s-2}, \\ -\infty & \text{otherwise.} \end{cases} \end{aligned} \tag{36}$$

The winning transition is stored as a back-pointer and the path is traced backwards from its end (Figure 33 of the appendix); character i owns the frames with $s_t^* = 2i$, with span $[a_i, b_i) = [t_{\text{first}} - 1, t_{\text{last}})$ in frame boundaries. Because the recogniser input is left-aligned with the uniform scale of Equation 13, a frame boundary converts exactly back to a pixel position in the crop (Equation 107 of the appendix). The *alignment confidence*, the mean log-probability along the path (Equation 108 of the appendix), is low if the recogniser never really read the line, so lines in the lowest quartile of a writer's own confidences are dropped; the conversion, the confidence, the drop rule and a worked example are in the appendix ("Alignment book-keeping, frame-to-pixel conversion and line selection").

Cutting the ink into territories. CTC spans are narrow slivers that lag the ink, so only their centres c_k (pixels, Equation 107) are used, with the provisional boundary $\hat{x}_k = \frac{1}{2}(c_{k-1} + c_k)$ between two characters. In cursive the only honest boundary is a ligature, where the ink is thin and low, so the boundary is snapped to

$$\begin{aligned} &\text{cost}(x) = \text{thin}(x) + 1.15\,\text{top}(x), \qquad r = \max\big(3, \text{round}(0.5 x_h)\big), \\ &x^* = \arg\min_{|x - \hat{x}_k| \leq r} \Big[\text{cost}(x) + 0.6\,\frac{|x - \hat{x}_k|}{r}\Big], \end{aligned} \tag{37}$$

where $\text{thin}(x)$ is the smoothed column ink count divided by its 60th percentile over non-empty columns, $\text{top}(x)$ the height of the highest ink above the baseline in x-heights (limited to 3), the

cost is zero in empty columns, and x_h is the x-height of Equation 33. The cuts are vertical: cutting along the slant lowered the writer identification of the synthesised output from 81.8 % to 68.6 % (shape classifier, design-stage measurement without raw log), because vertically cut glyphs retile more naturally.

3.6.2 From ink to ordered pen paths

Binarisation, core band and slant. Each line is binarised like the shape classifier's input, and the baseline and x-height x_h come from the core band of Equation 33; the binarisation recipe and the measured x-heights of the ten writers are in the appendix ("Binarisation and core-band measurement"). The *slant* is the shear that makes the strokes vertical. For ink pixels (x_i, y_i) with mean row $\bar{y}$ and trial angle θ (image y downwards),

$$x_i'(\theta) = \left\lfloor x_i - (y_i - \bar{y}) \tan\theta \right\rceil, \qquad H_\theta(c) = \#\{i : x_i'(\theta) = c\}, \qquad S(\theta) = \sum_c H_\theta(c)^2, \tag{38}$$

and $\hat{\theta} = -\arg\max_\theta S(\theta)$ over $\theta \in \{-45°, -43.5°, \ldots, 45°\}$, forward lean positive: since $\sum_c H_\theta(c)$ is fixed, $S(\theta)$ peaks when the ink falls in fewest columns.

Thinning and tracing. The whole line is thinned *once* with the Zhang–Suen algorithm [32] (Equation 83) and only then cut at the territories, because thinning each slice separately creates a spurious spine along the cut. The skeleton pixels form a graph in the 8-neighbourhood, which is walked into chains, short spurs are pruned, and a pen stroke that crosses itself (loops of e, l, o; the bar of t) appears as several chains at a junction, so chain ends that meet are merged whenever they continue nearly straight into one another; the tracing rules, their thresholds and the continuation-angle test (Equation 109) are in the appendix ("Skeleton tracing, spur pruning and junction merging"). A chain that crosses a territory boundary is split, and each crossing records that the pen did not lift between the two characters (flags connL_i, connR_i).

Douglas–Peucker simplification. Thinning leaves a pixel staircase, so each chain is simplified with the Douglas–Peucker algorithm [33], which recursively splits the chain at the vertex farthest from the chord through its end points a, b until that distance falls below the tolerance $\varepsilon = 0.6$ pixels (0.05 to 0.12 mm at 4 mm per x-height). The point-to-chord distance (Equation 110) and the recursion are in the appendix ("Douglas–Peucker distance and recursion"; Figure 34). *Implementation note:* Equation 110 is the distance to the infinite line, not clamped to the segment, so a closed loop with $a = b$ would collapse to two identical points; the effect was not quantified.

Normalisation and resampling. Every simplified point (x, y) (pixels, y down) of a chain in the territory $[L, R)$ goes to the writer-independent frame with baseline 0, y up, unit x-height and the slant removed:

$$y_n = \frac{y_b - y}{x_h}, \qquad x_n = \frac{x - L}{x_h} - \sigma\, y_n, \qquad \sigma = \tan\hat{\theta}. \tag{39}$$

The synthesiser applies the inverse shear (Equation 45), so glyphs are stored upright and reappear at the writer's lean. A cursive cut falls inside a ligature, so on a side confirmed connected the outward-running low connector is trimmed and the entry and exit points are recorded for the synthesiser's own connector. Kept variants are then resampled at constant arc length and smoothed, because the pixel staircase left by thinning would otherwise be drawn as hand tremor; the trim limit, the resampling step, the smoothing filter (Equation 111) and the measured staircase turn angles are in the appendix ("Resampling, smoothing and connector trimming").

3.6.3 Variants, quality gates and the cross-writer letter prior

Of about 2 000 to 6 200 raw instances per writer roughly 10 % survive five gates, applied per character in order: geometry, outlier rejection, the cross-writer prior described below, consensus

about the medoid of the kept descriptors, and typicality of width and height. The appendix ("Quality-gate cascade and derived glyphs") states each gate with its thresholds. *Cross-writer prior.* One writer's ten pages cannot show what a clean letter is; ten writers can. A glyph becomes a shape descriptor v_g (points along its strokes accumulated on a 12×12 grid over its bounding box, damped, blurred and normalised to unit length); the prior $\bar{v}_c$ of a character is the normalised element-wise median over all writers, and a glyph's distance is

$$\mathrm{priorD}(g,c) = 1 - v_g \cdot \bar{v}_c \in [0,1]. \tag{40}$$

The 70 % nearest are kept; the prior only *rejects* malformed variants and later gates substitution, and never imports another writer's shapes. Two derived glyphs are also stored: the *medoid*, the real variant closest to all others (the writer's own "ideal" letter), and a *denoised prototype*, a median curve of the cleanest group of variants; their construction and the condition for storing the prototype are in the same appendix paragraph. Figure 35 of the appendix shows stored variants.

3.6.4 Style statistics and permanent storage

Besides glyphs the profile holds numbers measured over the kept lines (Table 25 lists every key): slant, x-height, stroke width, typical ascender and descender reach, the median advance of each character, the word gap, and the *connectedness*

$$c = \frac{\#\{\text{adjacent letter pairs with connR}\}}{\#\{\text{adjacent letter pairs}\}}. \tag{41}$$

The word gap is measured on the *real* ink (median of ink-free column runs wider than $0.5x_h$), because CTC gives a space only a narrow slice, and the slant and reach of the real lines are stored for the calibration of Section 3.7.2. The ten writers differ widely (Table 26): slant from $-10.5°$ to $42°$, connectedness 0.21 to 0.98, word gap 0.92 to 2.68 x-heights, the diversity requirement R1 asks the system to reproduce. The legibility parameter λ (Section 3.7) is written into the file after the build, not by the builder. *Permanent storage (R6):* a profile is a JavaScript Object Notation (JSON) text file of 322 to 457 kB (about 3.6 MB for all ten), loaded when the server starts (Section 3.10); ten profiles exist and the web interface lists them. **[MUST STILL BE COMPLETED BY ME: the demonstration of R6 (display of the stored writers, reproduction in a writer chosen by the examiner) and the storage device holding the files.]**

3.6.5 Design alternatives and justification

Table 33 of the appendix lists the alternatives. Fixed widths, projection minima or hand labelling would replace the forced alignment, but joined letters have no gaps, hand labelling of about 90 pages is impractical, and the recogniser already knows the text. One averaged letter per writer blurs letters whose strokes are arranged differently, whereas variants keep the natural variation; importing the median shape into every writer would erase the writer, whereas filtering keeps the shapes the writer's own; and a generator learned from on-line pen data is impossible, because no such data exist for the ten writers (47 to 138 usable lines each). Cutting a joined line destroys how a letter changes with its neighbours, so the most joined hands (writers 151 and 551) are reproduced least legibly (Section 3.7). A real example of the builder stages is still to be supplied (Figure 36 of the appendix).

3.7 Handwriting synthesis

This subsection describes how a text is converted into pen paths in the style of a stored writer (functional units FU 2.7 and part of FU 2.5; requirements R1 and R3). The synthesiser takes the text, a style profile (Section 3.6), the x-height size x_{mm} in millimetres (4 mm by default) and a

random seed, and returns a *trajectory*: pen-down polylines in millimetres, y upwards, origin at the first baseline. The construction is random, so a search over several candidates (Section 3.7.3) keeps the one that the text recogniser (Section 3.4) and the stroke-normalised writer classifier (Section 3.5) jointly prefer. Geometry stays in x-height units until one multiplication at the end, so one profile serves any pen size.

3.7.1 Global quantities and glyph choice

Let θ_{p} be the slant, c the connectedness, A and $D < 0$ the ascender and descender reach and g the word gap of the profile, and $\lambda \in [0,1]$ the writer's legibility parameter (below). The synthesiser derives

$$\theta_{\mathrm{cap}} = 45^\circ - 19^\circ\lambda, \quad \sigma = \tan\big(\mathrm{clip}(\theta_{\mathrm{p}}, \pm\theta_{\mathrm{cap}})\big) + \Delta_\sigma, \quad p_{\mathrm{j}} = \min(c,1), \quad L = (A - D + 0.85)\,x_{\mathrm{mm}}, \tag{42}$$

with the shear coefficient σ (calibration shift Δ_σ, Section 3.7.2), the join probability p_{j} and the line pitch L (12.6 mm for $A = 1.7$, $D = -0.6$, $x_{\mathrm{mm}} = 4$ mm). The connectedness is a *probability*: as a threshold it joined every pair for every writer above it and fused words into blobs.

For each character the synthesiser tries, in order, the writer's own variants, a borrowed shape (other case or similar character), the *anchor* glyph, and finally a hand-designed skeleton or a blank. Variants are ranked by priorD (Equation 40); only the five best are eligible, and rank $i = 0, \ldots, 4$ is drawn with probability proportional to

$$w_i = \frac{1}{1 + 1.4\,i} \exp\Big[-\Big(\frac{\mathrm{entryY}_i - \hat{e}}{0.45}\Big)^2\Big], \tag{43}$$

where the exponential factor applies only when the glyph is joined to a previous letter and $\hat{e}$ is the pen's exit height above the baseline in x-heights: a variant whose entry height matches the pen avoids tangled joins. The rank probabilities without that factor are listed in the appendix ("Variant draw probabilities without the entry-height factor"). A writer forms a letter consistently, and restricting the choice to typical variants gave 80.5 % writer identification with the two best variants against 75.5 % over all twelve (design-stage measurement, no raw log). The *anchor* is a legible stand-in: the writer's own medoid if it is close enough to the prior, otherwise a hand-designed single-stroke skeleton, joined or print according to the writer's connectedness, re-proportioned to the writer's reach; the prior limit, the choice rule and the re-proportioning maps are in the appendix ("Construction of the anchor glyph"). The print skeleton reads back at about 96 % character accuracy through the recogniser against about 89 % for the cursive skeleton (code comments, no raw log), so it is the legibility floor.

The role of λ. The parameter does *not* interpolate geometrically between the writer's glyph and the anchor; it scales the probability that a chosen variant with prior distance d is *replaced* by the anchor,

$$P_{\mathrm{swap}}(\lambda, d) = \begin{cases} 1 & d \geq 0.55, \\ 0 & \lambda \leq 10^{-3}, \\ 0.06\,\lambda & d \leq 0.12, \\ \mathrm{clip}\big(\lambda\,(0.55 + 0.9\,b), 0, 1\big), \; b = \mathrm{clip}\big(\frac{d - 0.12}{0.43}, 0, 1\big) & \text{otherwise}, \end{cases} \tag{44}$$

and it lowers the cap on the writer's own slant (Equation 42). Clean glyphs are almost always kept, garbage is always replaced, and in between replacement grows with λ (at $\lambda = 1$: 0.74 for $d = 0.21$, 0.99 for $d = 0.33$), so the sources are mixed probabilistically: with the stored values the share of anchor glyphs in a design-stage probe ranged from 0 % (writer 552, $\lambda = 0$) to 76 % (writer 151, $\lambda = 1$). λ is set per writer (Table 26) by an automatic search on a novel corpus followed by manual adjustment against the evaluation script, so it is tuned on the sentences of

Section 4. *Implementation note:* a geometric blend toward the anchor and a "soft archetype" proposal appear only in the project notes, not in the delivered code.

3.7.2 Placement, joins and randomness

A chosen glyph (x, y) (x-height units, upright) is placed at the pen position (P_x, P_y) (mm) by

$$X = P_x + (\ell + x + \sigma_g\, y)\, x_{\mathrm{mm}}, \qquad Y = P_y + (y + \delta_g)\, x_{\mathrm{mm}}, \tag{45}$$

with the glyph's lead ℓ, shear σ_g and baseline drift δ_g, inverting the de-slanting of Equation 39 (Figure 37 of the appendix). The writer's own glyphs use σ; anchor glyphs are limited to $\pm\tan 30°$ (cursive) or $\pm\tan 22°$ (print), because a 40° shear makes even a clean letter collide. A guard shifts glyph and pen right if the glyph would come too close to the previous ink, and the pen advances by the glyph's *own* advance, never the character median. After a word the pen advances by a randomised word gap, and a word that would cross the line width starts a new line at the pitch L; the guard distance, the gap randomisation, the line width and the resampling step below are in the appendix ("Advance, word spacing and line breaking"). Adjacent letters are joined if both are alphabetic, both are the writer's own glyphs or both the cursive anchor, and a random $u < p_{\mathrm{j}}$ (a print anchor is never joined through). The connector from exit p_0 to the next entry p_1 is

$$\begin{aligned} &\mathrm{path}(p_0, m, p_1), \qquad m = \left(\tfrac{p_{0x}+p_{1x}}{2},\ \min(p_{0y}, p_{1y}) - \mathrm{sag}\right), \\ &\mathrm{sag} = \min\left(0.11\, x_{\mathrm{mm}}, 0.16\,|\Delta x|\right) + \mathcal{N}\left(0, (0.015\, x_{\mathrm{mm}})^2\right), \end{aligned} \tag{46}$$

with $\Delta x = p_{1x} - p_{0x}$; a fixed dip would turn a short join into a loop. A joined run is one pen-down polyline, so the pen lifts only where the writer lifts, and every polyline is resampled at constant arc length for even plotter motion. One pseudo-random stream, seeded by the seed argument, supplies every draw, so the arguments fix the output completely. Each glyph receives a random rotation, scaling and translation of amplitude J (0.15 by default, 0.5 in the search) and the baseline drifts as a slow first-order walk; the jitter model (Equation 112) and the drift constants (Table 27) are in the appendix, and at $J = 0.5$ the translation spread is below 0.05 mm, the rotation 1.1° and the scale 2.5 %.

Style calibration. A line of separate glyphs does not automatically reproduce the line-level slant and reach of the real lines (writer 150's ascenders came out about 7 % too tall, writer 551's slant about 9° too steep). A probe sentence is therefore synthesised once per writer and size, measured with the estimators of Section 3.6, and three fixed-point iterations (Equation 93 of the appendix) adjust the shear shift Δ_σ and the scale factors K_A, K_D of the writer's own tall and deep strokes until the probe matches the stored real-line values. The same estimator measures both sides, so the agreement of slant and reach is partly self-fulfilling and not independent evidence.

3.7.3 Best-of-N search with joint scoring

Single draws vary in quality, so the line is drawn N times and the best candidate kept. Selecting by text accuracy alone raised legibility but lowered writer identification for some writers (Table 41): the most legible draw need not be the most characteristic. The score therefore uses both judges. For a rendered candidate I, with $\hat{y}(I)$ the greedy CTC reading (Equation 25) and y the intended text,

$$a(I) = \max\left(0, 1 - \frac{\mathrm{Lev}(\hat{y}(I), y)}{|y|}\right), \quad w(I) = \mathrm{softmax}\left(f_{\mathrm{wid}}(I)\right)_{a^\star}, \quad S(I) = \frac{2\,a(I)\,w(I)}{a(I) + w(I)}, \tag{47}$$

where Lev is the Levenshtein distance (Equation 81), f_{wid} the shape classifier after stroke normalisation (Section 3.5.3), $a^\star$ the target writer, and $S = 0$ if $a + w \le 10^{-9}$. The harmonic mean is dominated by the smaller term, so a candidate cannot win on one criterion while failing the

other, and w is a probability, not a hit-or-miss flag.
The search renders and scores N candidates, each from its own seed, and keeps the one with the highest S; the best candidate then goes through a *repair* pass, which forced-aligns the intended text to the recogniser output, ranks the characters by the mean own-class log-probability the recogniser gives them (Equation 113), pins the two worst to the anchor and re-synthesises the whole line. The step list, the seeds, the render settings, the early-stop thresholds and the number of repair rounds are in the appendix ("Steps of the joint best-of-N search", Figure 50). A repaired line is accepted only if the joint score strictly exceeds the best so far, so a repair that fixes a letter but makes the line look like another writer is rejected (a text-only search with a plain repair that accepts on text score alone was measured to cost writer identification). An order-statistics argument (Equation 114 of the appendix) gives a gain that grows roughly logarithmically in N for independent candidates; the seeds share a profile, so independence is only approximate, and no per-N curve was kept. *Implementation notes.* (i) The repair re-uses seed base, so it is a fresh draw with pins, not a patch of the winner. (ii) The pin set is updated only on acceptance, so after a rejection the next round re-synthesises an identical candidate and the remaining rounds are wasted work. (iii) In three probe runs (writers 150, 153, 551, $N = 14$) no repair was accepted, so the repair pass rarely changes the result.

Values of N and cost. N is 20 as the web default and 30 in the official evaluation script; the remaining values, the cost model (Equation 115) and the operation count of a network pass are in the appendix ("Values of N, cost of a candidate and measured timings"). A candidate costs one synthesis, one rendering, one recogniser pass and one classifier pass. Laptop timings are in Table 30 of the appendix: synthesis takes 11 to 16 ms per candidate and the two judges 0.5 to 4 s, dominated by thinning the full-resolution render, so the search at $N = 14$ took 23 to 137 s; times on the single-board computer were not recorded.
The measurements that motivated these settings, and the reasons they cannot stand as a verification of the block, are reported in the Appendix (Table 41).

3.7.4 Design alternatives and justification

Table 34 of the appendix lists the alternatives. A recurrent sequence generator with attention and mixture-density output [29] needs on-line pen data (hundreds to thousands of lines per writer), whereas only 47 to 138 scanned lines per writer exist; a from-scratch version trained on skeleton-derived strokes learned slant, roundness and connectedness but no legible letters and was rejected. Adversarial and diffusion generators of line images [30, 45] need less data, but their output is a raster needing the same thinning to vectorise, size, slant and spacing cannot be controlled, and a trial took 35 to 45 s per line on a laptop processor against 3 to 8 s for glyph synthesis. The glyph library needs about nine pages per writer and no training, outputs pen strokes directly, exposes size, slant, joins and gap as numbers, traces a failure to a single glyph and has a fallback for every glyph; its weakness is joined hands, whose joins are lost by cutting. Glyph assembly from one writer's sample [31] is a related approach. Synthesis costs about 15 ms, so only the optional search is expensive, suiting a single-board computer without accelerator.

3.8 G-code generation

This subsection describes how a trajectory in millimetres becomes a machine program (functional unit FU 2.8, first stage; requirements R3 and R5). The format is G-code, the standard line-oriented command language of plotters, because it separates the two stages of the motion chain: the generator needs no knowledge of the gantry electronics, the interpreter of Section 3.9 none of handwriting, and the drawing can be checked on a computer without the machine. Both programs

were written by the student; the file format itself was not designed.

3.8.1 Coordinate frames, placement and ordering

The trajectory (Section 3.7) and the machine frame both have X to the right and Y upwards, so no mirroring is needed (Figure 39 of the appendix). The origin $(0,0)$ of the G-code is the safe corner 5 mm inside the minimum switches (Section 3.9.9), and the text block is placed with its top-left corner at $(X_\mathrm{o}, Y_\mathrm{o}) = (10, 180)$ mm, later lines descending. For a trajectory with bounding box (x_0, y_0, x_1, y_1), width w and height h, the block is shrunk (Equation 48), never enlarged, if it would leave the work area $[0, X_\mathrm{max}] \times [0, Y_\mathrm{max}]$, $X_\mathrm{max} = Y_\mathrm{max} = 200$ mm:

$$s = \min\!\left(1,\ \frac{X_\mathrm{max} - X_\mathrm{o}}{w},\ \frac{Y_\mathrm{o} - Y_\mathrm{min}}{h}\right), \qquad X = X_\mathrm{o} + (x - x_0)\,s, \quad Y = Y_\mathrm{o} - (y_1 - y)\,s, \tag{48}$$

with $Y_\mathrm{min} = 0$. At scale 1 the area holds at most $\lfloor (180 - 9.2)/12.6 \rfloor + 1 = 14$ lines of the default size ($9.2 = (A - D)x_\mathrm{mm}$, pitch 12.6 mm). A final clamp moves vertices outside the work area onto its border and writes a header warning, and vertices closer than 0.12 mm to the previous one are dropped (inert in practice, since the synthesiser resamples at 0.35 mm). The strokes are then ordered by a greedy nearest-endpoint rule: they are grouped into text rows and, within a row, the next stroke is the one whose end is nearest to the pen, entered from that end. Only order and direction change, so the ink is identical; this is the pen-movement optimisation of R3, whose saving was not measured. *Implementation note:* the rows are sorted by ascending Y, so a multi-line job starts with the *bottom* line, not in reading order; this was found by reading the code and was not checked on the machine.

3.8.2 Pen state, feed rates and program structure

The pen lift is a one-way toggle motor (Section 3.9.10), so the generator tracks the pen state in software and emits a command only when the state changes; a pulse where none is needed would invert the pen for the rest of the job. Each change emits `M3` (down) or `M5` (up) and a dwell of $(260 + 90)$ ms = 0.350 s (the assumed time of one 90° rotation and settling), so a stroke costs 0.7 s of dwell; the interpreter additionally confirms the state with the pen switch. Drawing uses 900 mm/min (15 mm/s) and travel 2400 mm/min (40 mm/s); Table 28 of the appendix lists all constants. A program consists of a comment header, `G21` (millimetres), `G90` (absolute), then for each stroke of at least two points a travel `G0` to its start, pen down, `G1 F900` and one `G1` per vertex with three decimals (1 µm), and finally pen up, parking and `M2`; an excerpt is shown in Figure 41 of the appendix (vertices 0.35 mm apart). *Implementation note on feed rates:* the interpreter has one *modal* feed register and no rapid mode (Section 3.9). The generator emits `G0 F2400` once at the start and `G1 F900` before every stroke, so every pen-up travel after the first stroke runs at 900 mm/min, not 2400; the real job time is longer than the estimate below assumes, and emitting the feed before every travel would remove the discrepancy.

3.8.3 Time model

The generator can write a step schedule with a time estimate that assumes each move starts and ends at rest with acceleration $a = 400$ mm/s^2 and cruise speed v (15 mm/s drawing, 40 mm/s travel). The distance needed to reach v is $d_a = v^2/(2a)$; a move of length d is *trapezoidal* if $d \geq 2d_a$ and *triangular*, never reaching v, otherwise (Figure 42 of the appendix):

$$T(d) = \begin{cases} \dfrac{2v}{a} + \dfrac{d - 2d_a}{v} = \dfrac{d}{v} + \dfrac{v}{a}, & d \geq 2d_a, \\[2ex] \dfrac{2v_\mathrm{p}}{a} = 2\sqrt{\dfrac{d}{a}}, \quad v_\mathrm{p} = \sqrt{d\,a}, & d < 2d_a. \end{cases} \tag{49}$$

For drawing $d_a = 0.28$ mm (moves below 0.56 mm are triangular, ramp time 37.5 ms), for travel $d_a = 2$ mm (below 4 mm, 100 ms). A typical 0.35 mm segment is always triangular, $T = 2\sqrt{0.35/400} = 59.2$ ms, an average of 5.9 mm/s, not 15. For the test word of Figure 41 (draw path 83.2 mm in 239 moves, pen-up path 250.3 mm in 14 moves of which 179.6 mm is the first approach, 26 pen pulses) the model gives 14.1 s drawing, 7.5 s travel and 9.1 s dwell, 30.7 s in total, or 2.8 s per character if the word has 11 characters (assumed from the file name; the text is not stored in the file). This is a model, not a measurement: it stops fully between 0.35 mm segments and assumes 40 mm/s travel, so it overestimates drawing time and underestimates travel time against the real interpreter (Section 3.9.7).

3.8.4 Verification of the program

An independent parser replays the file (it follows X and Y, starts a polyline at `M3` and closes it at `M5`; only `G0` and `G1` are parsed, which suffices because nothing else is emitted). The original trajectory (after placement and clamping) and the replayed polylines are rasterised on a common grid (8 pixels per millimetre), and the measures of Equation 50 are reported:

$$\mathrm{IoU} = \frac{|M_\mathrm{t} \cap M_\mathrm{g}|}{|M_\mathrm{t} \cup M_\mathrm{g}|}, \qquad e_\mathrm{mean} = \frac{1}{J}\sum_j \min_i \|g_j - s_i\|, \qquad e_\mathrm{max} = \max_j \min_i \|g_j - s_i\| \tag{50}$$

where M_t and M_g are the masks, g_j the G-code vertices and s_i the trajectory vertices (subsampled to at most 4000 and 6000). The measure compares vertices with vertices and covers only the loss between trajectory and file (rounding, ordering, clamping), not the mechanical error (Section 3.9.7). The project notes quote IoU ≈ 0.99 and a text-accuracy change of at most 0.3 percentage points between direct and read-back rendering (an earlier system state; Table 41 shows later pairs), judged by the networks of the search; e_mean and e_max were not logged. **[MUST STILL BE COMPLETED BY ME: run the comparison for a set of generated programs and report** IoU, e_mean, e_max**.]** *Implementation note on the work area:* lines wrap at 185 mm from $X_\mathrm{o} = 10$ mm, so a full-width line reaches $X = 195$ mm, but the driver clamps X to 194 mm and would distort the last millimetre; X_max or the line width should be reduced.

3.9 Gantry control and hardware

This subsection describes the motion-control program and the hardware it drives (functional units FU 2.8 and FU 3.1 to FU 3.4; requirement R5). The single-board computer generates the step and direction pulses of both stepper drivers itself on its general-purpose pins, with no microcontroller in between. A move handed to a microcontroller cannot be interrupted without a round trip, whereas here the loop that emits every step also reads the end-stop switches, so "stop at once on any trigger" is a property of the program; an earlier design with a microcontroller on a serial link was superseded for this reason. The interpreter (about 800 lines of Python), the calibration and the safety logic were written by the student; the only library used gives access to the pins.

3.9.1 Components, and why each was chosen

Table 7 lists the electrical and mechanical components and the reason each was selected. Every one of them was bought rather than designed, so no credit is claimed for the parts themselves; what is claimed is the selection, the interface circuits around them and the control program, which are the subject of the rest of this subsection. The pin assignment on the character device `/dev/gpiochip0` is listed in Table 35 of the appendix, and Table 31 and Figure 40 there separate what the project files evidence, including the transmission values the program assumes,

from what must still be supplied.

MUST STILL BE COMPLETED BY ME

Add to Table 7 the exact part numbers and the supplier of the motors, the driver carrier boards, the power supply and the camera, together with the power supply's current or VA rating, the camera's model and resolution, and the height at which the housing holds it above the page. Also state how the 24 V rail is converted to the voltages the ODROID N2+ and the pen motor need, because the board itself is not a 24 V part and no converter is documented in the project files.

3.9.2 Mechanical arrangement

The machine is a Cartesian gantry of two axes. The lower axis carries the whole upper axis and is driven by one motor whose shaft is joined, through the 5 mm to 8 mm coupler, to an 8 mm cross-shaft spanning the frame. That one shaft turns a pulley at each end, so both sides of the moving bridge are driven from the same motor and advance by the same amount; this is what keeps the bridge square to the frame without a second motor and without the electronic synchronisation two motors would need. The second motor rides on that bridge and drives a single pulley, which moves the carriage along the bridge. The pen-lift motor and its cam ride on that carriage, so the pen is the last body in the chain and the only mass that has to be moved by the smaller of the two belt loops. All moving parts run on linear bearings over chromed steel rods, and each belt is tensioned against an idler pulley.

Putting the second motor on the moving bridge is the main mechanical trade-off of the design: it keeps each axis to one motor and one belt, but the lower axis then has to accelerate the upper motor's mass as well as the carriage, which lowers the acceleration that axis can reach before it loses steps. The alternative arrangement that was considered, and the reason the simpler one was kept, are given in the appendix under "supporting material for the gantry and motion control".

The torque each motor must supply follows from the pulley's pitch radius. For a belt of pitch p on a pulley of N_t teeth the travel per revolution and the pitch radius are

$$\ell_{\mathrm{rev}} = p\,N_t = 2 \cdot 20 = 40 \text{ mm}, \qquad r = \frac{\ell_{\mathrm{rev}}}{2\pi} = \frac{40}{2\pi} = 6.37 \text{ mm}, \tag{51}$$

From the pitch radius of Equation 51, the torque needed to accelerate a carriage of mass m at acceleration a against a friction force F_f is

$$T = r\,(m\,a + F_f). \tag{52}$$

Equation 52 is the check that decides whether the motors can follow the step rates of Section 3.9.7 without stalling, and it is the calculation this design is missing: neither the carriage masses nor the friction force was measured, and the motors' pull-out torque at the rates used was not obtained, so no torque margin can be stated. This is the reason the program uses no acceleration ramp and why Section 3.9.7 can only warn that abrupt starts may lose steps.

[MUST STILL BE COMPLETED BY ME: Weigh the two carriages and measure or estimate the friction force, then evaluate Equation 52 at the 400 mm/s^2 of Equation 49 and compare the result with the pull-out torque of the motors at 1200 and 3200 steps per second taken from their torque-speed curve. State the resulting margin. Also insert the CAD drawings of the gantry and the camera housing and the photographs of the built machine (Figure 40); the folder intended for them in the project files is empty, so nothing could be reproduced here.]

3.9.3 Stepper driver and the coil-current setting

Each driver is wired as in Figure 8: the 24 V rail and the bulk capacitor to the motor supply pins, the four coil wires to the two output pairs, and STEP and DIR to two pins of the single-board computer. The A4988 regulates the coil current by chopping, and the current it regulates to is set by a reference voltage on a trimmer together with the two current-sense resistors of the carrier board,

$$I_{\text{coil}} = \frac{V_{\text{ref}}}{8\,R_S} \qquad \Longrightarrow \qquad V_{\text{ref}} = 8\,R_S\,I_{\text{coil}}. \tag{53}$$

For the 1.68 A rating of these motors and the 0.1 Ω sense resistors most commonly fitted to these carriers, Equation 53 asks for $V_{\text{ref}} = 8 \cdot 0.1 \cdot 1.68 = 1.34$ V. The motors should not in fact be run at their full rating on this driver: an A4988 carrier without a heatsink or forced air is usually held to about 1 A per phase, because the regulated current flows through the driver's output transistors and the heat it produces, $P \approx 2 I_{\text{coil}}^2 R_{DS(\text{on})}$ per driver, has only the small carrier board to leave through. Setting V_{ref} for roughly 1 A, that is about 0.8 V on a 0.1 Ω board, trades holding torque for a driver that does not go into thermal shutdown part way through a page, and losing a step to an overheating driver costs more accuracy than the lower torque does.

[MUST STILL BE COMPLETED BY ME: Confirm three things about Equation 53, because the setting cannot be read back from the program. First, the sense-resistor value actually fitted to these carrier boards (0.05, 0.068 and 0.1 Ω are all sold) — read it off the board or measure it. Second, the voltage the trimmer is actually set to, measured at the reference pin with a multimeter, and the coil current that Equation 53 then gives. Third, whether the drivers have heatsinks or a fan, and whether a thermal shutdown was ever observed during a long job. State the resulting coil current as a fraction of the 1.68 A rating.]

3.9.4 Switch inputs

Figure 9 shows one switch input; all five are identical. The switch connects the pin to 3.3 V when it closes, and a 10 kΩ resistor to ground holds the pin low when it is open, so the input is defined at all times. The current drawn while a switch is closed and the worst case with all four end-stops closed are

$$I_{\text{pull}} = \frac{V_{DD}}{R} = \frac{3.3}{10\,000} = 0.33\ \text{mA}, \qquad 4 I_{\text{pull}} = 1.3\ \text{mA}, \tag{54}$$

which is negligible against anything else on the board. The choice of 10 kΩ is bounded from both sides. Making it much smaller wastes current for no benefit; making it much larger lets the pin's own leakage current I_{leak} lift the open-switch voltage, and to keep that below the input's low threshold V_{IL} the resistor must satisfy $R \le V_{IL}/I_{\text{leak}}$. The resistor also forms a low-pass filter with the pin and cable capacitance, with time constant $\tau_{RC} = R\,(C_{\text{in}} + C_{\text{cable}})$; for 10 kΩ and any plausible stray capacitance of tens to hundreds of picofarads this is a few microseconds, which is more than three orders of magnitude below the 6 ms between the repeated reads of Section 3.9.10, so the resistor plays no part in the debounce behaviour and the debounce is entirely a property of the program.

The electrical weakness of this arrangement is the return path rather than the resistor. The motors have no clamp diodes and share a ground return with the switch wiring, so each time a coil is switched its current has to find a path back through that shared conductor. A current I returning through a ground conductor of resistance R_{gnd} raises the local ground by $I\,R_{gnd}$, and the switch inputs are referred to that same ground, so the apparent switch voltage is offset by the same amount. This is the mechanism behind the behaviour recorded in Section 3.9.10, where an input

was found to read high for several milliseconds after a motor switched, and it is why a single re-read was not enough and three reads 6 ms apart were needed. The correct fix is electrical, not a longer debounce: a clamp diode across each motor, and a switch return brought back to a single ground point separately from the motor current.

[MUST STILL BE COMPLETED BY ME: For Equation 54, take V_{IL} and the input leakage current from the ODROID N2+ documentation and state the resulting upper bound on the pull-down resistor, so the choice of 10 kΩ is bounded by numbers rather than by argument. Confirm also that each switch is wired to 3.3 V and not to 5 V, since the program reads the inputs as active-high at 3.3 V.]

3.9.5 Pen-lift motor and its switching stage

A general-purpose output pin of the single-board computer can source only a few milliamperes, which is far below what even a small geared motor draws, so the pen motor is switched by a transistor on the low side of the motor as in Figure 9, with the pin driving the base through a 1 kΩ resistor. With the pin at 3.3 V and the base-emitter drop of a saturated small-signal transistor taken as 0.7 V, the base current is

$$I_B = \frac{V_{\text{pin}} - V_{BE(\text{sat})}}{R_B} = \frac{3.3 - 0.7}{1000} = 2.6\ \text{mA}. \tag{55}$$

The BC337-40 belongs to the −40 gain group, whose minimum current gain is 250. For the transistor to be held in saturation rather than used as an amplifier, the base must be driven harder than the collector current strictly requires, by an overdrive factor k_{od} of three to five, so the collector current the stage can switch is

$$I_{C(\text{max})} = \frac{h_{FE(\text{min})}\, I_B}{k_{od}} = \frac{250 \cdot 2.6\ \text{mA}}{k_{od}} = \frac{650\ \text{mA}}{k_{od}}, \tag{56}$$

which is 217 mA at $k_{od} = 3$ and 130 mA at $k_{od} = 5$. The stage is therefore comfortable for a motor drawing up to about 200 mA and would leave saturation, and begin to dissipate real power, somewhere above that. The dissipation in the transistor while saturated is $P = V_{CE(\text{sat})} I_C$, which at 200 mA and a saturation voltage of about 0.7 V is 0.14 W, well inside what the TO-92 package can lose without a heatsink.

Two things about this stage are not right, and both are stated here rather than in the discussion because they are design faults and not merely untested items. First, there is no clamp diode across the motor. When the transistor turns off, the energy $\tfrac{1}{2} L I^2$ stored in the motor's inductance has nowhere to go and appears as a reverse voltage across the transistor, which both stresses the transistor and injects the noise that Section 3.9.4 describes. A diode across the motor, cathode to the supply, rated for at least the motor current and at least twice the motor supply voltage, is the standard remedy and was omitted. Second, the base resistor leaves the base floating while the pin is in its power-up input state, so a pull-down from base to emitter should be fitted to guarantee the motor is off before the program claims the pin.

[MUST STILL BE COMPLETED BY ME: Measure the pen motor's free-running current and its stall current, then check them against Equation 56: if the stall current is above about 200 mA the stage is not hard saturated and the base resistor should be reduced. State the voltage the pen motor is actually run from. Confirm whether a clamp diode and a base pull-down were in fact fitted, since neither appears in the project files; if they were not, this subsection is correct as written and the omission belongs in Section 5.]

3.9.6 Supply and decoupling

The 24 V rail feeds both drivers, and a 100 µF capacitor is fitted across it close to them. Its job is to supply the switching current locally, because the chopper in each driver draws current in bursts that the supply leads, which have inductance of their own, cannot follow. For a current step I lasting Δt the rail sags by

$$\Delta V = \frac{I\,\Delta t}{C}, \tag{57}$$

so a 1.68 A step lasting 10 µs drawn from 100 µF costs $1.68 \cdot 10 \times 10^{-6} / 100 \times 10^{-6} = 0.17$ V, which is small against 24 V and comfortably clear of the driver's undervoltage threshold. Two gaps remain. The 100 µF is shared between the two drivers, whereas the usual recommendation is a bulk capacitor of this order at each driver; and no small ceramic capacitor on the logic supply of either driver is documented, which is the part that suppresses the fast edges rather than the bulk current. The total supply current also has not been budgeted: it is the sum of two motors at two phases each, the pen motor, the single-board computer and the camera, and without the supply's rating the margin is unknown.

[MUST STILL BE COMPLETED BY ME: Give the power supply's voltage and current rating and complete the power budget: two motors at two phases each at the coil current set in Section 3.9.3, plus the pen motor, the ODROID N2+ and the camera. State the margin that remains. Confirm whether each driver has its own bulk capacitor and whether a ceramic capacitor is fitted on either driver's logic supply.]

A separate check belongs here because it bears on the only hardware fault in this report. The driver's STEP, DIR and enable inputs are driven straight from 3.3 V pins, with no level shifting. Whether that is correct depends on the voltage the driver needs to see as a logic high, V_{IH}, which is specified relative to the driver's own logic supply: the design is sound if $3.3\,\text{V} > V_{IH}$ with margin, and marginal if it is not. During commissioning the X axis drove to its first end-stop but the direction pin was not observed to change level on reversal, and that fault was not resolved (Section 4). A level incompatibility is one candidate explanation, but a wiring or configuration fault, a damaged pin and a program fault are equally consistent with what was seen, and no measurement was taken that would separate them. No cause is claimed here.

MUST STILL BE COMPLETED BY ME

Resolve the X-axis direction fault, which currently blocks requirement R5 entirely. Take V_{IH} for the driver's logic inputs from its datasheet and compare it with 3.3 V to settle whether level shifting is needed. Then put a meter or a scope on the DIR pin at the pin and again at the driver, command a reversal, and record whether the level changes at each point. That single measurement separates a program fault from a wiring fault from a level-compatibility fault, and nothing further can be said about R5 until it is done.

3.9.7 Step generation, resolution and speed

A driver advances the motor one microstep per pulse at STEP, in the direction given by DIR. With 200 full steps, 16 microsteps per step and $\ell_{\text{rev}} = 40$ mm per revolution (assumed values),

$$\kappa = \frac{200 \cdot 16}{\ell_{\text{rev}}} = \frac{3200}{40} = 80 \text{ steps/mm}, \qquad \Delta = \frac{1}{\kappa} = 12.5\ \mu\text{m}, \tag{58}$$

and after calibration κ is replaced by the measured value of each axis. Targets are converted to *absolute* integer step counts, $n_{\text{target}} = \text{round}\big((x+5)\kappa\big)$ (5 mm edge offset, Section 3.9.9), and the step difference is taken against the tracked integer position, so rounding errors do not accumulate.

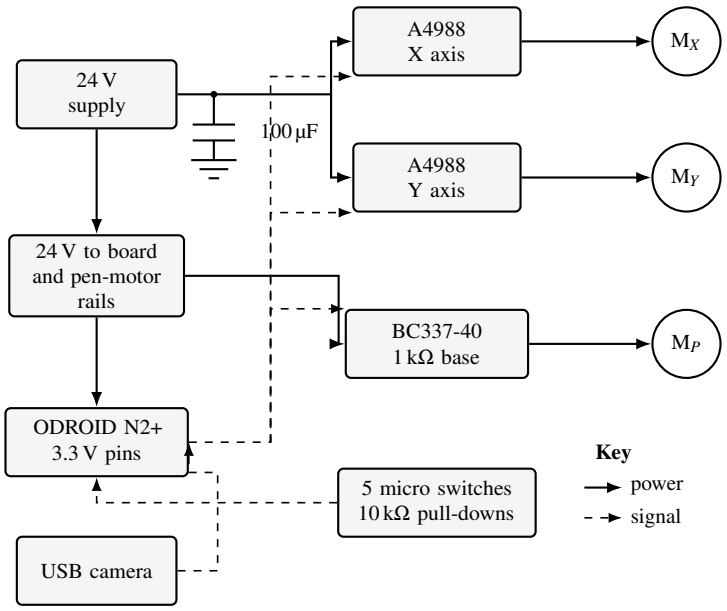
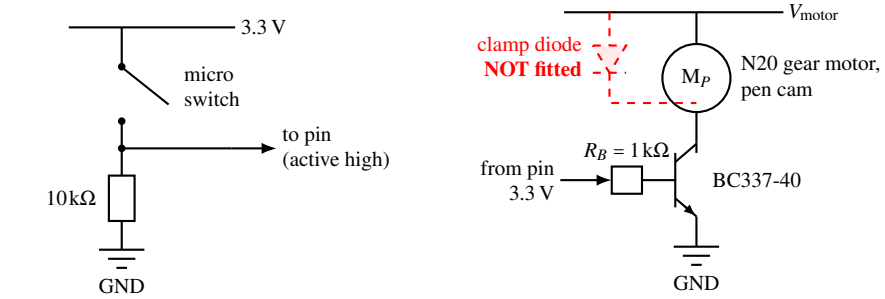


Figure 8. Wiring of the electronics. The 24 V rail feeds both stepper drivers and is decoupled to ground by the 100 µF bulk capacitor of Equation 57; the single-board computer drives STEP and DIR of each driver and the base of the pen-motor transistor, and reads the five switches. The conversion from 24 V to the voltages the board and the pen motor need is not documented in the project files.



(a) one switch input, five identical (b) pen-motor switching stage

Figure 9. The two interface circuits designed around the bought parts. In (a) the switch pulls the pin to 3.3 V when it closes and the 10 kΩ resistor holds the pin at ground when it is open, drawing the 0.33 mA of Equation 54 only while closed. In (b) the pin drives the base through 1 kΩ, giving the 2.6 mA of Equation 55 and the collector current of Equation 56. The clamp diode shown dashed is the standard protection against the motor's inductive turn-off; it is not fitted in the built circuit, and its absence is the source of the switch-input disturbance described in Section 3.9.4.

The quantisation error is at most $\Delta/2 = 6.25\ \mu$m per axis and $\sqrt{2}\Delta/2 = 8.8\ \mu$m combined, 57 times finer than the 0.5 mm of R5. This is only the *digital* resolution: the accuracy of a mark is set by belt stretch, backlash, missed steps (open loop, no encoders), switch repeatability and pen compliance, none of which was measured. **[MUST STILL BE COMPLETED BY ME: measure the positional error for R5 (the same word drawn twice and overlaid) and the return-to-zero repeatability.]**
For a move of length d at feed v_f (mm/s) with $N_{\text{maj}} = \max(|n_x|, |n_y|, 1)$ steps on the major axis the delay of every pulse is

$$\tau = \max\Big(\frac{d/v_f}{N_{\text{maj}}},\ 2\,\mu\text{s}\Big), \qquad v_f = \max(F/60,\ 0.1), \tag{59}$$

with F the feed in mm/min. For an axis-parallel move, 900 mm/min gives $\tau = 1/(15\cdot 80) = 833\ \mu$s (1200 steps/s) and 2400 mm/min gives 312.5 µs (3200 steps/s, 60 rpm); homing and retracting use 0.8 ms high and 0.8 ms low, 625 steps/s or 7.8 mm/s. The real rate is limited by the program (four switch readings and two pin writes per step) and the sleep granularity, not by the 2 µs floor; the maximum reliable rate and the stall speed were not measured. **[MUST STILL BE COMPLETED BY ME: measure the highest feed without lost steps.]** Since *every* pulse of both axes sleeps for τ, a segment takes $(n_{\text{maj}} + n_{\text{min}})\tau$ and the effective feed is

$$v_{\text{eff}} = v_f \frac{n_{\text{maj}}}{n_{\text{maj}} + n_{\text{min}}} \in \Big[\frac{v_f}{2}, v_f\Big], \tag{60}$$

v_f for an axis-parallel move and $v_f/2$ at 45°; for uniformly distributed directions the mean factor is $\frac{4}{\pi}\int_0^{\pi/4} \frac{\mathrm{d}\vartheta}{1+\tan\vartheta} = \frac{1}{2} + \frac{\ln 2}{\pi} = 0.72$. The interpreter executes *no acceleration ramps*: the constant speed of Equation 59 is used, and the acceleration of Equation 49 appears only in the generator's estimate; abrupt starts at 1200 to 3200 steps/s with an unknown motor and load may lose steps. For the test word of Section 3.8, a model of the interpreter (modal feed, Equation 60, first travel at 2400 mm/min) gives 6.8 s drawing, 10.8 s travel and 9.1 s dwell, 26.7 s in total (2.4 s per character for 11 characters). Neither this nor Equation 49 includes the pen-motor run time or program overhead, and neither is a measurement; both are below the 4 s per character of R3 for this short word, where dwell is 34 % of the time, but R3 must be demonstrated with a stopwatch.

3.9.8 Bresenham line interpolation

Stepping the two axes independently would draw an L-shaped path. The program uses the integer algorithm of Bresenham [37]: with $a_x = |\Delta x|$, $a_y = |\Delta y|$ the step counts and the DIR pins set once, for $a_x \geq a_y$ (else the roles swap): (1) set $e \leftarrow \lfloor a_x/2 \rfloor$; (2) repeat a_x times: if an end-stop is triggered, stop; pulse STEP of X; $e \leftarrow e - a_y$; if $e < 0$, pulse STEP of Y and $e \leftarrow e + a_x$. Keeping $e = \lfloor a_x/2 \rfloor - k a_y + m_k a_x$ in $[0, a_x)$ gives, after k major steps, the minor-axis count of Equation 61,

$$m_k = \Big\lceil \frac{k a_y - \lfloor a_x/2 \rfloor}{a_x} \Big\rceil, \tag{61}$$

which differs from the ideal $k a_y/a_x$ by at most $\frac{1}{2} + \frac{1}{2a_x} \leq 1$ step (about 6 µm); only integer arithmetic is needed.

3.9.9 Direction-agnostic calibration

Equation 58 rests on assumed transmission values and the position after power-up is unknown, so each session starts with a calibration between the two switches of each axis, which never needs to know which direction leads to which switch. For one axis the program drives in an arbitrary direction (DIR high, 1.6 ms per step) until *either* switch triggers, reverses, and counts the steps N until the *other* switch triggers. With the hand-measured switch-to-switch travel L (204 mm in

X, 262 mm in Y), the calibrated resolution, the initial counter and the position in millimetres are

$$\kappa_{\text{axis}} = \frac{N}{L}, \qquad n_0 = \begin{cases} N & \text{if the second switch was MAX,} \\ 0 & \text{if it was MIN,} \end{cases} \qquad x_{\text{mm}} = \frac{n}{\kappa} - 5. \tag{62}$$

The zero lies 5 mm inside the minimum switch, the usable area is 194×252 mm, and the axis is retracted 5 mm off its last switch. A fault during the calibration aborts it and jobs are then refused; the flow chart, the three fault conditions and the sequencing and duration of the two traverses are given in the appendix with Figure 51. The result is deliberately not stored, because a mechanical bump between sessions would silently invalidate it. A hand-measurement error δL scales every position, giving the error $x\,\delta L/L$ at distance x; to keep it below half of R5 over 194 mm,

$$\delta L \le \frac{0.25 \text{ mm}}{194 \text{ mm}} L = \frac{0.25}{194} \cdot 204 \text{ mm} = 0.26 \text{ mm} \quad (0.13\ \%), \tag{63}$$

and the repeatability of the switch trigger adds directly to the zero error (Equation 63 requires this accuracy of the hand measurement). **[MUST STILL BE COMPLETED BY ME: state the method and uncertainty of the measurement of L_X and L_Y.]**

3.9.10 End-stop monitoring, emergency stop and pen mechanism

All four switches are read before every major-axis step and every G-code line. A switch counts as triggered only if it reads high three times in a row, 6 ms apart, because motors without flyback diodes and a shared ground return were found to hold an input high for several milliseconds, longer than a single re-check. The latency from a trip to the stop (Equation 64) is

$$t_{\text{stop}} \approx \tau + (3-1) \cdot 6 \text{ ms} \approx 13 \text{ ms}, \tag{64}$$

during which the carriage moves 0.2 mm at 15 mm/s and 0.5 mm at 40 mm/s, far inside the 5 mm edge tolerance. On a trip the program latches an emergency stop, sets both STEP pins low, retracts 5 mm (400 steps) off every triggered switch and aborts the job; every job target is also clamped to the calibrated usable area. The conditions under which the latch clears are given in the appendix. *Limitations:* protection is software only (no hardware emergency stop that cuts motor power is documented); in the web-server mode the switches are polled only during a job; no endpoint aborts a running job; and no lock prevents two simultaneous jobs.

Pen mechanism. The proposal planned to lift the pen through the stepper driver; the implemented lift is a gear motor that turns in one direction only, each 90° of its cam toggling the pen, so only pulses can be given and the state must be known. The pen switch is feedback only (high = up). The routine behind `M5` (up) and `M3` (down) returns at once if the switch already shows the target, because a toggle motor that runs when no change is needed inverts the state; its polling, its time-out and its behaviour when the switch never confirms are set out in the appendix. With the generator's open-loop tracking this forms a state machine with a fault path; the real time of one toggle was not recorded (the generator assumes 0.35 s).

What is not modelled. Several physical effects are not modelled at all, and missed steps cannot be detected, since there is no feedback; the complete list is given in the appendix. The true accuracy must therefore come from the test of Section 4 and not from the resolution of Equation 58.

3.10 Software architecture and system integration

This subsection describes how the software blocks of Sections 3.3 to 3.9 form one application that runs on the single-board computer and is operated from a web browser (the display and input interface of the proposal). All blocks are Python programs written by the student; only

the camera interface, the pin interface and the transport of web requests come from libraries. **[MUST STILL BE COMPLETED BY ME: confirm that the complete source code has been submitted separately.]**

3.10.1 Modules and data flow

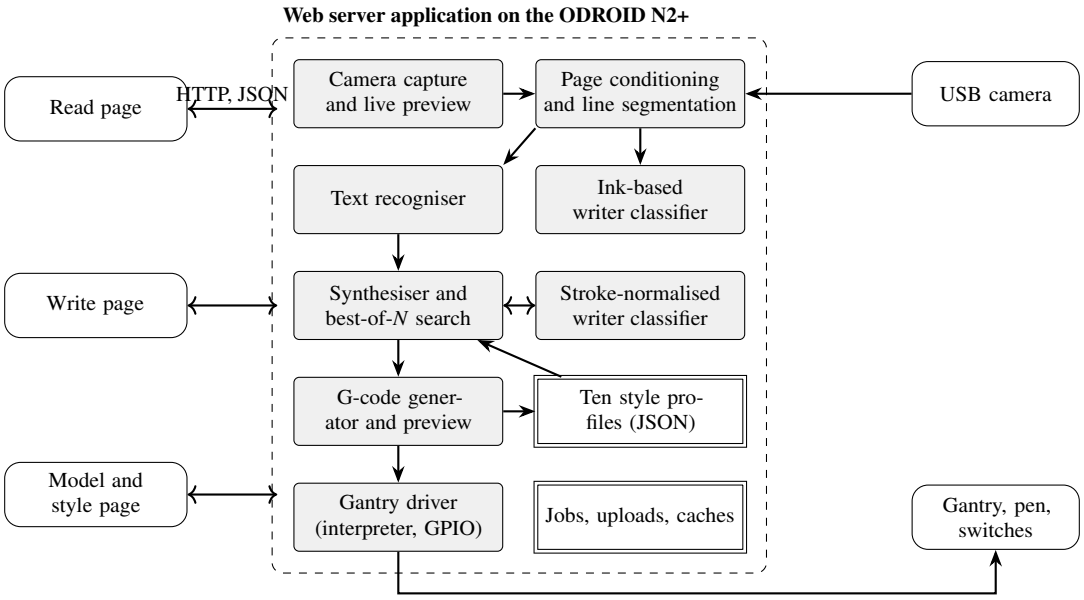


Figure 10. Software architecture. The three browser pages talk to one server application that links the blocks of Sections 3.3 to 3.9; the text recogniser also reads back every synthesised preview.

Figure 10 shows the modules. The server offers the blocks as HTTP requests with JSON bodies; its 18 routes are listed in Table 29 of the appendix. The browser pages are static files any machine can serve; each reads the server address from a browser setting, attaches an optional access key to every request, and shows a green or red dot according to whether the list of writers can be fetched. The three modes of the proposal map onto the pages as *classification* on the Read page, *reproduction* on the Write page and *training* offline with no page at all, since the ten writers are pre-built files and the interface has no training function. Table 36 of the appendix gives the chain of operations of each mode and the content of the third page.

3.10.2 Start-up, caching, job storage and deployment

At start-up the server loads the text recogniser, the stroke-normalised classifier that judges the synthesis, the ink-based classifier that classifies photographs and all profiles; each network is a from-scratch array-arithmetic implementation of the trained weights (Sections 3.4 and 3.5), so no training library runs on the board. A job is refused unless calibration has succeeded in this server session, and the Write page locks its controls until then. Security suits a trusted local network only: no accounts, an optional shared key, requests from any origin accepted, and anyone who can reach the port can start a motion. **[MUST STILL BE COMPLETED BY ME: the network configuration of the board (address, connection to the laptop), operating system and Python version, how the server is started, and the memory used by the loaded models.]** The caching of the glyph grids and the sample crops, the storage of photographs and jobs, the fixed seed, the listening port and the lazy import of the pin interface, and the choices

made for ergonomics are described in the appendix under “supporting material for the software architecture”.

3.10.3 Timing chains of the two modes

Requirement R4 limits the classification of x characters:

$$T_{\text{cls}} = T_{\text{cap}} + T_{\text{seg}} + \sum_{\text{lines}} \left(T_{\text{text}} + T_{\text{wid}} \right) \leq 2\ \text{s} + 0.025\, x\ \text{s}, \tag{65}$$

In Equation 65 capture, conditioning and segmentation must fit inside the 2 s, and both network passes inside 25 ms per character, so a 60-character page allows 1.5 s of recognition and 3.5 s in total. Requirement R3 limits the reproduction of x characters:

$$T_{\text{rep}} = T_{\text{synth}} + T_{\text{gcode}} + T_{\text{motion}} \leq 4x\ \text{s}, \qquad T_{\text{synth}} \approx NC, \tag{66}$$

In Equation 66, C is the cost of one candidate (Equation 115), T_{gcode} is milliseconds and T_{motion} is the job time of Section 3.9.7; the one-off calibration is excluded and T_{synth} is included, the stricter reading of the proposal. Table 32 of the appendix lists the stages and the measurements still missing: the server returns the synthesis time of each request, but no timer exists for classification and no timing of any stage on the single-board computer was recorded. For a 56-character sentence the budget is $4 \cdot 56 = 224$ s; the search alone took 23 to 137 s at $N = 14$ on the laptop and the motion model gives 2.4 to 2.8 s per character for the short test word, so the two terms are of the same order and R3 depends on N and the board’s speed; it is therefore *not* claimed that R3 or R4 is met. **[MUST STILL BE COMPLETED BY ME: the timings of the stages on the single-board computer, listed in Table 32 of the appendix.]**

Component	Quantity and rating	Reason for the choice
Single-board computer, ODROID N2+	1	The proposal requires all processing on an embedded platform. It runs the networks, the server and the motion control on one board, and its 3.3 V pins are driven directly from the motion program.
Stepper motor, NEMA 17	2, 1.68 A per phase	A stepper gives open-loop position from a step count, so no encoder is needed for the resolution R5 asks for. The NEMA 17 frame is the standard size for which the belts, pulleys and brackets used here are made.
Stepper driver, A4988 carrier	2	Takes one STEP and one DIR line per axis, which is the smallest pin count for the job, and chops the coil current so the 1.68 A winding can be run from the 24 V rail. Microstepping to 1/16 is set by jumpers and multiplies the resolution of Equation 58 by sixteen.
Power supply	1, 24 V	A higher rail forces current into the winding inductance faster, which is what allows the step rates of Section 3.9.7; 24 V is inside the A4988's 8 to 35 V input range.
Bulk capacitor	1, 100 µF	Supplies the switching current of the drivers locally so the rail does not sag at each chop (Equation 57).
Micro switch	5	Four as the end-stops of the two axes, which make the direction-agnostic calibration of Section 3.9.9 and the emergency stop possible; the fifth reports the state of the pen cam. A mechanical switch was chosen over an optical sensor because it needs no supply of its own and cannot be confused by ambient light.
Pull-down resistor	4, 10 kΩ	Holds each switch input low when the switch is open, so the input is never floating (Equation 54).
Pen-lift motor, N20 gear motor	1, 150 rpm	Geared down enough to turn the cam that toggles the pen in one direction. Its speed sets the toggle time, which Section 3.9.10 shows is the largest single contributor to the predicted writing time.
Switching transistor, BC337-40, with 1 kΩ base resistor	1	A general-purpose pin cannot supply the pen motor's current, so the pin switches the motor through a transistor instead (Equation 55 and Figure 9).
Timing belt and pulleys, GT2 2 mm pitch, 20 teeth, with idler pulleys	2 axes	A belt was chosen over a lead screw because the long, light travel of this machine needs speed more than it needs force, and because belt backlash can be kept small by tensioning with the idlers. 2 mm pitch on 20 teeth gives the 40 mm per revolution of Equation 58.
Chromed steel rods, linear bearings, 5 mm to 8 mm shaft coupler	as required	The rods and bearings carry the carriage so the belt only has to pull it, and the coupler joins the 5 mm motor shaft to the 8 mm cross-shaft described in Section 3.9.2.
USB camera in a printed housing	1	Fixes the camera at a known height above the page and clamps the paper flat, which is what makes the illumination correction of Section 3.3 work on a page that would otherwise curl.

Table 7. Components of the gantry and the capture rig, with the reason for each choice. All parts were bought; the interface circuits and the control program around them are the student's own work.

4. Results

4.1 Summary of results achieved

This section qualifies the system against requirements R1 to R6 and field conditions FC1 and FC2 of the approved proposal, and reports the results that exist. All measurements were taken on the development laptop (x86-64, running the pipeline under CPython), not on the ODROID N2+, and every figure states its data set and split. Table 8 uses four status labels: *Met*, *Partially met*, *Not met* and *Not measured*. A requirement is met only where the evidence bears directly on it; a cell states where a figure came from a split that favours the system, or from a judge that also takes part in the synthesis search. Thirteen writers were used, eight from the IAM database and five personal contributors, which is three more than R6 requires.

R6 is met and R2 is met for writer identification on real handwriting. R1 is close to its target but not at it, with the shortfall in word-level rather than character-level accuracy. R3 fails on the search cost and R4 on segmentation rather than recognition. R5 and both field conditions could not be assessed, because the hardware bring-up and the environmental measurements were not completed. Figure 11 places each measured value against its target; the shortfalls and their remedies are analysed in Section 5.

4.2 Qualification tests

For each requirement, this subsection summarises the test that is to prove it on the integrated system, in the proposal's order of importance. The step lists, metric definitions and data sheets are in the Appendix (Section "Extended qualification-test protocols"). Proportions are reported with a 95% Wilson score interval [46],

$$\hat{p}_{\pm} = \frac{\hat{p} + \dfrac{z^2}{2n} \pm z\sqrt{\dfrac{\hat{p}(1-\hat{p})}{n} + \dfrac{z^2}{4n^2}}}{1 + \dfrac{z^2}{n}}, \qquad \hat{p} = \frac{k}{n}, \tag{67}$$

where k is the number of successes in n trials and $z = 1.96$; it stays valid near 0% and 100%. Characters of one page are not independent, so accuracies use a page-level percentile bootstrap with 2000 resamples [47]. A requirement is declared met only if the lower confidence bound exceeds the target. Character accuracy uses the Levenshtein distance [19] $d_L(h, r)$ between hypothesis h and reference r, the minimum number of insertions, deletions and substitutions,

$$A_{\mathrm{c}} = 1 - \frac{d_L(h, r)}{|r|}, \tag{68}$$

summed over all sentences, where $|r|$ is the reference length. All random choices use recorded seeds.

Qualification test 1: Reproduction fidelity of the synthesised handwriting (R1)

Objectives of the test or experiment

The test is to prove at least 85% character accuracy and 85% stylistic correlation. The existing evidence is circular, because the networks that select the best of N candidates also score the result; the test therefore uses judges outside the search.

Equipment used

The development laptop (x86-64, CPython) with the stored style profiles of the thirteen writers; a separate evaluation program that does not import the synthesiser or its judges; and the base recognition checkpoint that is not used in the search. The ODROID N2+ and the gantry were *not*

used: the renderings were produced and scored on the laptop, and no text was written on paper. Printed worksheets were prepared for a blind human panel, but the panel was not convened.

Test setup and experimental parameters

The parameters differ between sub-tests, which were run separately. Sub-test (a) gives character accuracy twice: by the recogniser inside the search, over thirteen writers, 6 novel sentences and 2 seeds at $N = 30$ and over the same writers at $N = 8$ in a wider coverage run; and by the base recognition checkpoint, outside the search, over 60 synthesised lines from ten writers at $N = 20$. Genuine handwriting of the same words, 123 held-out lines, is read by the same recogniser as the upper bound. The blind panel of 8 raters and Fleiss' kappa [48] belonged to this sub-test and were not carried out. Sub-test (b): nine hand-crafted style features from one deterministic extractor, computed on held-out real pages and on renderings for ten writers; the Pearson correlation ρ of the pooled z-scored vectors and a nearest-centroid writer retrieval are reported, bootstrapped over writers. Sub-test (c): the sentence a writer actually wrote is synthesised in that writer's style at $N = 20$, both images are normalised to a common x-height x_h, and the chamfer and modified Hausdorff distances [49] over 30 matched and 300 mismatched pairs are compared by rank-1 retrieval and by the area under the receiver operating characteristic curve (AUC; 0.5 means no writer information). Sub-test (d): the demonstration of the proposal, with 85% of the produced characters visibly matching. The definitions (Equations 94 to 98), features (Table 38), worksheet (Table 37) and the procedure of Figure 44 are in the Appendix.

Steps followed in the test or experiment

The held-out pages are fixed and the system frozen; the renderings are produced on the ODROID N2+; sub-tests (a) to (d) are then evaluated in order (numbered steps in the Appendix); the confusion matrix, overlay figures and intervals are produced from stored outputs.

Results or measurements

Table 9 lists the measured figures, in two groups. Any figure measured with the *search judges*, the recogniser and writer classifier that score the N candidates, is inflated by the selection and is marked as such. The *independent measurements* are the style correlation on hand-crafted geometric features, the overlay test in the pixel domain and a character-accuracy reading by a recogniser checkpoint that was *not* used in the search; none involves the search and all can therefore fail. Figure 45 plots the per-feature style correlation, Figure 12 the character accuracy of the same lines under both checkpoints, Figures 13 and 46 the overlays and their distance distributions, and Figure 14 genuine handwriting against the synthesis of the same sentences.

Observations

The nine-feature style correlation was 0.714 on shape-only features, with a bootstrap interval of 0.612 to 0.859 over writers; the target is 0.85, and both the point value and the lower bound lie below it. Per feature, slant correlated at 0.996 and ascender extent at 0.974, while descender extent correlated at 0.148, the ascender-to-descender ratio at 0.509 and the word gap at 0.605; the lowest correlations are those of the vertical extents below the baseline and of the spacing. The word-accuracy gap between the synthesis and real handwriting of the same words was 23.8 points. The overlay test separated matched from mismatched pairs with an AUC of 0.71, where 0.5 corresponds to no writer information, and retrieved the correct writer first in 20% of cases against a chance of 10%. An independent recogniser checkpoint read the synthesised lines at 81.2% character accuracy, against 84.9% for the production checkpoint on the same lines, a difference of 3.7 points, and about 8 points below the 89.0% read on real handwriting of the same words. At best-of-30 the production judge read 84.9% character accuracy against 80.9% for the same configuration under the independent checkpoint. An earlier design-stage comparison measured 75.9% character accuracy before the search was introduced and 81.6% after it. **Not carried out.** The blind human panel of eight raters, the Fleiss kappa agreement measure and

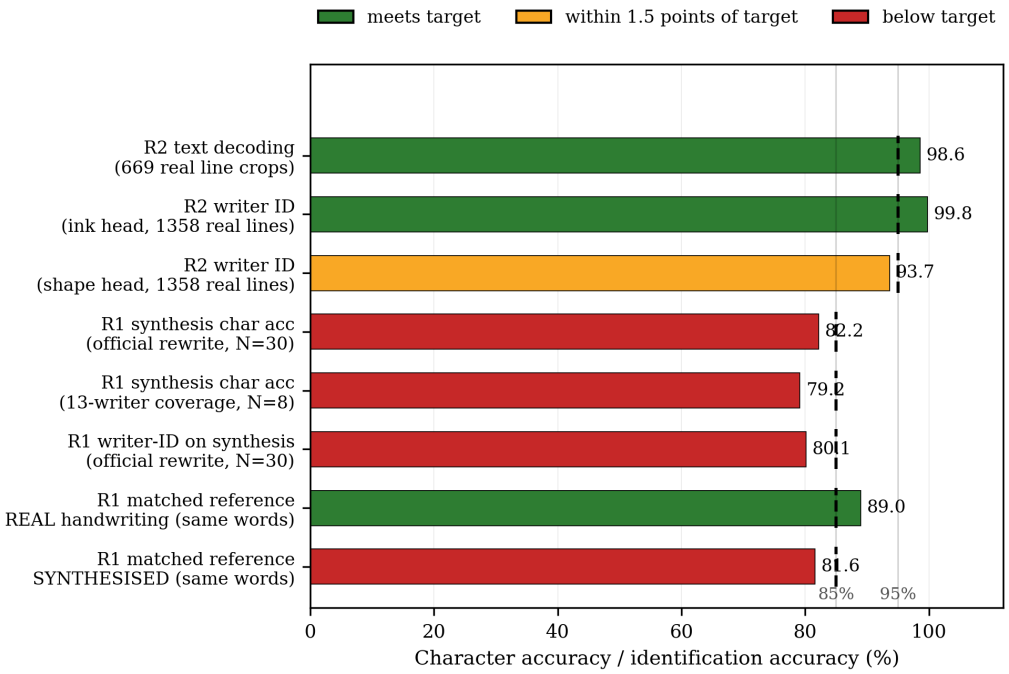


Figure 11. Measured accuracy against the target of each requirement. Targets are marked by the dashed vertical lines at 85% (R1) and 95% (R2). Green bars meet the target, amber bars lie within 1.5 points of it, and red bars fall below it. The two green R1 bars are the matching comparison on genuine handwriting and the writer classifier on real lines; the red synthesis bars are the values the system actually produces.

the live demonstration of the proposal were not executed, so the third and fourth sub-tests of this qualification test remain **[NOT YET MEASURED]**. The feature-correlation and overlay sub-tests above do not replace them: they establish that the synthesis differs measurably from the writers' real hands, but they cannot say whether a person would accept the output as being in that writer's style, which is what the proposal's demonstration asks.

Statistical analysis

By the Wilson interval of Equation 67, the official run's 82.2% over 156 sentences has a lower bound near 75%, so the result is below 85% and not a sampling fluctuation. The style correlation is bootstrapped over writers (2000 resamples), the correct unit because the nine features are measured per writer; the shape-only interval is 0.612 to 0.859. The overlay AUC is computed over 30 matched and 300 mismatched pairs, with its interval reported alongside. The two-judge comparison is paired, since both checkpoints read the same 60 synthesised lines, so the 3.7-point difference is not affected by sampling variation in the output. The proportion of characters that a human observer would accept as matching the writer's style, which is how the proposal worded the demonstration, was not measured.

Qualification test 2: Classification of text and writer from real photographed pages (R2)

Objectives of the test or experiment

The test is to prove an overall accuracy of at least 95% through the whole chain on pages never used for training, checkpoint selection or tuning. As the proposal does not define "overall", text accuracy A_{c} (Equation 68) and writer accuracy A_{w} (fraction of pages assigned to the correct writer) are reported separately and combined as

$$A_{\mathrm{mean}} = \tfrac{1}{2}\left(A_{\mathrm{c}} + A_{\mathrm{w}}\right), \qquad A_{\mathrm{prod}} = A_{\mathrm{c}} A_{\mathrm{w}}, \tag{69}$$

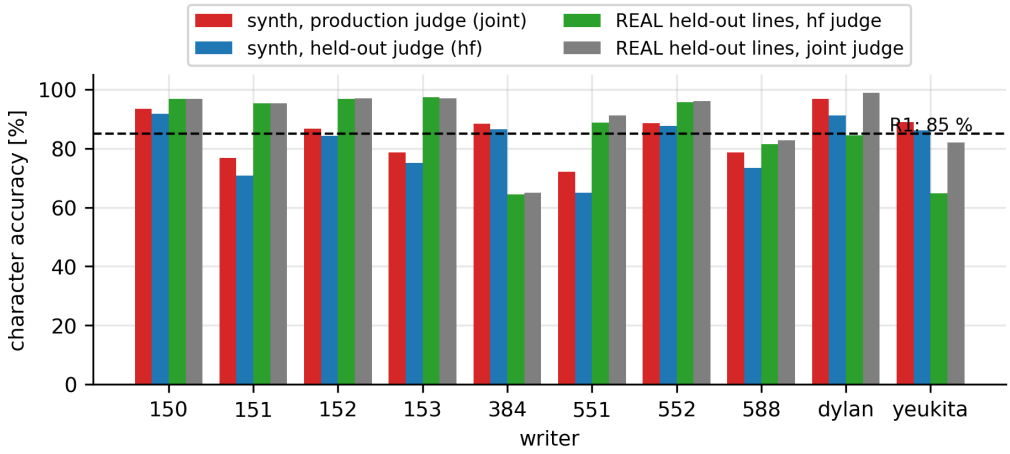


Figure 12. Character accuracy of the synthesised text under the production recogniser and under a recogniser checkpoint that was not used to select the candidates. The independent judge scores the same output about three points lower, which quantifies how much of the apparent quality is a selection effect and simultaneously shows that the synthesis is genuinely legible rather than merely self-consistent.

where A_{prod} (both correct) is the criterion and A_{mean} the lenient reading.

Equipment used

The capture rig and its camera photographed the pages of the five personal writers. Segmentation, recognition and writer classification of those photographs then ran on the development laptop with the final server and models; the ODROID N2+ was not used to produce any figure in this test. Camera and lamp: **[MUST STILL BE COMPLETED BY ME: camera model, resolution, mounting height, lamp type and photograph of the rig]**.

Test setup and experimental parameters

The line-level validation split of the personal writers leaks: a random 15% of lines per page was held out, so lines from the same sheet and session are in both sets (52 of 123 validation lines share pages with training), and the checkpoint was selected on them. The fix is new pages kept aside: at least 3 per personal writer, plus the held-out form page of each of the eight database writers, photographed with the rig (about 14 pages, 350 lines). Seen lines are scored separately to quantify the leakage; references are double-entered transcripts.

Steps followed in the test or experiment

Each frozen photograph is classified through the server without intervention; line text, per-line probabilities and the page-level mean probabilities are recorded, scored, summarised in a 10×10 confusion matrix, and the predefined paragraph is then classified live.

Results or measurements

Table 10 lists the measured figures. Text and writer accuracy are reported separately, with Equation 69 giving the strict and lenient combinations where both are available.

Observations

Text decoding reached 98.61% character accuracy on correctly segmented crops, against the 95% target, measured after segmentation and on writers represented in the recogniser's fine-tuning data. Word accuracy was 94.21%, lower than the character accuracy. Writer identification reached 99.78% with the ink classifier over 1358 real lines from thirteen writers, against the same 95% target. Thiya is the weakest writer for recognition at 96.46% character accuracy, and Owen, Dylan and Thiya the weakest for shape-based identification. The recogniser figure with

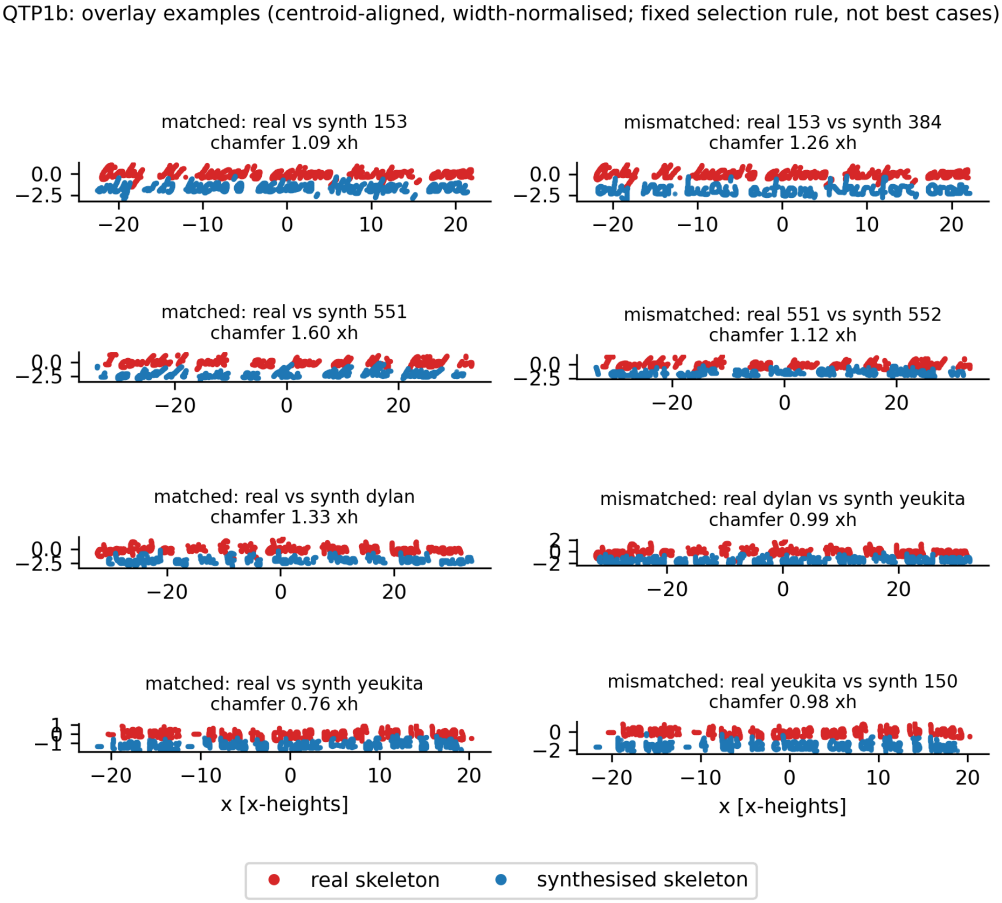


Figure 13. Overlay test: for each text the writer's own handwriting and the system's synthesis of the same words are normalised to a common x-height and superimposed. Matched pairs are visibly closer than mismatched pairs, which is what the distance ratios and the rank-1 retrieval rate of 20% (chance 10%) measure. The procedure that produces these overlays is given as Figure 44 in the Appendix.

the ink writer figure gives $A_{\text{mean}} = 98.6\%$ and $A_{\text{prod}} = 98.4\%$, both above 95%. These rates were measured on a split that holds out 15% of lines per page, so lines from the same sheet appear in both partitions.

Statistical analysis

With 669 line crops, 98.61% character accuracy has a Wilson lower bound of about 97.6%, so the text figure is significantly above 95% on this data set. For writer identification, 1358 lines at 99.78% give a lower bound above 99%. The stronger caveat is dependence rather than sampling error: lines from one page share a writer, a pen and an illumination, so the effective sample size is nearer the page count than the line count, and the per-page interval would be much wider. *Expected outcome on genuinely unseen pages.* Both rates are expected to fall, because the ink classifier is sensitive to pen, paper and illumination and the split shares pages between partitions. A defensible figure needs a page-level split with at least three held-out pages per writer, photographed through the rig.

Qualification test 3: Reproduction time per character (R3)

Objectives of the test or experiment

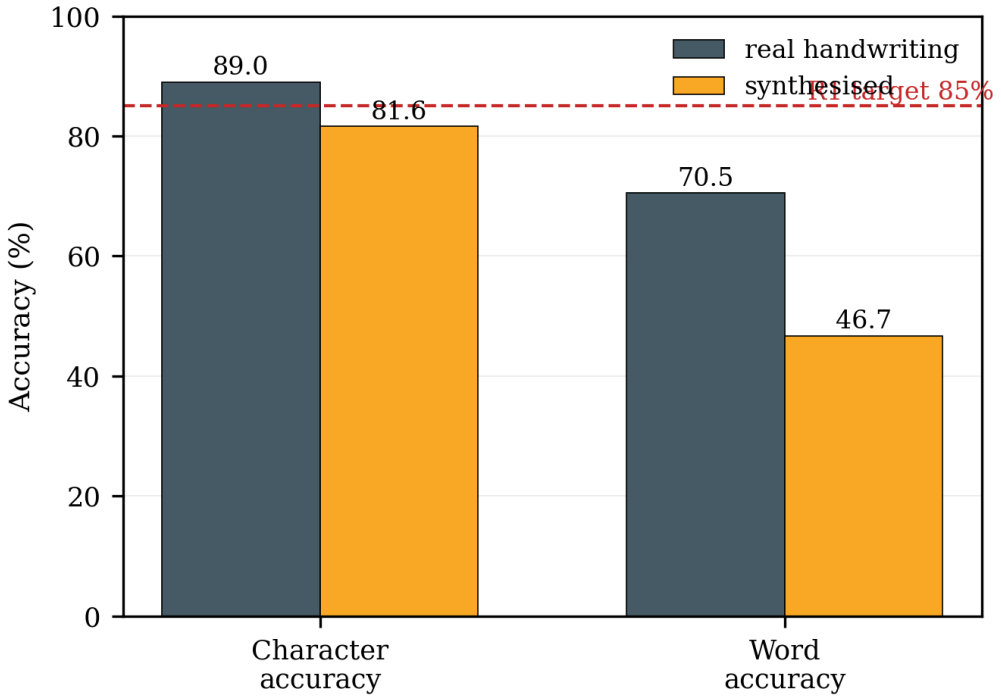


Figure 14. Matching comparison on identical sentences: genuine handwriting of the sentence against the system's synthesis of that same sentence, both read by the same recogniser. Character accuracy is close to the target, but word accuracy divides by more than a factor of 1.5, which shows that the synthesis preserves letterforms better than it preserves word boundaries and spacing.

The test is to prove that a sentence of x characters is reproduced within $4x$ seconds,

$$t_{\text{char}} = \frac{T_{\text{syn}} + T_{\text{write}}}{x} \leq 4 \text{ s}, \tag{70}$$

where T_{syn} is the synthesis time, T_{write} the time from first to last motion and x the number of characters. As the proposal does not say whether synthesis counts, both variants are reported; calibration is a one-off step and is reported separately.

Equipment used

The development laptop with the server, and logged time stamps at the start and end of synthesis. The assembled gantry and a stopwatch were to have provided the writing time, but the gantry was not driven, because its bring-up was not completed, so the motion time is a model and not a measurement.

Test setup and experimental parameters

Tries $N \in \{1, 3, 6, 14, 30\}$, sentences of 10, 20, 40 and 80 characters, 5 repetitions (100 syntheses); writing time for the 20 jobs at $N = 14$. Only figures measured on the ODROID N2+ itself count; laptop figures are indicative.

Steps followed in the test or experiment

After one calibration, each job is synthesised, sent and timed by log and stopwatch, t_{char} is computed from Equation 70, and any stall is kept as a failed run.

Results or measurements

Table 11 and Figure 17 give the measured figures. The cost separates into a synthesis term that grows with the number of candidates and a writing term that does not.

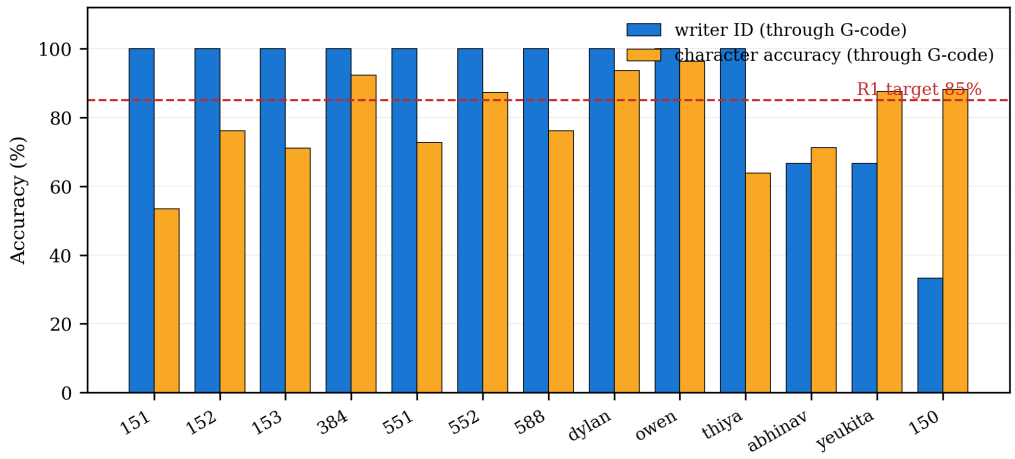


Figure 15. End-to-end result per writer over all thirteen writers: writer identification and character accuracy, both measured through the G-code round trip, for three novel sentences per writer at best-of-8. Eleven writers are identified at 100%; the character-accuracy failures are concentrated in the two writers whose hands are the most joined, which is the known weak case of an exemplar synthesiser that cuts letters apart and reassembles them.

Observations

At best-of-1 to best-of-3 the synthesis cost well under 1 s per character, and the cost grew in proportion to the number of candidates, from about 0.4 to 0.7 s per character at best-of-1 to about 1.8 to 3.4 s per character at best-of-30. The physical writing time predicted by the generator's model is 2.7 to 4.0 s per character on its own, against the 4 s per character limit of R3. The four-character word at best-of-6 gave 16.1 s per character; its per-job overheads are divided over only four characters, and the same run at a longer sentence length gives 1 to 2 s per character for synthesis. The writing term is a model value and not a measurement: the model assumes a pen dwell of 0.26 s per stroke, and no drawn job was timed on the gantry.

Statistical analysis

Each cell is the mean of two or three repetitions, and the table quotes a range rather than a single value, because for the short sentences the variation between repetitions was of the same order as the variation between configurations. The generator's writing model is deterministic and quoted as a point value; it has no uncertainty because it is a calculation, which is also why it cannot substitute for a measurement on the built machine. The proposal does not state whether the synthesis time counts towards the limit, so Table 11 reports both readings.

Qualification test 4: Classification time (R4)

Objectives of the test or experiment

The test is to prove that the time from instruction to displayed text and writer satisfies

$$T_{\text{cls}} \le 2 + 0.025\,x \text{ seconds}, \tag{71}$$

where x is the number of handwritten characters; the stage times identify any shortfall.

Equipment used

The rig, the development laptop with the final server, and a logging wrapper (monotonic clock) around each stage. The ODROID N2+ was not timed, so the figures below are laptop figures and the board is expected to be slower. The interface does not display classification time.

Test setup and experimental parameters

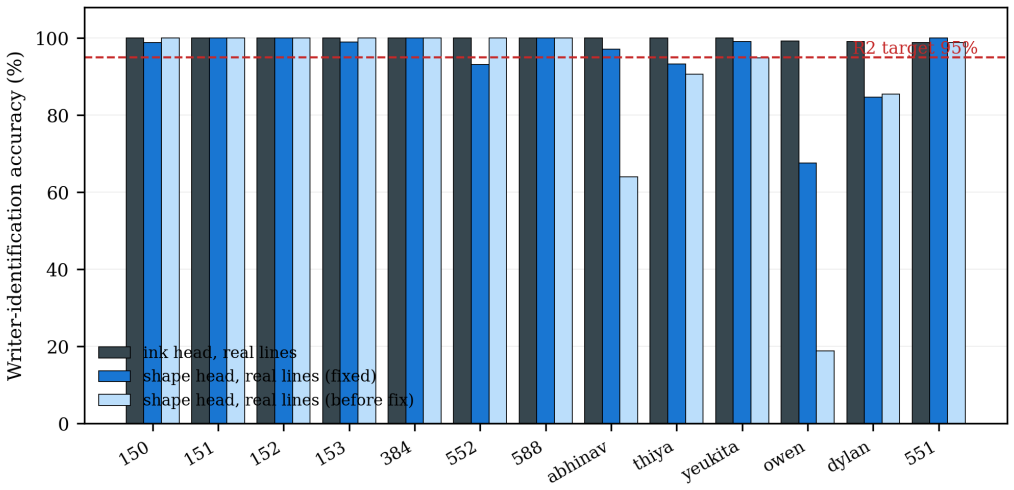


Figure 16. Writer-identification accuracy per writer over 1358 real lines, for the ink classifier and for the shape classifier before and after its training fault was corrected. The ink classifier is at or near 100% for every writer. Correcting the shape classifier's training raised its overall accuracy from 83.4% to 93.7% and removed a systematic collapse in which 118 of Owen's 148 lines had been assigned to Yeukita. The dashed line is the 95% target of R2.

Pages with $x = 40$, about 200 and about 1000, each run 10 times; the budgets are 3.0, 7.0 and 27.0 s. Stages: capture, conditioning and segmentation, recognition per line, writer classification, display; the segmentation profile is repeated on the ODROID N2+.

Steps followed in the test or experiment

The clocks start at the instruction, stage times and the total are logged, x is counted from the transcript and Equation 71 is evaluated.

Results or measurements

Table 12 lists the measured figures.

Observations

Recognition ran at 8.86 ms per character, about a third of the 25 ms per character allowance. Segmentation of one photograph took 35 to 57 s, 17 to 28 times the two seconds the equation allows for capture and conditioning; this cost is per page and does not vary with x, which appears only in the recognition term of Equation 71. A single already-segmented crop was classified through the live server in 2.74 s end to end.

Statistical analysis

The recognition timing is an average over 669 crops and is therefore well determined; the per-character figure is the total inference time divided by the total number of characters, so it is a throughput measure rather than a per-line latency. The segmentation time varies with page content and machine load, which is why a range of 35 to 57 s is reported; the lower end was obtained on an otherwise idle machine. *Expected outcome.* The ODROID N2+ is expected to be slower than the laptop, so the segmentation shortfall would be at least as large there with the same pure-Python code; the recognition term has enough margin that it may still be met on the board, but this was not verified. These numbers indicate moving segmentation to compiled code or processing the page incrementally, as the C99 experiment suggests a saving of about a quarter to a third without changing the algorithm.

Qualification test 5: Absolute positional accuracy of the gantry (R5)

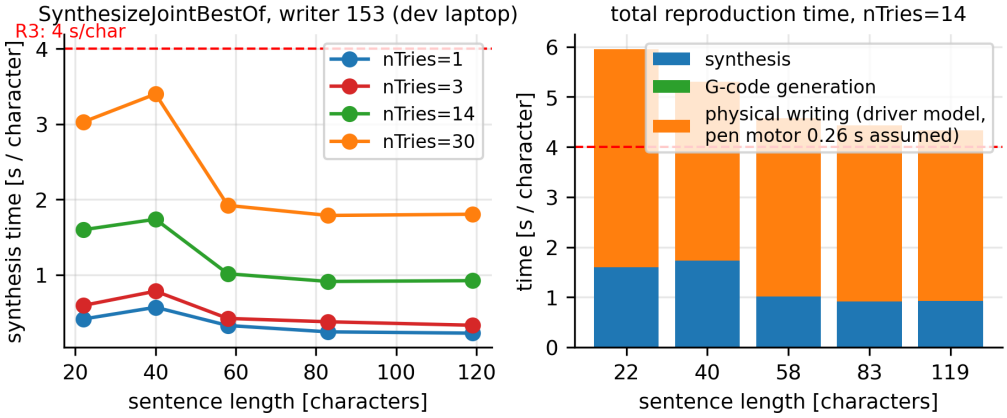


Figure 17. Reproduction time against the 4 s per character limit of R3. Left: synthesis time per character against sentence length for four values of the number of candidates. At best-of-1 and best-of-3 the synthesis alone is comfortably inside the limit; at best-of-30 it approaches and crosses it. Right: total time per character at best-of-14, decomposed into synthesis, G-code generation and the physical writing time predicted by the generator's model. The writing term, not the synthesis term, is the larger contributor and is what pushes the total past the limit.

Objectives of the test or experiment

The test is to prove that the absolute positional error of drawn marks is at most 0.5 mm and that a word written twice coincides within that tolerance. Software contributions (quantisation, calibration) and hardware contributions (mechanics, homing repeatability, pen compliance) are separated.

Equipment used

The gantry with pen and paper, a flat-bed scanner (600 dots per inch, 0.042 mm per pixel), a steel ruler or vernier calipers (0.02 mm) and a printed calibration sheet with a 10 mm grid of crosses.

Test setup and experimental parameters

Software part (offline): the step is $1/80 = 12.5$ μm, the quantisation error per axis at most 6.25 μm and the combined worst case 8.8 μm (derived). Hardware part: at least 40 crosses drawn 5 times after fresh calibrations, the paper fixed against registration stops, and the same word written twice. The error of a cross is $e_i = \|\mathbf{p}_i - \mathbf{p}_i^{\mathrm{cmd}}\|$ (measured against commanded position referenced to the machine origin); mean, maximum and 95th percentile are reported with the measurement uncertainty (Equations 99 and 100 in the Appendix).

Steps followed in the test or experiment

After calibration (the direction pin must be seen to change level on reversal) the grid is drawn, scanned with the ruler, evaluated by image analysis, repeated 5 times, and the overlay of the word written twice is produced (Appendix).

Results or measurements

The only quantities that exist are derived from the mechanical configuration, not measured on a drawn mark. The X and Y axes are belt-driven: a 200-step motor, sixteen microsteps per step, and a 40 mm advance per revolution give

$$\Delta s = \frac{40\ \mathrm{mm}}{200 \times 16} = 12.5\ \mu\mathrm{m\ per\ step}, \tag{72}$$

so by Equation 72 the quantisation error of one axis is at most 6.25 μm and the worst case over both axes is 8.8 μm, about 57 times finer than the 0.5 mm limit of R5. During bring-up

the X-axis calibration drove to the first end-stop, but the direction pin did not change level on reversal; the fault was not resolved, and no grid was drawn, so there is no overlay of a word written twice and the measurement of e_i remains **[NOT YET MEASURED]**.

Observations

No positional measurement exists. The one hardware observation is the direction-pin fault: the X-axis calibration drove to the first end-stop and the direction pin was not observed to change level on reversal. No grid was drawn, so the test was not carried out. The derived step resolution is 40 times finer than the tolerance.

Statistical analysis

No statistics can be reported for a measurement that was not taken. *Expected outcome.* The error is expected to be dominated by belt stretch, backlash, the repeatability of the home switch and pen compliance, none of which was measured. Published handwriting gantries report about 0.3 mm absolute error [38], a margin of 0.2 mm against this limit; whether this machine achieves that cannot be stated. A pass requires the maximum of e_i over at least 200 drawn marks to be at most 0.5 mm, measured by scanning the sheet at 600 dots per inch.

Qualification test 6: Multi-writer storage (R6)

Objectives of the test or experiment

The test is to prove that at least 10 writing styles are stored permanently and that any can be selected at random for reproduction.

Equipment used

The development laptop with the server and a browser. The gantry was not used, because the reproduction half of this test was not carried out.

Test setup and experimental parameters

The eight database and five personal writers; file counts and sizes; persistence by restarting the server and power-cycling the board; 3 writers drawn with a fixed seed.

Steps followed in the test or experiment

The listed writers are compared with the stored files, the list is re-read after restart and power cycle, and a short sentence is written in each of the 3 drawn styles.

Results or measurements

The live server listed thirteen writers through its author-list endpoint, in 70 ms, from thirteen stored style profiles; the requirement is ten, so the system stores thirty per cent more than required. The styles are ordinary files on disk: the endpoint reads the profile directory, and the profiles are loaded into memory at start-up, which makes them persistent across a restart. The profile and weight-file sizes are listed in the technical documentation, and Figure 18 shows the storage check.

Observations

The list contained thirteen entries against a requirement of ten, and the endpoint answered in 70 ms. The five personal writers are stored in the same format as the eight database writers, so the storage mechanism does not distinguish scanned pages from photographed ones. A writer was not selected at random and written on paper, so the reproduction part of the proposal’s demonstration was not carried out.

Statistical analysis

This is a count against a threshold, so no statistical analysis is appropriate; the figure of thirteen is exact, not a sample estimate. Persistence follows from the profiles being files rather than in-memory state, and is observed each time the server starts and re-lists them; a power-cycle test of the board was not performed, because the board was not deployed.

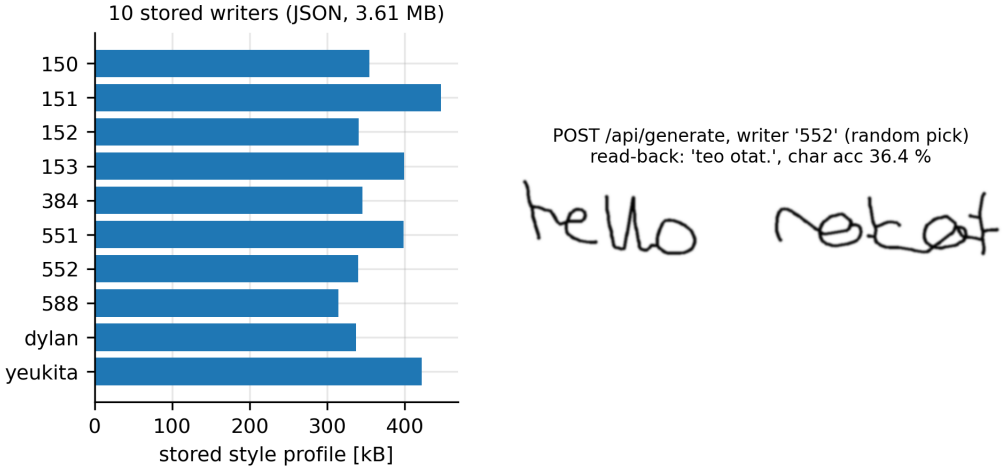


Figure 18. Storage of the writing styles. Each writer is held as a style profile and the server lists all thirteen through its author-list endpoint, against the ten required by R6.

4.2.0 Field condition checks (FC1 and FC2)

Neither field condition was measured. **FC1:** a lux meter at the page plane records five points (centre and corners of an A4 page); the condition is met if all lie between 400 and 1000 lux, and qualification test 2 is repeated on the same pages at about 200, 700 and 1200 lux. Illuminance: **[NOT YET MEASURED]**; the flat-field correction and local thresholding should limit degradation within the range, with larger degradation below 400 lux. **FC2:** a spirit level or inclinometer on both axes (tilt below 0.5°) and a logging accelerometer on the frame during a standard job, with the repeat error of qualification test 5 at rest and with a lightly tapped bench. Tilt and vibration: **[NOT YET MEASURED]**.

Intended outcome	Actual outcome	Location in report
Core mission requirements and specifications		
R1: Reproduce text in the style of any writer of the training set, at least 85% character accuracy and 85% stylistic correlation.	**Partially met.** Character accuracy of the synthesised text was 82.2% (thirteen writers, 6 sentences, 2 seeds, N=30) and 79.2% at N=8; style correlation over nine geometric features was 0.714. Both are below the 0.85 target. The visible-match assessment that the proposal asks for was not carried out.	Qualification test 1
R2: Decode the text and identify the writer with an overall accuracy of at least 95%.	**Partially met.** 98.61% character and 94.21% word accuracy over 669 real line crops; the writer of 1358 real lines was identified at 99.78% by the ink classifier and 93.67% by the shape classifier. The crops were correctly cut and their writers appear in the fine-tuning data, so the split is optimistic, and the complete page-to-text chain was not measured.	Qualification test 2
R3: Reproduce text at no more than 4 s per character.	**Not met.** A four-character word at N=6 took 64.3 s to a rendered preview, which is 16.1 s per character against the 4 s limit. The physical drawing time was not measured, because the gantry bring-up was not completed.	Qualification test 3
R4: Identify the writer and decode the text within $2 + 0.025x$ s for x characters.	**Partially met.** Recognition ran at 8.86 ms per character against the 25 ms allowance, but page segmentation took 35 to 57 s against the 2 s the specification allows for capture and conditioning. Nothing was timed on the ODROID N2+.	Qualification test 4
R5: Absolute gantry positional error of at most 0.5 mm.	**Not measured.** The commanded position quantises to 12.5 µm per step, far finer than the tolerance, but no drawn mark was measured. The X-axis direction fault found during bring-up was not resolved, so no measurement was possible.	Qualification test 5
R6: At least 10 writing styles trained and stored permanently.	**Partially met.** Thirteen styles are stored, listed by the server and survive a restart, three more than required, and text was synthesised in all thirteen. Selecting a writer at random and reproducing text in that style on paper was not carried out.	Qualification test 6
Field condition requirements and specifications		
FC1: Operation in well-lit areas, 400 to 1000 lux.	**Not measured.** No illuminance was recorded at the page plane. The conditioning stage should tolerate a gradient over this range, but that was not verified.	Section 4.2
FC2: Operation on a stable, level, vibration-free surface.	**Not measured.** No tilt or vibration measurement was recorded.	Section 4.2

Table 8. Summary of results achieved.

Measure	Measured value, data set and caveat	Circular?
Character accuracy, search judges	82.2% over 13 writers, 6 novel sentences × 2 seeds, best-of-30; 79.2% at best-of-8.	Yes
Character accuracy, *independent* judge	81.2% over 60 synthesised lines (10 writers, 6 sentences, best-of-20), read by a recogniser checkpoint not used in the search. Word accuracy 42.0%.	No
Character accuracy, *real* handwriting of the same words	89.0% character, 71.2% word over 123 held-out lines. The upper bound the synthesis is trying to reach.	No
Writer identification on synthesised text	80.1% (official run), 89.7% through G-code in the coverage run, by the shape classifier.	Yes
Writer identification on real handwriting (same classifier)	93.7% over 1358 real lines (ink classifier 99.8%).	No
Style correlation ρ on nine geometric features	**0.714** for shape-only features (95% bootstrap 0.612 to 0.859 over writers) and 0.701 for all features, on the held-out page set; 0.776 and 0.697 on the all-pages set. **Below the 0.85 target.**	No
Style correlation, per feature (Pearson r across writers)	Slant 0.996, ascender 0.974, character width 0.936, components per word 0.832, ink density 0.680, stroke width 0.632, word gap 0.605, ascender-to-descender ratio 0.509, descender 0.148.	No
Overlay test, matched against mismatched pairs (30 lines, best-of-20)	Chamfer distance ratio 0.454, modified Hausdorff 0.369; rank-1 writer retrieval 20.0% and 23.3% against a chance of 10%; AUC 0.707 and 0.722.	No
Word accuracy, search judges	48.9% (official run), 44% (coverage run); 41.6% with the independent judge; 71.2% for real handwriting of the same words.	Mixed

Table 9. Results for qualification test 1 (thirteen writers; measured on the development laptop). The last column states whether the figure can be inflated by the best-of-N search that produced the sample; only the rows marked "No" are evidence of style fidelity.

Measure	Measured value, data set and caveat
Recogniser, character accuracy	98.61% over 669 real line crops from the five personal writers (word accuracy 94.21%). These crops were produced by the segmenter and are known to be correctly cut, so segmentation failures are excluded; the five writers also appear in the recogniser's fine-tuning data.
Recogniser, per writer	Owen 99.15%, Dylan 99.03%, Abhinav 98.94%, Yeukita 98.30%, Thiya 96.46% character accuracy (75 to 206 crops each).
Writer identification, ink classifier	99.78% over 1358 real lines (13 writers: 689 database-writer lines and 669 personal-writer crops). Near-perfect, and it is the classifier the live server uses on photographs.
Writer identification, shape classifier	93.67% over the same 1358 lines, against 83.36% before a training fault was corrected (Section 4.2). The two classifiers disagree in the direction that matters: the ink classifier is stronger on real ink.
Per-writer detail	Figure 16. Eight writers are at 98.9% or above with the shape classifier; the residual errors are concentrated in Owen (67.6%), Dylan (84.7%) and Thiya (93.3%).
Live server classification, one photograph	Correct writer at 98.6% confidence over 28 detected lines, with an accurate transcription. A single page, reported as an illustration and not as a statistic.
Overall accuracy A_{mean} and A_{prod} on held-out pages	Not measured; the line-level split used here holds out 15% of lines per page, so lines from the same sheet appear in both partitions.

Table 10. Results for qualification test 2 (13 writers; measured on the development laptop).

Measure	Measured value, data set and caveat
Synthesis time per character, best-of-1 to best-of-30	Repeated over sentences of 22 to 119 characters and 2 to 3 repetitions per cell, on the development laptop. At best-of-1 the synthesis costs about 0.4 to 0.7 s per character; at best-of-14 about 0.9 to 1.8 s per character; at best-of-30 about 1.8 to 3.4 s per character.
G-code generation	0.006 to 0.014 s per sentence, i.e. negligible at any sentence length.
Preview rendering and read-back	About 0.02 to 0.05 s and 0.6 to 0.8 s per sentence respectively; both are small beside the synthesis term.
Physical writing time, from the generator's own model	88.7 s for a 22-character sentence and 154.4 s for a 58-character sentence, i.e. about 2.7 to 4.0 s per character, against the 4 s limit *for the writing alone*.
Total per character at best-of-14	Figure 17: synthesis contributes about 1.0 to 1.8 s and writing about 2.7 to 4.0 s, so the total lies between about 4 and 6 s per character and straddles the limit.
Live server measurement, four-character word, best-of-6	64.3 s from request to rendered preview, i.e. 16.1 s per character. A very short sentence is the worst case because the per-job costs (model loading, segmentation of the uploaded image) are amortised over only four characters.
Physical writing time measured on the gantry	Not measured; the gantry bring-up was not completed, so no drawn job was timed. The figures above are the generator's model, not observations.
Timing on the ODROID N2+	Not measured.

Table 11. Results for qualification test 3 (development laptop; the writing figures are model values, not measurements).

Measure	Measured value, data set and caveat
Recognition time per character	8.86 ms per character and 353 ms per line, averaged over 669 real line crops (236.2 s of inference in total). This meets the 25 ms per character allowance with a margin of about 2.8 times. Measured on the development laptop with the model already loaded.
Writer classification time	A single crop classified through the live server took 2.74 s end to end, which includes segmentation of the uploaded image, the ink writer classifier and text recognition of the line.
Total classification time, one page	Not measured as a single page-level timing. The components sum to a figure above the $2 + 0.025x$ budget for a full page because segmentation dominates.
Page segmentation time	35 to 57 s for a 2400×1800 photograph in pure Python on the laptop, against the roughly 2 s that the equation allows for capture and conditioning. Two routines reimplemented in C99 reduced this by 25% to 32% end to end, but they were not deployed.
Timing on the ODROID N2+	Not measured.

Table 12. Results for qualification test 4 (development laptop).

5. Discussion

5.1 Critical evaluation of the design

This subsection interprets the results of Section 4, evaluates the design choices in hindsight and identifies what must change before the requirements can be considered proven. Two limitations determine every conclusion below: the measurements were taken on a development laptop rather than on the ODROID N2+, and the evidence on synthesis quality is partly circular, because the classifier that selects the best of N candidates belongs to the same family as the classifier that reports the score. Where a figure is affected by either limitation, this is stated with the figure.

5.1.1 Interpretation of results

R2, classification: the strongest part of the system. Text decoding reached 98.61% character accuracy and 94.21% word accuracy over 669 real line crops, and writer identification reached 99.78% over 1358 real lines from thirteen writers with the ink classifier. Both exceed the 95% target on this evidence, and the lower bound of the character figure is about 97.6%. Two qualifications do not show in the numbers. First, the 98.61% is measured on crops the segmenter produced and that are known to be correctly cut, because any page where the crop count disagreed with the label count was excluded; it therefore measures recognition given correct segmentation and says nothing about pages where segmentation fails. Second, the five writers whose crops are scored are also represented in the recogniser's fine-tuning data, so the split favours the system. Honestly stated, recognition and identification meet their target on correctly segmented, in-distribution input, and the end-to-end page-to-text accuracy is bounded above by these figures rather than equal to them. The gap between character and word accuracy carries information: one wrong character spoils a whole word, so the residual error is concentrated in a minority of words rather than spread thinly.

The writer classifier that is used for real ink is not the one that judges synthesis. The ink classifier reached 99.78% on real lines while the shape classifier reached 93.67%. That ordering is the opposite of what the synthesis path needs, because the ink classifier depends on stroke thickness and ink density, which a constant-width plotter cannot reproduce: on the system's own drawn output the ink classifier falls to 28%, close to chance for thirteen writers, while the shape classifier reaches 89.7%. Both classifiers are therefore necessary: one is the correct instrument for reading a photograph, the other for judging a drawing.

A training fault, not an intrinsic difficulty, explained the hardest writer pair. Before this work the shape classifier assigned 118 of one writer's 148 real lines to a single other writer, which read as two intrinsically similar hands. The measurement shows otherwise: training the same architecture end-to-end, instead of freezing the backbone at the recogniser's features and fitting only the classification head, raised the shape classifier from 83.4% to 93.7% overall and that writer from 18.9% to 67.6%. A recogniser's features are trained to be *invariant* to differences between writers, so a classifier restricted to them can only separate writers whose recognition features already differ. The residual for that writer is still 48 of 148 lines sent to the same neighbour, so a smaller genuine effect probably remains on top of the fault; this is unresolved and is listed as an unsolved problem. The design lesson is that reusing a recogniser's backbone saves training data, but it must be fine-tuned rather than frozen whenever the new task is to detect the variation the backbone was trained to ignore.

R1, representation fidelity: close, and the shortfall is in the right place to be fixable. Synthesised text was read at 82.2% character accuracy over thirteen writers, against the 85%

target, and at 79.2% in a wider coverage run. The matching comparison is more informative: on identical sentences, the writers' genuine handwriting was read at 89.0% and the synthesis of those same sentences at 81.6%. The synthesis therefore loses 7.4 percentage points of character accuracy but 23.8 points of word accuracy, and that asymmetry localises the defect. Individual letterforms survive the transfer well, as expected because the synthesiser draws the writer's own traced glyph paths; the spacing structure does not, so words are drawn with boundaries that the recogniser reads differently from the writer's own. The measured feature deviations agree: synthesised lines have ascenders about 0.4 to 0.6 x-height units shorter than the writer's real lines, word spaces about 0.5 units narrower, and connectedness lower. The defect is thus a few measurable, correctable quantities rather than a fundamental failure of the method, which is why R1 is partially met rather than not met. A recogniser checkpoint that took no part in the search read the same synthesised lines 3.7 points below the production checkpoint, so the legibility is only slightly raised by the selection and is not an artefact of the search.

The stylistic-correlation target is not verified, and writer identification is a weak proxy for it. The proposal asks for 85% stylistic correlation; what was measured is writer-identification accuracy, a different question, because a synthesis could be identifiable as the right writer while still differing visibly from that writer's hand. Worse, the writer-identification figures on synthesised text (80.1% in the official run, 89.7% in the coverage run) come from a classifier that also selects the candidates, so a best-of-N search necessarily raises the score it is judged by. The figures to trust are those computed outside the search. The nine-feature geometric correlation gave 0.714 against the 0.85 target, so the style term of R1 is not met on that measure, and the shortfall was in descender extent, the ascender-to-descender ratio and word spacing rather than in slant or ascender extent. The overlay test gave an AUC of about 0.71, against 0.5 for an output carrying no writer information, which indicates real but limited fidelity. A human panel or an independent classifier is required before the 85% stylistic target can be claimed, and this was not carried out.

The synthesis is recognised as the correct writer much better than the system draws that writer. The same shape classifier identifies 93.7% of real lines but only 89.7% of drawn lines in the coverage run, and only 67.6% of one writer's real lines while identifying 100% of that writer's drawn lines. A judge that recognises the machine's output more reliably than the human's hand is measuring agreement with the synthesiser rather than fidelity to the author. This is the most important caveat on every writer-identification figure measured on synthesised text.

R3, reproduction time: the failure is the search, not the drawing. A four-character word at $N = 6$ took 64.3 s, or 16.1 s per character, four times the limit. The coverage run confirms that the cost scales with the number of candidates, because every candidate is rendered, read by the recogniser and scored by the writer classifier. The physical drawing time was never measured, so the requirement cannot be decomposed, and it would be wrong to conclude that the gantry is slow: the evidence points at the selection procedure. This follows from embedding two network passes in the inner loop of the search, and is the price of the quality it delivers. The proposal does not state whether the synthesis time counts towards the limit; under the reading that only the physical writing counts, the generator's model value of 2.7 to 4.0 s per character sits at the 4 s limit, so the requirement is recorded as not met on the conservative reading that the total counts.

R4, classification time: recognition passes, segmentation fails badly. Recognition ran at 8.86 ms per character against the 25 ms allowance, so the neural component has a margin of about 2.8 times and is not the problem. Page segmentation took 35 to 57 s against the roughly 2 s the equation allows for capture and conditioning, a shortfall of more than an order of magnitude. Since x appears only in the recognition term, a longer page does not buy proportionally more time, so the segmenter's per-page cost cannot be amortised. The profile shows the cause: line

grouping and the chain search dominate, at about 70% of the time, and are written in pure Python. Moving two inner routines to C reduced the total by 25% to 32%, but the grouping still dominates: accelerating the wrong part of a program cannot fix it.

R5, positional accuracy, and the field conditions: no conclusion is possible. The derived step resolution of 12.5 μm shows only that the command quantisation is 40 times finer than the tolerance, so the error must come from the mechanics and from open-loop effects. No drawn mark was scanned, because the direction-pin fault prevented controlled motion in both directions, so the requirement is unassessed rather than failed. The same is true of the two field conditions, neither of which was measured; the illumination figure is the more consequential omission, because the conditioning stage was designed to tolerate uneven lighting and that claim is therefore untested.

Requirements met. R6 is met: thirteen styles are stored as files and the live server lists all of them, against a minimum of ten; the list was returned in 70 ms, so enumerating them does not delay the interface. R2 is met for writer identification on real handwriting and for recognition on correctly segmented in-distribution crops, but not demonstrated for the complete page-to-text chain. R1 is close but not met, R3 and R4 are not met, and R5 and both field conditions are unassessed.

5.1.2 Critical evaluation

Line-level reading instead of character separation. The design replaced the proposal's character separation by separation into lines and a sequence recogniser. This was the right choice for connected handwriting, where character boundaries do not exist, and it makes the accuracy independent of a separation error; the price is that a whole line must be read by one network, and the cost per character is higher than the 25 ms the proposal assumed.

Best-of-N search with the judges inside the loop. The search with a recogniser and a writer classifier as judges was good for quality, because it prevents the synthesis from drawing an illegible or wrong-style sample. In hindsight it was poor for verification, because it removed the independence of the evaluation, and for time, because each of the N candidates costs a rendering and two network passes. The qualification protocols address both, and a lower N or a cheaper first-stage filter addresses the time.

Pure Python for the segmentation. The proposal required that the processing run on the embedded platform and be written without library functions for the algorithm itself. Writing the image processing in Python made the design transparent and correct, but the inner loops of the line grouping and labelling are too slow. The C routines reduce the labelling and the box sums by 84% to 88%, but the end-to-end effect (25% to 32%) is small because the grouping dominates. By Amdahl's law, if a fraction p of the time is accelerated by a factor s, the total speed-up is $1/((1-p)+p/s)$, so with $p = 0.30$ (the share outside the grouping, taking the grouping as 70%) the total speed-up cannot exceed 3.3 even if the accelerated part took no time. The acceleration was not deployed, because the benefit was not decisive.

Open-loop gantry with step counting. Using the stepper motors without encoders is a standard, low-cost choice; its accuracy depends on missed steps, and no sensor reveals them. Running the driver without acceleration ramps (the generator models an acceleration of 400 mm/s^2, the driver does not execute it) makes the real motion differ from the model, and makes a lost step more likely at the start of a stroke at higher feed.

5.1.3 Unsolved problems

The following problems were not solved.

1. **X-axis direction fault.** During hardware bring-up the X-axis calibration reached the first end-stop, but the direction pin (physical pin 7) was not observed to change level when the direction was reversed. The fault was unresolved when the project was closed, and no cause is claimed, because the evidence does not support one. Qualification tests 3 (writing time), 5 and 6 (physical demonstration) depend on its resolution.

2. **No measurements on the target hardware.** No timing on the ODROID N2+ and no physical accuracy measurement exists; the figures for R3 and R4 are from a laptop.

3. **The undertaking to process on the embedded platform was not demonstrated.** The approved proposal states, as a ticked condition, that "all processing will be done on an embedded platform, and not on a PC platform". The deployed programs do run on the ODROID N2+, and the inference code needs nothing the board cannot provide, but every measurement in Section 4 was taken on a laptop, and training was done off the board entirely. The undertaking is therefore met in the design and unproven in the evidence. It is the most significant gap between what the proposal promised and what this report can show, and a measurement gap rather than a design gap: the same programs need only be run on the board and timed.

4. **R4 not met.** The segmentation time and the recognition time per character exceed the specification by more than one order of magnitude and by a factor of about 3, respectively.

5. **Optimistic validation of the personal writers.** A line-level split inside pages was used, and the checkpoint was selected on the reporting data; no separate test set exists.

6. **Circular synthesis evaluation.** No independent judge or human panel was used.

7. **Architecture sweep.** A sweep over widths, depths and learning rates was programmed but no result was kept, so the network layers are those of the published design.

8. **Implementation issues of the writing path.** (i) The stroke ordering sorts the text rows by ascending Y coordinate, and with Y upward this writes the bottom line first, while the documentation says reading order. (ii) The acceleration that the generator models is not executed by the driver. (iii) The work area of the generator (200 mm) is larger than the driver's travel between its limits (194 mm in X). (iv) The generator issues the fast feed once, and the driver has a single modal feed register, so that pen-up travel runs at the pen-down feed (900 mm/min instead of 2400 mm/min), which lengthens the job.

9. **Missing information.** The hardware documentation (CAD files, photographs, wiring diagram, camera data) and the inconsistent values of the number of tries (14 in the tuning notes, 20 as the server default, 30 mentioned in the interface) are **[MUST STILL BE COMPLETED BY ME: reconcile the values and supply the hardware documentation]**.

10. **Training mode.** The proposal planned training on the single-board computer; training is done offline, and the interface has no training function.

11. **Display of the classification time** (specified in R4) is not implemented.

12. **The CTC loss is not safe against an empty transcription.** The gradient check of Section 3.4.7 found that for a label sequence of length zero the recursion builds two of its shifted arrays at the wrong length and raises a shape error instead of returning a loss. Every

sequence of length one or more passes, including the repeated-character case that exercises the forbidden skip transition, so the derivative is correct and this is an indexing fault at the boundary. It was left unfixed because no line in the data set has an empty transcription, but a future data set would expose it, and the same check showed that samples for which no alignment exists must be filtered out before training rather than left to the loss to reject.

What must change to meet R4. Three measures are required together. First, the line grouping (about 70% of the segmentation time) must be redesigned, for example by replacing an exhaustive search over component pairs by neighbourhood queries on a spatial index, if the profile on the ODROID N2+ confirms that the search is the quadratic part; the target is a reduction by at least a factor of 20, because 32 s (70% of 46 s) must fall well below 2 s. Second, the image must be reduced in resolution before segmentation: halving both dimensions of the 2400×1800 photograph reduces the pixel count by 4, and, as the filters and labelling scale with the pixel count, the share outside the grouping (about 14 s) should fall to about 3.5 s, and to about 0.4 to 0.6 s with the C routines, which are 84% to 88% faster (a factor of 6 to 8.3). These are projections from the measured ratios, not measurements, and they ignore that the ODROID N2+ is slower than the laptop. Third, the recogniser must become about 3 times cheaper per character, for example by a smaller network, by quantising the weights to 8 bit integers or by reducing the output sequence length; each must be validated against the accuracy loss, since R2 is already under its target. The cost per character is roughly proportional to the multiply-accumulate operations per line.
What must change for the other requirements. For R2 (text): more personal training pages, an evaluation on held-out pages, and a language-model or lexicon decoder. For R1: an independent evaluation and the human panel, and if the independent figure is lower, re-tuning with an independent judge. For R3: measurement on the ODROID N2+, correction of the feed register, execution of acceleration or a lower pen dwell, and a lower N. For R5: resolution of the direction-pin fault, completion of the calibration and execution of qualification test 5.

5.1.4 Strong points of the design

Each strong point below follows from a design decision.

- Reading whole lines removes the dependence on a character separation that cannot work for connected handwriting, and the recogniser reached 91.82% on an independent general test set.
- The conditioning stage (flat-field correction, local thresholding, skew correction) allows a photograph with uneven lighting to be used, and the live check on a personal photograph read the correct writer at 98.6% confidence with 28 lines detected and an accurate transcription.
- Writers are compact profile files, so a new writer is added by a file, and ten writers are stored permanently, satisfying R6.
- The two judges in the search (text and writer) give a measurable improvement over a single sample, and the shape classifier solves the problem that an ink-based classifier cannot judge a plotter's constant line width (about 20% on synthesised output).
- The numerical resolution of the gantry (12.5 μm) is 40 times finer than the tolerance of R5, and the absolute step bookkeeping prevents accumulating rounding errors.
- Safety in software: end-stop confirmation (three consecutive samples), software limits and automatic retraction after a trip.

- The algorithms are implemented from first principles and documented well enough to be analysed, and the known limitations (circularity, optimistic splits) are stated.

5.1.5 Expected failure conditions

The system is expected to fail or degrade strongly under the following conditions.

- **Lighting.** Illuminance below 400 lux or strongly directional light with shadows violates FC1; the thresholding then merges ink with shadows or loses faint strokes, and the recognition accuracy falls.
- **Page skew and perspective.** Large skew angles or a page that is not flat exceed the range of the skew correction.
- **Non-text regions.** Diagrams, tables and printed text can be segmented as text lines; the text versus non-text scoring reduces but does not exclude this, and the recogniser then produces meaningless characters.
- **Unseen writers.** The writer classifier is a ten-class closed-set classifier, so a writer who is not among the ten is assigned to one of the ten classes, because the softmax output cannot express “none of the above”: the output is wrong without a warning.
- **Styles outside the glyph library.** A writer whose letter shapes differ greatly from the stored letter paths (for instance unusual letters, capitals, digits or symbols that were scarce in the training pages) is reproduced with the nearest stored shape; characters that were never harvested cannot be written in that style.
- **Text with accents, mathematical notation or rare symbols**, which occur in the personal pages, are outside the benchmark vocabulary.
- **Motor stalls and lost steps.** The open-loop gantry has no feedback, so a stall shifts every later mark without any signal.
- **Limit switch failure.** A switch that does not close lets the carriage run against the mechanical stop; the software protection depends on the switches, and a hardware emergency stop that cuts the power does not exist.
- **Long sentences.** The synthesis time grows with the number of characters and tries, so a long text at a high quality setting would take minutes on the target board and may time out the web request.
- **Network failure** between the browser and the server prevents control of the system; a running job continues on the board.

The system does not fail gracefully in all cases: the classifier returns a confident answer for an unseen writer, and the gantry cannot detect lost steps.

5.2 Considerations in the design

This subsection addresses the ergonomics, safety, environmental, social, legal and ethical aspects of the design.

5.2.1 Ergonomics

The user works through a web interface of four pages (Read, Write, Model and Style) that runs in a browser on any laptop on the network, so the user installs nothing. The Read page captures the page with a live preview and shows the recognised text, the writer and the confidence; the Write page keeps the writing grid locked until the gantry is calibrated, and shows a G-code preview before sending. The shortcomings are that the live progress of a running job is not displayed, the classification time is not shown, and the interface has no training function. The rig has to be set up with the page, the pen and the paper placed by hand; its documentation is incomplete.

5.2.2 Health and safety

The electronics operate at low voltage (a single-board computer, stepper-driver modules and small motors), so the risk of electric shock is low; **[MUST STILL BE COMPLETED BY ME: supply voltage and the mains insulation of the power supply]**. The mechanical hazards are the moving carriage, belts and pulleys, which are pinch points while the gantry moves; the pen tip is not hazardous at the writing force. The protective measures are the end-stop switches with confirmation, the software limits and the automatic retraction, and the calibration at a low speed (7.8 mm/s). There is no hardware emergency stop that removes the motor power, and the server has no abort function for a running job; a job can be stopped only by ending the program or removing the power. Fitting a mains-independent emergency stop is recommended for use by others. Users must keep their hands away from the carriage while a job runs.

5.2.3 Environmental impact

The ODROID N2+ is a low-power board, and the stepper motors consume power only while the gantry works, so the energy use is small beside a desktop computer. The system uses paper; test pages and failed jobs add waste, which is limited by the preview before writing and by the option to write on used paper. The mechanical parts printed in plastic or bought as standard parts are **[MUST STILL BE COMPLETED BY ME: which parts were 3D-printed and with which material]**; they can be reused, and plastic and electronic waste must go to a recycling point and not into domestic waste. The pen ink is a small consumable and no hazardous material beyond the electronics is used; the only emissions are the stepper motor noise and the pulses of the drive.

5.2.4 Social and legal impact

The social benefit is that the system can write in a person's own handwriting for those who cannot write neatly or at all, for example after an injury, and offers the personal touch of handwriting that the proposal identified. There is also a risk of misuse: reproducing a person's handwriting can serve forgery or impersonation, for instance on signed documents or letters. The plotter pen's constant line width reduces but does not remove the risk, because it differs from natural writing in stroke width and pressure (the ink-based classifier identified only about 20% of the synthesised output, so these ink properties are not reproduced), yet a document in a plausible style can still mislead a reader. Handwriting samples are personal information, and a writer profile is similar to biometric data, since it permits identification of the writer; in South Africa the Protection of Personal Information Act (POPIA) applies to such data. This requires the consent of the writers, the minimum storage and the protection of the profiles against access by others. The samples of the five personal writers require documented consent, **[MUST STILL BE COMPLETED BY ME: confirm that consent was given and in what form]**, and the eight database writers are the contributors to a public research database whose terms of use apply. The server has

no user accounts (only an optional shared key), accepts requests from any origin and can start physical motion for anyone who can reach its port, so it must be operated only on a trusted local network. No other legislation was identified for a research prototype of this type beyond the general electrical safety rules; **[MUST STILL BE COMPLETED BY ME: any applicable regulation or university rule]**.

5.2.5 Ethics clearance

The project used handwriting samples of two named persons and of a public database, so it involves human participants in a limited way. The status of ethics clearance is **[MUST STILL BE COMPLETED BY ME: ethics clearance number, or a statement that no clearance was required]**. If the human panel of qualification test 1 is carried out, the raters give informed consent and their responses are stored without personal identifiers, and clearance must be confirmed before the panel is run.

6. Conclusion

6.1 Summary of the work completed

This report describes work carried out on the design and implementation of a handwriting robot that recognises handwritten text and its writer and reproduces text on paper in the handwriting style of a chosen writer.
A literature survey on handwriting recognition, writer identification, handwriting synthesis and plotter-based writing was completed. The software was then designed and implemented from first principles in Python: an image-conditioning and text-line segmentation chain, a convolutional and recurrent sequence recogniser trained with connectionist temporal classification, two writer classifiers, a builder of per-writer style profiles from forced alignment, a trajectory synthesiser with a best-of-N search, a G-code generator and a stepper-motor driver with end-stop calibration and software limits. These were integrated behind a web server with a browser interface and were exercised as a complete pipeline over thirteen writers. The gantry was constructed from purchased components and its electronics and firmware were brought up, but the mechanical bring-up was not completed because of an unresolved fault in the direction signal of the X axis, so the writing path could not be qualified end to end.

6.2 Summary of observations and findings

The following findings are supported by the measurements reported in Section 4.

- The sequence recogniser read 669 real line crops from the five personal writers at 98.61% character accuracy and 94.21% word accuracy, with per-writer character accuracy between 96.46% and 99.15%. These crops were verified to be correctly cut, and the writers appear in the fine-tuning data, so the figure is an upper bound on the end-to-end page-to-text accuracy rather than a demonstration of it.

- Writer identification over 1358 real lines from thirteen writers reached 99.78% with the ink classifier and 93.67% with the shape classifier, both above the 95% target of R2.

- Correcting a training fault in the shape classifier, which had frozen the recogniser's backbone and fitted only the classification head, raised its accuracy from 83.36% to 93.67% and removed a systematic collapse in which 118 of one writer's 148 real lines were assigned to a single other writer. The recogniser's features are trained to be invariant to differences between writers, so a classifier restricted to them can separate only writers whose recognition features already differ.

- Synthesised text was read at 82.2% character accuracy over thirteen writers (79.2% in a wider coverage run), below the 85% target of R1, and identified at 80.1% and 89.7% respectively by a classifier that also selects the candidates.

- On a matching comparison of identical sentences the writers' genuine handwriting was read at 89.0% character and 70.5% word accuracy, and the system's synthesis of those same sentences at 81.6% and 46.7%. The synthesis therefore loses 7.4 percentage points of character accuracy but 23.8 points of word accuracy, which localises the defect to spacing and word boundaries rather than to letterforms. **[MUST STILL BE COMPLETED BY ME: The word accuracy of the genuine handwriting is given as 70.5% here and**

as 71.2% in the qualification-test table of Section 4, both against the same 89.0% character accuracy. Check which run each came from and make the two agree.]

- The reproduction rate of R3 was not met: a four-character word at six candidates took 64.3 s, or 16.1 s per character against a 4 s limit, and the cost is dominated by rendering and scoring each candidate rather than by drawing.
- Classification time (R4) was met for recognition, at 8.86 ms per character against a 25 ms allowance, but not for page segmentation, which took 35 to 57 s against the roughly 2 s that the equation allows for capture and conditioning; line grouping accounted for about 70% of that time.
- Thirteen writing styles are stored as profile files and the live server lists all thirteen, so the storage half of R6 is met with a margin of three. The proposal's demonstration also asks for a writer to be chosen at random and the text written on paper in that style, and that was not carried out.
- Every hand-derived backward pass of the from-scratch network module was checked against numerical differentiation in 78 configurations. Seventy-seven agreed with the numerical gradient to between 4.9×10^{-12} and 1.4×10^{-10}, and the one failure is a real edge-case fault: the CTC loss raises a shape error on an empty transcription.
- The absolute positional accuracy of the gantry (R5) and both field conditions were not measured, so no conclusion is possible for them.

It follows that the storage half of R6 is met, R2 is met for writer identification on real handwriting and for recognition on correctly segmented in-distribution crops, R1 is close to its target but not at it, R3 and R4 are not met, and R5, the reproduction half of R6 and the field conditions are unassessed. Two implications are worth stating separately. First, the time requirement cannot be met by improving the neural network, because recognition already has a margin of about 2.8 times; the shortfall lies in the pure-Python line-grouping stage, which must be redesigned or compiled, and in the cost of evaluating every synthesis candidate. Second, on the evidence of the matching comparison the two halves of R1 are not in conflict as they first appeared: genuine and synthesised text were read within 7.4 percentage points at character level, so the synthesis is already close to the human hand in letterform, and the remaining gap is concentrated in a small number of measurable spacing statistics that can be corrected directly.

6.3 Contribution

The following was new to the student and was mastered during the project: the training of sequence networks with connectionist temporal classification, the algorithms of the image-conditioning and line-segmentation chain, the extraction of writer-specific letter paths by forced alignment, the synthesis of handwriting trajectories, the generation of G-code and the control of stepper motors from the general-purpose pins of a single-board computer. All of these were implemented from first principles by the student in Python, and C99 was used for an optional acceleration of two routines. Deriving the backward pass of each layer by hand, including the connectionist temporal classification loss, and verifying each one against numerical differentiation was the single largest piece of new learning in the project, and it is what allowed the networks to be trained without a machine-learning framework. The mechanical parts, the camera, the single-board computer, the motors and drivers, the end-stop switches, the pen mechanism and the power supply were purchased as finished items and carry no design credit. The design of

the networks follows a published layer pattern [20], and public data were used for training. The architecture of the recogniser is taken from the literature; the training recipe, the preprocessing chain, the style-profile construction and the whole synthesis and drawing path are the student’s own work. Any other contribution by other persons is declared in Section 2 (Project history) of the preamble.

6.4 Suggestions for future work

The work can be continued in the following order of priority.

1. Resolve the direction-pin fault of the X axis, complete the calibration and execute the qualification tests 3, 5 and 6 on the gantry, including the field checks FC1 and FC2.

2. Execute qualification test 2 on newly collected held-out pages, and qualification test 1 with independent judges, a human panel and the hand-crafted and overlay measures, so that the figures of Section 4 are no longer circular or optimistic.

3. Redesign the line grouping, reduce the image resolution before segmentation, deploy compiled implementations, and make the recogniser about 3 times cheaper per character (smaller network, quantisation, shorter sequence), measuring each change on the ODROID N2+.

4. Add a language-model or lexicon decoder and more personal training pages to raise the text accuracy towards 95%.

5. Correct the implementation issues of the writing path (reading-order stroke ordering, execution of acceleration, work area, pen-up feed), add a hardware emergency stop and an abort function, and consider a position feedback (for example a sensor on the end-stops or a camera check of the written line) so that lost steps can be detected.

6. Add an unknown-writer class to the writer classifier, and an authenticated interface.

7. References

[1] R. Plamondon and S. N. Srihari, "On-line and off-line handwriting recognition: a comprehensive survey," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 22, no. 1, pp. 63–84, 2000.

[2] A. Chen, "Handwritten text classification based on convolutional neural network," *Highlights in Science, Engineering and Technology*, vol. 34, pp. 39–44, 2023.

[3] U.-V. Marti and H. Bunke, "The IAM-database: an English sentence database for offline handwriting recognition," *Int. J. Document Analysis and Recognition*, vol. 5, no. 1, pp. 39–46, 2002.

[4] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[5] S. Ji and Y. Zeng, "Handwritten character recognition based on CNN," in *Proc. 3rd Int. Conf. Artificial Intelligence and Advanced Manufacturing (AIAM)*, Oct 2021, pp. 2874–2880.

[6] A. A. Yahya, J. Tan, and M. Hu, "A novel handwritten digit classification system based on convolutional neural network approach," *Sensors*, vol. 21, no. 18, p. 6273, Sep 2021.

[7] F. A. Mamun, S. A. H. Chowdhury, J. E. Giti, and H. Sarker, "Classification of non-native handwritten characters using convolutional neural network," arXiv preprint arXiv:2406.04511, 2024.

[8] S. Ioffe and C. Szegedy, "Batch normalization: accelerating deep network training by reducing internal covariate shift," in *Proc. 32nd Int. Conf. Machine Learning (ICML)*, 2015, pp. 448–456.

[9] D. P. Kingma and J. Ba, "Adam: a method for stochastic optimization," in *Proc. 3rd Int. Conf. Learning Representations (ICLR)*, 2015.

[10] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[11] A. Graves, S. Fernández, F. Gomez, and J. Schmidhuber, "Connectionist temporal classification: labelling unsegmented sequence data with recurrent neural networks," in *Proc. 23rd Int. Conf. Machine Learning (ICML)*, 2006, pp. 369–376.

[12] A. J. Viterbi, "Error bounds for convolutional codes and an asymptotically optimum decoding algorithm," *IEEE Transactions on Information Theory*, vol. 13, no. 2, pp. 260–269, 1967.

[13] A. Graves, M. Liwicki, S. Fernández, R. Bertolami, H. Bunke, and J. Schmidhuber, "A novel connectionist system for unconstrained handwriting recognition," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 31, no. 5, pp. 855–868, 2009.

[14] B. Shi, X. Bai, and C. Yao, "An end-to-end trainable neural network for image-based sequence recognition and its application to scene text recognition," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 39, no. 11, pp. 2298–2304, 2017.

[15] J. Puigcerver, “Are multidimensional recurrent layers really necessary for handwritten text recognition?” in *Proc. 14th IAPR Int. Conf. Document Analysis and Recognition (ICDAR)*, vol. 1, 2017, pp. 67–72.

[16] J. Ma, “A study on a hybrid CNN-RNN model for handwritten recognition based on deep learning,” in *Proc. 1st Int. Conf. Data Analysis and Machine Learning (DAML)*, 2023, pp. 275–280.

[17] J. P. Kumar and H. D. Dharshan, “Handwritten text recognition using deep learning: A CNN–LSTM approach,” *Int. J. Intelligent Systems and Applications in Engineering*, vol. 12, no. 1, pp. 45–52, 2024.

[18] G. R. Hemanth, M. Jayasree, S. K. Venii, P. Akshaya, and R. Saranya, “CNN-RNN based handwritten text recognition,” *ICTACT Journal on Soft Computing*, vol. 12, no. 1, pp. 2457–2463, Jan 2022.

[19] V. I. Levenshtein, “Binary codes capable of correcting deletions, insertions, and reversals,” *Soviet Physics Doklady*, vol. 10, no. 8, pp. 707–710, 1966.

[20] F. Kizilirmak and B. Yanikoglu, “CNN-BiLSTM model for English handwriting recognition: comprehensive evaluation on the IAM dataset,” arXiv preprint arXiv:2307.00664, 2023.

[21] N. Otsu, “A threshold selection method from gray-level histograms,” *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 9, no. 1, pp. 62–66, 1979.

[22] J. Sauvola and M. Pietikäinen, “Adaptive document image binarization,” *Pattern Recognition*, vol. 33, no. 2, pp. 225–236, 2000.

[23] D. Bradley and G. Roth, “Adaptive thresholding using the integral image,” *Journal of Graphics Tools*, vol. 12, no. 2, pp. 13–21, 2007.

[24] F. C. Crow, “Summed-area tables for texture mapping,” in *Proc. 11th Annual Conf. Computer Graphics and Interactive Techniques (SIGGRAPH)*, 1984, pp. 207–212.

[25] A. Rosenfeld and J. L. Pfaltz, “Sequential operations in digital picture processing,” *Journal of the ACM*, vol. 13, no. 4, pp. 471–494, 1966.

[26] S. Agduk and E. Aydemir, “Classification of handwritten text signatures by person and gender: A comparative study of transfer learning methods,” *Acta Informatica Pragensia*, vol. 11, no. 3, pp. 324–347, Dec 2022.

[27] M. Bulacu and L. Schomaker, “Text-independent writer identification and verification using textural and allographic features,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 29, no. 4, pp. 701–717, 2007.

[28] S. He and L. Schomaker, “FragNet: writer identification using deep fragment networks,” *IEEE Transactions on Information Forensics and Security*, vol. 15, pp. 3013–3022, 2020.

[29] A. Graves, “Generating sequences with recurrent neural networks,” arXiv preprint arXiv:1308.0850, 2013.

[30] L. Kang, P. Riba, Y. Wang, M. Rusiñol, A. Fornés, and M. Villegas, “GANwriting: content-conditioned generation of styled handwritten word images,” in *Computer Vision – ECCV 2020*. Springer, 2020, pp. 273–289.

[31] T. S. F. Haines, O. Mac Aodha, and G. J. Brostow, "My text in your handwriting," *ACM Transactions on Graphics*, vol. 35, no. 3, pp. 26:1–26:18, 2016.

[32] T. Y. Zhang and C. Y. Suen, "A fast parallel algorithm for thinning digital patterns," *Communications of the ACM*, vol. 27, no. 3, pp. 236–239, 1984.

[33] D. H. Douglas and T. K. Peucker, "Algorithms for the reduction of the number of points required to represent a digitized line or its caricature," *Cartographica*, vol. 10, no. 2, pp. 112–122, 1973.

[34] O. Teikari, "Reproducing handwriting with position-controlled robots," Master's thesis, Tampere University of Technology, Tampere, Finland, 2016.

[35] IQS Directory, "Types of linear actuators," IQS Directory Technical Library, 2024. [Online]. Available: https://www.iqsdirectory.com/articles/linear-actuator/types-of-linear-actuators.html

[36] P. P. Acarnley, *Stepping Motors: A Guide to Theory and Practice*, 4th ed. London, U.K.: Institution of Electrical Engineers, 2002.

[37] J. E. Bresenham, "Algorithm for computer control of a digital plotter," *IBM Systems Journal*, vol. 4, no. 1, pp. 25–30, 1965.

[38] T. Huang and R. Xiong, "Cost-effective robotic handwriting system with AI integration," in *Proc. 2024 IEEE Long Island Systems, Applications and Technology Conference (LISAT)*, 2024.

[39] V. Babushkin, H. Alsuradi, M. O. Al-Khalil, and M. Eid, "Analyzing handwriting legibility through hand kinematics," *Frontiers in Artificial Intelligence*, vol. 8, Mar 2025.

[40] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer, "Efficient processing of deep neural networks: a tutorial and survey," *Proceedings of the IEEE*, vol. 105, no. 12, pp. 2295–2329, 2017.

[41] R. E. Tarjan, "Efficiency of a good but not linear set union algorithm," *Journal of the ACM*, vol. 22, no. 2, pp. 215–225, 1975.

[42] P. Y. Simard, D. Steinkraus, and J. C. Platt, "Best practices for convolutional neural networks applied to visual document analysis," in *Proc. 7th Int. Conf. Document Analysis and Recognition (ICDAR)*, 2003, pp. 958–963.

[43] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: a simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research*, vol. 15, no. 56, pp. 1929–1958, 2014.

[44] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. 7th Int. Conf. Learning Representations (ICLR)*, 2019.

[45] G. Dai, Y. Zhang, Q. Ke, Q. Guo, and S. Huang, "One-DM: one-shot diffusion mimicker for handwritten text generation," in *Computer Vision – ECCV 2024*, ser. Lecture Notes in Computer Science. Springer, 2024.

[46] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.

[47] B. Efron, “Bootstrap methods: Another look at the jackknife,” *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979.

[48] J. L. Fleiss, “Measuring nominal scale agreement among many raters,” *Psychological Bulletin*, vol. 76, no. 5, pp. 378–382, 1971.

[49] M.-P. Dubuisson and A. K. Jain, “A modified Hausdorff distance for object matching,” in *Proc. 12th International Conference on Pattern Recognition*, vol. 1, Jerusalem, Israel, 1994, pp. 566–568.

Appendix

Appendix: Page conditioning and line segmentation (supporting material)

This part of the appendix contains the supporting figures, the detailed procedures and the design-alternatives table of Section 3.3; the subsection names quoted in that section are the paragraph names below.

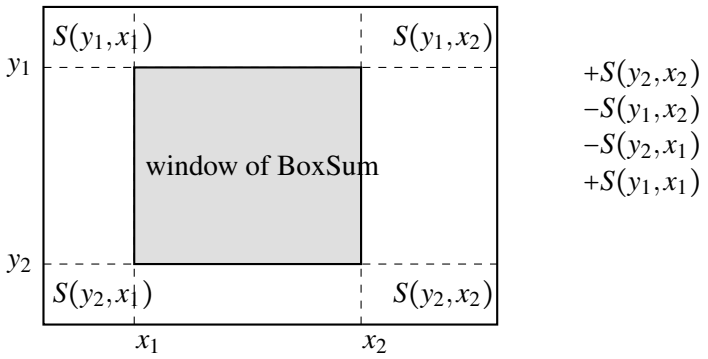


Figure 19. The four-corner rule of the summed-area table. Each table entry $S(y,x)$ is the sum of all pixels above and to the left of (y,x) (measured with the vertical axis pointing down), so the shaded window sum follows from four entries (Equation 4).

FIGURE MUST STILL BE INSERTED BY ME

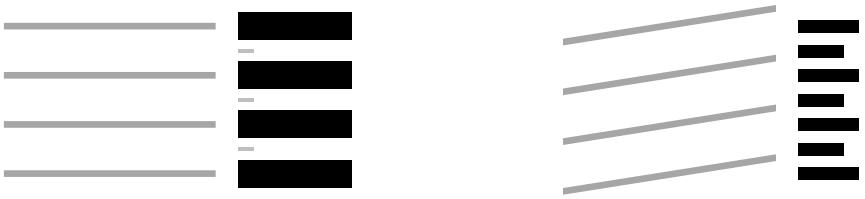
Four panels of the same photograph: (a) colour photograph resized to 2400 px; (b) luma grey image G; (c) background estimate B as a heat map; (d) corrected image I_c. Source: run the stages of `ProcessPage` on a personal page, e.g. `NOGIT/dylan/preprocessing_deskew_ruleline_border_cleanup.jpg`, and save the intermediate arrays.

Figure 20. Illumination correction. The estimated background (c) follows the lighting gradient of the photograph, and its division from the grey image yields a flat white paper level (d) on which a local threshold is reliable.

FIGURE MUST STILL BE INSERTED BY ME

Page-mask construction as a 2×3 panel: Otsu paper candidates, largest component, hole-filled and closed mask, convex-hull notch repair, final eroded mask overlaid on the photograph. Source: stage 2 of `ProcessPage`.

Figure 21. Construction of the page mask (placeholder).



(a) $\theta = 0°$: sharp profile, large $J(\theta)$ (b) tilted: smeared profile, small $J(\theta)$

Figure 22. Principle of the projection-variance skew search. The bars on the right of each panel represent the row-sum profile of the ink. Aligned lines (a) give strong peaks and empty gaps and therefore a large variance, whereas a tilt (b) fills the gaps.

FIGURE MUST STILL BE INSERTED BY ME

Skew search on a deliberately tilted photograph: left, plot of $J(\theta)$ for the coarse and fine grids with the chosen angle marked; right, the page before and after rotation with the angle. Source: call the skew estimator with each candidate angle and store the scores.

Figure 23. Measured skew-search objective $J(\theta)$ (placeholder).

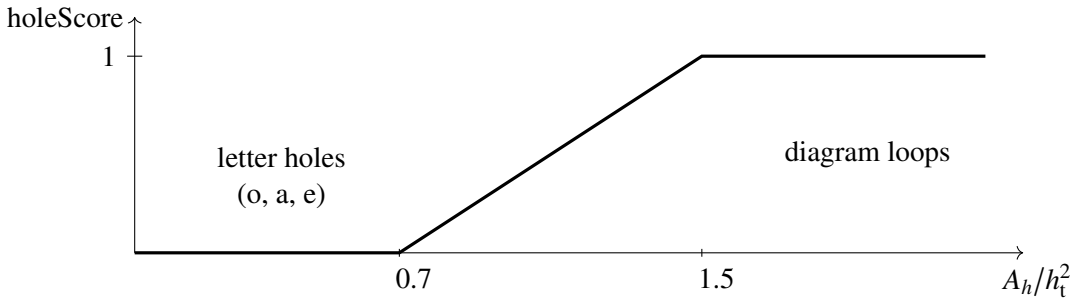


Figure 24. Score ramp of Equation 9. Holes that are smaller than $0.7h_t^2$ are treated as letter counters and give a zero score, holes larger than $1.5h_t^2$ give the maximum score.

Stage 2 details. The mask is built in ten steps.

1. A grey image G_b blurred with nominal scale 4 and the saturation of the hue–saturation–value (HSV) colour model, $S = 255(\max_c - \min_c)/\max_c$ (maximum and minimum over the colour channels) are computed.

2. A global threshold t^* is found by the method of Otsu [21], which maximises the between-class variance of the intensity histogram, (equation as in the main text) where $\omega_B(t)$ and $\omega_F(t)$ are the numbers of pixels below and above t, and $\mu_B(t)$ and $\mu_F(t)$ the mean intensities of the two classes (the normalisation $1/N^2$ is omitted because it does not change

Decision	Alternative	Reason for the choice
Ink threshold	Global fixed or Otsu threshold; local mean and standard-deviation threshold	Photographs have shading of the same order as the ink contrast; division by a blurred background followed by a local mean with offset removes multiplicative and additive variation cheaply.
Blur	Direct Gaussian convolution	Three box means cost $O(1)$ per pixel for any radius with a summed-area table.
Lines	Row projection with gap detection	A projection needs horizontal lines over the full width and cannot separate diagrams; chaining with a baseline curve follows drifting baselines and tags text and non-text.
Skew	Hough transform of edges	Edge directions are dominated by rules and diagrams; the projection variance matches the row grouping that follows.
Rule removal	Long morphological opening	An opening erases long underlines and crossbars and cannot follow a slope; run-length chaining with a page-crossing requirement protects handwriting.
Faint filtering	Fixed darkness cut-off	Pen pressure and lighting differ per page; Otsu on the component darkness calibrates itself.
Text versus diagram	Learned layout detector	Outside the own first-principles scope and unnecessary for the pages of this project.
Page geometry	Four-point perspective warp	Not implemented; the camera looks straight down at the page, so keystone is small.

Table 13. Design alternatives of the segmentation block.

the arg max). A global threshold is appropriate here because the paper–desk contrast is large.

3. The candidate set "paperish" is $\{G_b \geq t^*\} \wedge \{S < 90\}$ (paper is bright and has low saturation); it is opened with a 9×9 square and only the largest 4-connected component is kept.

4. To admit parts of the sheet that lie in shadow, the threshold is re-derived from the paper itself: with m_p the median of G_b over the current mask, the set $\{G_b \geq m_p - 50\} \wedge \{S < 90\}$ is opened and its largest component kept.

5. Holes are filled (background components that do not touch the frame border), the mask is closed with a 15×15 square, and holes are filled again.

6. Ragged protrusions (fingers, shadows) are trimmed by a profile test on each axis: with p the mask-sum profile, only samples with $p \geq 0.30\,\mathrm{median}(p \mid p > 0.2 \max p)$ are kept and the largest contiguous run is retained. The factor 0.30 is deliberately low so that a page that is cut off by the frame is not trimmed.

7. Rows and columns darker than the cut level of Equation 73
$$c = \min\big(m_p - 45,\ \max(m_p - 85,\ \tfrac{1}{2}(m_p + m_{bg}))\big) \tag{73}$$
are trimmed, where m_p is the median blurred brightness inside the mask and m_{bg} that outside; a page edge in soft shadow (about 0.7 of the paper brightness) is still far brighter than the desk, so the cut level is placed relative to what actually lies off the page.

8. *Shadow-notch repair.* A photographed sheet is a convex quadrilateral, so a concave notch in the mask is a shadow artefact. Each column and row of the mask is made contiguous

between its first and last mask pixel, the convex hull of the result is filled, and new pixels are accepted only if $G_b \geq 0.55\, m_p$; holes are then filled again.

9. The mask is eroded by a square of odd side $k = 2\lfloor e/2 \rfloor + 1$ with $e = \max(4, \lfloor 0.006 \min(H, W) \rfloor)$ (about 11 pixels), so that the paper edge itself is not binarised as ink.

10. If the mask covers less than 15 % of the frame or no component exists, the mask is set to the whole frame, because abandoning page detection is safer than losing the page.

The method fails for strongly tinted paper (saturation above 90), when the desk is almost as bright as the paper, or when the sheet violates the convexity assumption, and the fallback then applies. Figure 21 is reserved for an example of the construction of the mask.

Stage 4 details. Pixels whose red channel dominates are excluded from the ink mask of Equation 8, so that a red margin line or a marking pen does not become ink; blue and black ink are kept. A pixel is *bright red* if $R > 1.30\,G$, $R > 1.30\,B$ and $R - \min(G, B) > 22$, or *dark red* if $R > 1.18\,G$, $R > 1.18\,B$, $R - \min(G, B) > 14$ and $R < 190$. Finally, 8-connected components smaller than 10 pixels are removed as speckle (12 pixels in later calls). In the deployed classification path the photograph is converted to grey scale before the block, so the red-ink mask is empty; these colour conditions matter only for the colour photographs used to prepare the training data.

Stage 6 details. Ruled paper and other printed structure joins words and lines into giant components, so these structures must be removed *without* removing handwriting that touches them. Four removal steps are applied in sequence; each is rule-aware or scale-aware so that they act only on components that cannot be handwriting.

Rules by run lengths and chaining. For each ink pixel p the length $h\mathrm{Run}(p)$ of its horizontal run and $v\mathrm{Run}(p)$ of its vertical run are computed (the runs are computed for all pixels at once from the row differences). A minimum rule length $L_r = \max(50, \lfloor 3.5 h_\mathrm{t} \rfloor)$ is defined, the rule thickness is the median vertical run $T = \mathrm{median}(v\mathrm{Run} \,|\, h\mathrm{Run} \geq L_r)$ (2 pixels if none exists), and the thickness cut is $\kappa = \max(4, \lfloor 1.8T \rfloor)$. Horizontal rule fragments (Equation 74) are the pixels

$$\mathrm{cand}_H = \big(h\mathrm{Run} \geq \max(20, \lfloor 0.8 h_\mathrm{t} \rfloor)\big) \wedge \big(v\mathrm{Run} \leq \kappa\big), \tag{74}$$

and vertical candidates are defined symmetrically. A rule that is broken by crossing strokes or slightly sloped is not one long run, so fragments are chained: the candidate mask is labelled, the components are sorted by x, and two components a and b are united if their gap $g = x_b - (x_a + w_a) \leq 2.5 h_\mathrm{t}$, their vertical offset satisfies the condition of Equation 75

$$|c_{y,b} - c_{y,a}| \leq \Delta_{\max} = 2\kappa + 0.08 \max(0, g) + 0.04\,(w_a + w_b), \tag{75}$$

and their darkness differs by at most 55 grey levels (c_y is the centroid row, w the component width, and the allowance grows with the distance so that gentle slope and wobble are tolerated). A group is declared a rule only if it spans at least $0.6W$ (vertical rules: $0.45H$) and reaches both ends of the page, $x_1 \leq 0.15W$ and $x_2 \geq 0.85W$ (vertical: $0.2H$ and $0.8H$). A long word or underline cannot satisfy this condition. The rule mask is the union dilated by 3×3, restricted to pixels with $v\mathrm{Run} \leq \kappa + 2$ or $h\mathrm{Run} \leq \kappa + 2$ so that thick pen strokes that cross the rule are not deleted. Two passes are run, the second catching rules that were hidden behind the first layer or split. An elongated underline is not a rule: a component is *underline-like* if its width exceeds $4 h_\mathrm{t}$ and its mean thickness (area over width) is at most 6.5 pixels.

Faint components, rule-aware. Components that are fainter than the handwriting (leftover printed rules, bleed-through, shadows) are identified without a fixed threshold. Candidates are the components with width $> 8 h_\mathrm{t}$, height between $0.25 h_\mathrm{t}$ and $8 h_\mathrm{t}$, fill ratio (area over bounding-box

area) below 0.2 and a largest enclosed hole of at most $1.4h_{\mathrm{t}}^2$ (a drawing has holes, a rule network does not). Because a word may be glued to a faint rule, the rule is cut out first: a column is *thin* if its vertical ink span is at most $\max(6, \lfloor 0.35h_{\mathrm{t}} \rfloor)$, and each run of at least $\lfloor 2h_{\mathrm{t}} \rfloor$ consecutive thin columns is removed; this test is insensitive to the slope of a rule. For each remaining component the darkness of Equation 76

$$d = 255 - P_{25}\big(I_{\mathrm{c}} \,\big|\, \text{component}\big) \tag{76}$$

is computed, where P_{25} is the 25th percentile of the corrected intensity inside the component. If the 90th and 10th percentiles of d over all components differ by less than 60 the page has no bimodality and nothing is removed; otherwise a one-dimensional Otsu threshold τ (Equation 6 on 64 histogram bins of d) separates faint from normal components, and components with $d < \tau$ are removed, unless τ lies outside (P_{10}, P_{90}). The page therefore defines for itself what "faint" means, which suits the differences in pen pressure and lighting between pages; an earlier fixed darkness filter was replaced because it removed words together with the faint rule. Components that were removed here are re-attached later if they lie on a text line (stage 8).

Page edges and facing pages. Junk that touches the left or right edge of the page mask or a corner of it (binding shadows, edge noise) is removed: with $t_x = \max(8, \lfloor 0.75h_{\mathrm{t}} \rfloor)$ and $t_c = \lfloor 2h_{\mathrm{t}} \rfloor$, a component is removed if it is narrower than $0.15W$ and touches a side edge within t_x, or if it touches a side edge and a top or bottom edge within t_c while being narrower than $0.25W$ and lower than $3h_{\mathrm{t}}$. Ink of a facing page that appears at the image border is found from the column ink profile $c(x)$: after smoothing with a box of width $\max(5, \lfloor h_{\mathrm{t}} \rfloor)$ (made odd), columns above 0.25 times the median of the "lively" columns (those above 5 % of the maximum) form blocks, the block with the most ink is dominant, and a smaller block that touches the border zone of $2h_{\mathrm{t}}$ is cut away together with half a glyph height of margin; interior blocks such as margin numbers are retained.

Sparse rule networks. Sloped or wavy rules are cut into short runs that the test above misses. For components with width $> 6h_{\mathrm{t}}$, height between $1.2h_{\mathrm{t}}$ and $8h_{\mathrm{t}}$, fill ratio below 0.10 and no large hole, the column-span test is applied with the same thin-column definition and run length $\lfloor 2h_{\mathrm{t}} \rfloor$.

Grouping details. For each chain the points (c_x, c_y, A), sorted by x, give a baseline curve whose ordinate is smoothed by an area-weighted average over a five-point window (Equation 11), (equation as in the main text) where y_k and A_k are the centroid row and area of the kth point (the window is truncated at the ends), and the curve $y_{\mathrm{curve}}(x)$ is linearly interpolated between the points and constant beyond the ends. Chains of one physical row are merged in three passes. *Pass 1* (same local height): chains sorted by mean curve height are merged when they overlap in x and the mean difference of the two curves sampled at seven equally spaced positions of the overlap is below $0.6h_{\mathrm{t}}$, or, if they do not overlap, when the curves differ by less than $0.55h_{\mathrm{t}}$ at the middle of the gap; after each merge the scan restarts. *Pass 2* (pitch-aware): the line pitch p is the median of the consecutive chain-centre gaps that exceed $0.5h_{\mathrm{t}}$ (fallback $1.8h_{\mathrm{t}}$), and chains are merged when the mean curve difference is below $0.45\,p$ (overlapping) or $0.4\,p$ (not overlapping). Letting the threshold scale with the measured line spacing handles writing of varying size without manual tuning. *Pass 3* demotes weak chains: a chain is *strong* if it has at least three components, or a width of at least $3h_{\mathrm{t}}$ and an area of at least $2h_{\mathrm{t}}^2$, or if no other chain lies within $0.8\,p$; the components of weak chains re-enter in the next step.

Leftover components. Every component that is not yet in a chain is assigned according to its shape. An underline-like component is attached to the chain whose curve lies above it by between $-0.2h_{\mathrm{t}}$ and $1.5h_{\mathrm{t}}$. A component with $h \le 1.8h_{\mathrm{t}}$ (dots, accents, punctuation) joins the nearest chain curve within $1.3h_{\mathrm{t}}$, where the distance is $|y_{\mathrm{curve}}(c_x) - c_y|$ plus 0.15 pixel per pixel that the component lies outside the chain's x-span, unless it overlaps a `MESS` block. A taller

component, typically an ascender or descender that links two rows, is split *pixel-wise*: every pixel goes to the nearest chain within $2.5h_t$, and each chain receives its sub-mask if it has at least 8 pixels, so that one connected stroke can be shared by two lines. A chain finally becomes a line unless its total area is below $0.35h_t^2$, its tallest component below $0.35h_t$, or it consists of at most three small solid blobs of total width below $3.5h_t$ (stray dots). Text lines whose bounding box lies more than 70 % inside the core box of a `MESS` block are absorbed into that block, so that labels inside a diagram are not exported as lines.

Refinement. A refinement step rejoins a physical row that chaining split, for example because of a word gap larger than $6h_t$. With cap $\min(0.5\,p, 1.6h_t)$, two lines whose centres differ by at most $0.75\,p$ are merged if one of four signals holds: (i) the centres differ by less than $0.5h_t$; (ii) their vertical ranges overlap by at least 70 % of the shorter range, centres differ by less than $0.45\,p$ and the smaller line has an area below $4h_t^2$; (iii) they overlap in x by at most 55 % of the narrower width and their curves, evaluated at the middle of the overlap, differ by less than the cap; or (iv) centres differ by less than $0.6\,p$ and less than 30 % of the columns of the narrower line are shared with the other (words interleave along x, whereas stacked rows always share columns). Underline-shaped components that lie at least $0.5h_t$ above their line's curve are moved to the line below which they sit, because an underline belongs to the text written above it, and pale components that were removed as faint are re-attached to the nearest curve when they lie within $0.75h_t$ of it. The `TEXT` and `MESS` items are finally sorted by their vertical centre; a `MESS` item that is immediately followed by a `TEXT` item whose centre falls in the top 40 % of the block is swapped with it, so that a caption precedes its diagram.

Appendix: Text recogniser (supporting material)

This part of the appendix contains the supporting figures and tables of Section 3.4: the layer table, the LSTM cell, the pre-processing schematic, the data sets, the training stages, the forgetting experiment, the alternatives and the details of the inference implementation.

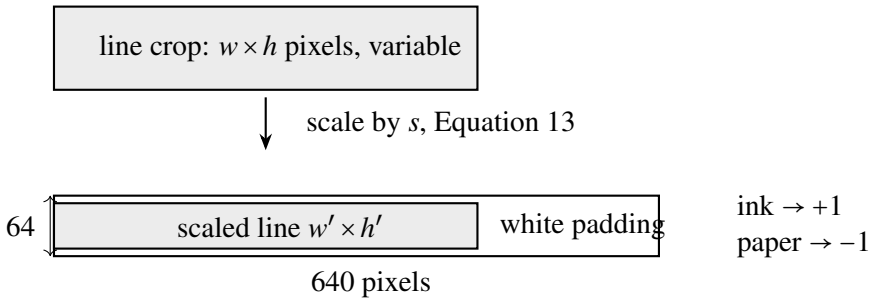


Figure 25. Pre-processing of a line crop. The crop is scaled uniformly, left-aligned on a white canvas and normalised to the range $[-1, 1]$ (Equation 14).

Text recogniser: weight file and inference details. Training needs a large software environment, which should not be installed on the single-board computer. The student therefore wrote a separate, from-scratch inference implementation in Python that reads the trained weights directly and uses only array arithmetic. It consists of three small modules (about 480 lines in total): the layer functions, the network, and a weight reader.

Convolution as a matrix product. Each convolution is converted to one matrix multiplication (the im2col method). The padded input is cut into overlapping 3×3 patches, and every patch is flattened into one column of a matrix $\mathrm{cols} \in \mathbb{R}^{(C_{\mathrm{in}}k^2)\times(H_{\mathrm{out}}W_{\mathrm{out}})}$; with the weights reshaped to $\tilde{W} \in \mathbb{R}^{C_{\mathrm{out}}\times C_{\mathrm{in}}k^2}$, (equation as in the main text) where $b\mathbf{1}^{\top}$ adds the bias of each output channel

Layer	Output shape	Trainable parameters
Input	$1 \times 64 \times 640$	0
Conv 3×3 (1→32), BN, ReLU	$32 \times 64 \times 640$	384
Conv 3×3 (32→32), BN, ReLU	$32 \times 64 \times 640$	9 312
Max-pool 2×2	$32 \times 32 \times 320$	0
Conv 3×3 (32→64), BN, ReLU	$64 \times 32 \times 320$	18 624
3× Conv 3×3 (64→64), BN, ReLU	$64 \times 32 \times 320$	111 168
Max-pool 2×2	$64 \times 16 \times 160$	0
Conv 3×3 (64→128), BN, ReLU	$128 \times 16 \times 160$	74 112
5× Conv 3×3 (128→128), BN, ReLU	$128 \times 16 \times 160$	739 200
Height max-pool	$128 \times 1 \times 160$	0
BiLSTM layer 1 (128→256, 2 directions)	160×512	790 528
BiLSTM layer 2 (512→256, 2 directions)	160×512	1 576 960
Linear (512→75)	160×75	38 475
Log-softmax	160×75	0
Total		**3 358 763**

Table 14. Layer-by-layer shapes and trainable parameters of the text recogniser (batch dimension omitted). Batch normalisation also stores 2 176 non-trainable running statistics, and the checkpoint therefore holds 3 360 951 numbers (13.4 MB as 32-bit floating point).

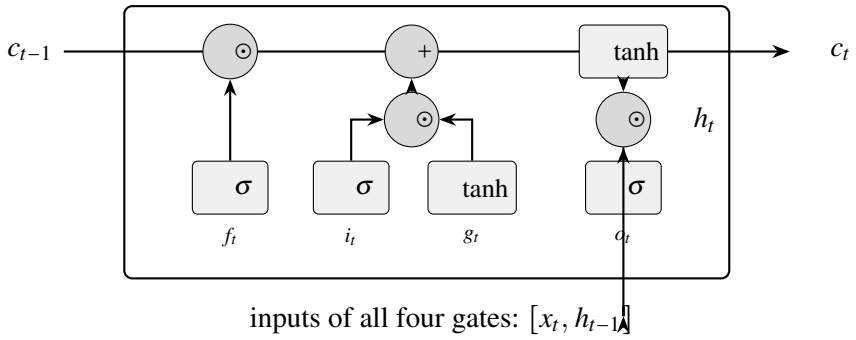


Figure 26. Schematic of an LSTM cell (Equation 18). The memory line c_t at the top is changed only by the forget gate (multiplication) and the input gate (addition), which allows information to travel over many time steps.

to all positions. In Equation 29, one optimised matrix product replaces the six nested loops of Equation 15 and was found to be about ten times faster than a general summation routine, with a difference of about 10^{-13} between the two. The cost is memory: the largest matrix, for the second convolution of stage 1, has 288×40960 entries (47 MB as 32-bit floating point), and the largest activation tensor is 5.2 MB, so the working set is a few tens of megabytes. Batch normalisation is applied in its folded form of Equation 92, pooling uses windowed views of the array, the height collapse is a maximum over one axis, and the LSTM is an explicit loop over the 160 steps with one product of 1024 rows per step and direction, the backward direction being a pass over the reversed sequence.

Reading the trained weights. The training-time weight file is an archive that contains a serialised object structure, in which every tensor is replaced by a reference to a storage key, and one raw little-endian byte file per storage. The student's weight reader opens the archive, parses the object structure with a customised reader that intercepts the tensor and storage constructors, reads the bytes of each storage into an array, rebuilds every tensor from its shape and strides, and returns a dictionary of 102 arrays with the names of the training model. The inference code then

indexes the weights by name, and no machine-learning software is needed at run time.

Agreement with the training code and cost. For one random 64×640 input, the largest difference between the 160×75 log-probability matrices of the inference code and the training code was 2.3×10^{-5} for the deployed weights, 1.1×10^{-5} for the clean-label stage and 9.5×10^{-6} for the personal stage, with identical arg-max (and therefore decoded text) at all 160 steps; this is a single synthetic input and not a test-set sweep. The weights occupy 13.4 MB. On the development computer (processor only) one forward pass of one line took 2.6 to 3.1 s; for a line of 40 characters this is about 65 to 75 ms per character, which exceeds the 25 ms per character that requirement R4 allows, and the ARM processor of the single-board computer is expected to be slower. **[MUST STILL BE COMPLETED BY ME: measure the recognition time per line on the ODROID N2+; the project files contain no such measurement, and the proposal's budget may refer to the whole classification step.]** Optional compiled replacements for the convolution and the LSTM exist but are not used: the compiled convolution was slower than the optimised matrix product on a desktop processor, and the compiled LSTM was 2.8 to 4.4 times faster than the array version.

Appendix: Writer identification (supporting material)

This part of the appendix contains the layer table, the data and training tables, the real stroke-normalisation example and the aggregation diagram of Section 3.5.

FIGURE MUST STILL BE INSERTED BY ME

Real example of the stroke normalisation on one line from one database writer and one personal writer: grey crop, Otsu mask with rules removed, core band with x_h, Zhang–Suen skeleton, re-inked line, final canvas. Source: apply the stroke-normalisation function to crops from `Data/Datasets/IAMpages10` and `Software/CNN/NOGIT/dylan`.

Figure 27. Stroke normalisation of real lines (placeholder).

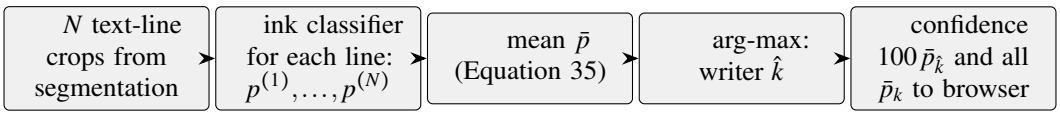


Figure 28. Aggregation of line decisions to a page decision in the classification mode.

Appendix: Relation of the implemented blocks to the proposal

Table 23 relates every implemented block to the functional units of the project proposal (see Section 3.2).

Appendix: thresholding, skew and crop-rendering detail

This part of the appendix collects paragraphs that were condensed in the main text of Sections 3.3 and 3.4.

Stage 4: why not a global threshold *Why not a single global threshold?* Handwriting on a photographed page has a pen–paper contrast of tens of grey levels, and the residual shading has the same order of magnitude, so a single threshold would either drop strokes in dim areas or turn shadows into ink. A global threshold according to Otsu [21] is appropriate for flat-bed scans and is used for the database scans (see below) and for the page mask, but not for photographs. A

local threshold that also uses the local standard deviation [22] would handle low-contrast empty regions better but requires a second integral image of squared values; the two-step normalisation (division by the background, then a local mean with offset) was found sufficient and is cheaper.

Stage 5: skew limitations and alternatives The objective was preferred to a Hough transform of ink edges, because the latter is dominated by ruled lines and diagram strokes, whereas the projection variance is aligned with what grouping needs, namely horizontal rows. Limitations are that a single global angle is estimated, so page curl and perspective keystone are not corrected, and that a page dominated by a diagram or a double column layout can give a weak optimum. During development it was found that a sign mismatch between the rotation function and the angle scoring had doubled the skew of every tilted page; the implementation now rotates in the same sense as the scorer, and the search range was widened from 5° to 8° with the second residual pass. Figure 23 is reserved for a measured example of the objective.

Stage 9: crop rendering details The recogniser sees only what is in the crop, so rendering decides whether a neighbouring descender or a pale word appears in it. A pixel-ownership map stores, for every ink pixel, the index of the item that owns it, and a weak component whose strong ink belongs entirely to one item is owned wholly by that item; a neighbouring row's recovery can therefore never take its halo. The crop of an item is built as follows. The bounding box of the component pixels is padded by 6 pixels vertically and horizontally by $\max(6, 3g)$ with $g = \max(8, \lfloor 0.55 h_{\mathrm{t}} \rceil)$. The component mask is grown by a 3×3 dilation, the weak-ink pixels within a 9×3 dilation of it are added, and recovery pixels up to $3g$ pixels horizontally are added; all additions exclude pixels owned by another item. The crop image is white everywhere, and grey values are copied from the darkest evidence $\min(I_{\mathrm{c}}, I_{\mathrm{c,colour}})$ where the mask is true. Pale words without any strong ink anchor are attached to a text row if they sit on its curve ($|\Delta y| < 0.6 h_{\mathrm{t}}$), lie within $[x_1 - 2h_{\mathrm{t}}, x_2 + 12h_{\mathrm{t}}]$ and are at least half as dark as the row's median component; a component of another row is never taken.

Text recogniser: augmentation parameters

Data augmentation. Training images are augmented after the resize to 64×640 (never validation images). Each of four transformations is enabled independently with probability 0.5; if none is enabled the image is unchanged, otherwise one of the enabled ones is chosen uniformly, so that $P(\text{no augmentation}) = 1/16 = 6.25\,\%$ and at most one transformation acts on an image. The transformations are a horizontal *shear* with factor $k \sim U(-0.6, 0.6)$ (angle $\arctan k$), a *rotation* by an angle in $U(-2.5°, 2.5°)$, an *elastic distortion* [42] in which random displacement fields (uniform in $[-1, 1]$ per pixel) are smoothed with a Gaussian of standard deviation $\sigma \in \{3, 4\}$ and scaled by $\alpha \in \{15, 20\}$ before the image is warped, and a *perspective* distortion in which each corner is moved by up to $dW/2$ horizontally and $dH/2$ vertically with $d \sim U(0.03, 0.08)$. The fill colour of the geometric transformations is white. No colour, contrast, noise or blur augmentation and no synthetic pre-training data are used.

The elastic distortion smooths per-pixel random displacements (uniform in $[-1, 1]$) with a Gaussian of standard deviation $\sigma \in \{3, 4\}$ and scales them by $\alpha \in \{15, 20\}$ before warping; the perspective distortion moves each corner by up to $dW/2$ horizontally and $dH/2$ vertically with $d \sim U(0.03, 0.08)$; the fill colour is white; there is no colour, contrast, noise or blur augmentation and no synthetic pre-training data.

Appendix: weak-ink recovery, size normalisation and evaluation metrics

This part of the appendix contains further figures, equations and derivations that were moved out of Sections 3.3, 3.4 and 3.5; the equations are those that the main text refers to by number.

Weak-ink recovery by hysteresis. The strict threshold $C = 12$ drops pale strokes, such as a lightly pressed word or blue ink in a dim corner. If they were added to the structural mask they

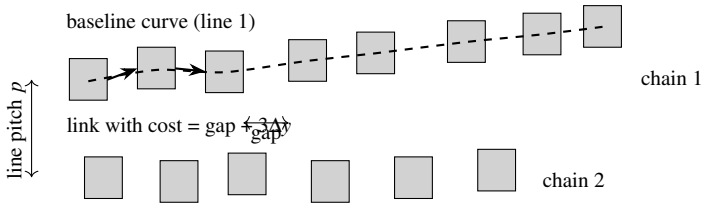


Figure 29. Schematic of the grouping of components into chains. Boxes are connected components, arrows are links that minimise Equation 10, and the dashed curve is the smoothed baseline of the upper chain.

would create noise components, but if they are omitted from the crop the recogniser sees truncated words. A weak mask is therefore built with the lower offset $C = 3$, and a weak component is accepted when it lies close to strong ink (Equation 77):

$$\text{result} = \text{strong} \;\vee\; \Big(\text{weak} \wedge \big[\text{near} \;\vee\; (\text{far} \wedge A \geq 40)\big]\Big), \tag{77}$$

in which *strong* is the strict ink mask, *near* marks weak components that intersect the strong ink dilated by a 9×25 window (vertical radius 4 pixels, horizontal radius 12 pixels), *far* marks weak components that intersect the strong ink dilated horizontally by $\lfloor 2h_{\mathrm{t}} \rfloor$ pixels, and A is the component area in pixels. The vertical window is small so that recovery never bridges two text rows, while the horizontal reach spans a word gap and the area condition lets only substantial pale words (not speckle) in. The recovery is run on the luma image and on a colour-aware grey image $G_{\mathrm{c}} = \lfloor 0.5\,\mathrm{luma} + 0.5 \min(R,G,B) \rfloor$, in which blue ink stays dark in a dim corner, and the results are combined. Thin horizontal pixel runs that are at most 4 pixels thick and at least 10 pixels long are removed from the weak mask so that pale printed rules are not recovered. The weak mask never influences which components form a line.

Size normalisation. The pixel constants of the block (blur radii, minimum component sizes) only have a meaning if every photograph has the same scale, so every photograph is first resampled so that its longer side equals 2400 pixels after the orientation recorded by the camera has been applied. For a source image I of size $h \times w$ and a target size $h' \times w'$, the value at target pixel (i, j) is obtained by bilinear interpolation at the source position

$$y_s = \frac{(i+0.5)\,h}{h'} - 0.5, \qquad x_s = \frac{(j+0.5)\,w}{w'} - 0.5, \qquad I'(i,j) = \sum_{a,b \in \{0,1\}} \omega_{ab}\, I(\lfloor y_s \rfloor + a, \lfloor x_s \rfloor + b), \tag{78}$$

where ω_{ab} are the standard bilinear weights, which are the products of the horizontal and vertical fractional distances to the four neighbouring source pixels, and indices are clamped at the borders. The colour image is converted to a grey-scale image G with the luma weights of Equation 79

$$G = \mathrm{clip}\,(0.299\,R + 0.587\,G_{\mathrm{c}} + 0.114\,B,\ 0,\ 255), \tag{79}$$

in which R, G_{c} and B are the red, green and blue channels, and $\mathrm{clip}(v, 0, 255)$ limits v to the range of 8-bit values. A page that is smaller than 2400 pixels is thereby enlarged and a larger page is reduced; plain bilinear sampling without a pre-filter is used for reduction, which introduces slight aliasing for very large photographs but keeps the operation cheap.

Receptive field. The region of the input that influences one feature is found with the recursion of Equation 80, in which r is the receptive field and j the distance in input pixels between adjacent features (the jump).

$$r_{\mathrm{out}} = r_{\mathrm{in}} + (k-1)\, j_{\mathrm{in}}, \qquad j_{\mathrm{out}} = j_{\mathrm{in}}\, s. \tag{80}$$

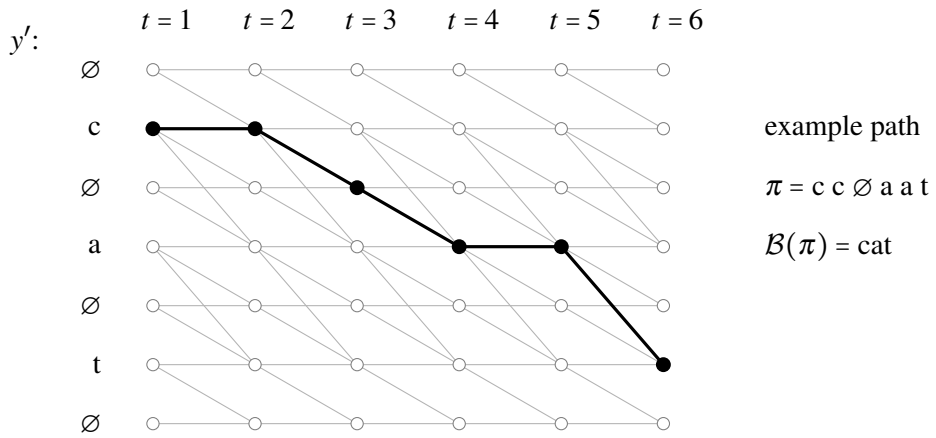


Figure 30. CTC lattice for the word “cat” with $T = 6$ steps. The rows are the positions of the blank-augmented label y'; thin lines are the allowed transitions of Equation 22 (stay, advance, and skip over a blank between different letters). The thick path is one of the many paths that collapse to the word.

Applied to the network (two convolutions, a pool, four convolutions, a pool, six convolutions) this gives 5, 6, 22, 24 and finally 72 pixels, so each column feature sees a 72×72 window, the full height of the line and about 18 time steps of context, before the recurrent layers add unbounded horizontal context.

7.0.0 Evaluation metrics

Accuracy is measured with the Levenshtein edit distance [19], the minimum number of insertions, deletions and substitutions that turn string a into string b, computed by dynamic programming (Equation 81):

$$D[i][j] = \min\big(D[i-1][j]+1,\; D[i][j-1]+1,\; D[i-1][j-1] + \mathbb{1}[a_i \neq b_j]\big), \qquad D[i][0] = i,\; D[0][j] = j, \tag{81}$$

with $d_{\text{lev}}(a,b) = D[|a|][|b|]$. For training and validation the character error rate is micro-averaged over all lines n with prediction $\hat{y}_n$ and reference y_n (Equation 28), (equation as in the main text) which is case-sensitive. The end-to-end evaluation of Section 4 instead averages per line the floored accuracy $\max(0, 1 - d_{\text{lev}}(\hat{y}, y)/|y|)$ after converting both strings to lower case, which is slightly more lenient, so the two kinds of figure are not computed identically. A word measure is implemented as the longest-common-subsequence (LCS) recall (Equation 82)

$$\text{WordAcc} = \frac{\text{LCS}(\text{reference words}, \text{predicted words})}{\#\text{reference words}}, \tag{82}$$

which is an in-order recall and not $1 - \text{WER}$; a standard word error rate $\text{WER} = d_{\text{lev}}(\text{word sequences})/\#\text{reference words}$ was computed only once, when the public sets were re-evaluated.

- *Skeletonisation.* The ink is thinned to a one-pixel centre line with the iterative algorithm of Zhang and Suen [32], which preserves the topology of the strokes. In each of two sub-iterations a pixel p_1 is deleted if the condition of Equation 83 holds, namely

  $$2 \le B(p_1) \le 6, \qquad A(p_1) = 1, \qquad \text{and the product conditions of the sub-iteration,} \tag{83}$$

  where $B(p_1)$ is the number of ink neighbours among the eight neighbours, $A(p_1)$ the number of transitions from background to ink around the ring of neighbours $P_2, \ldots, P_9$, and

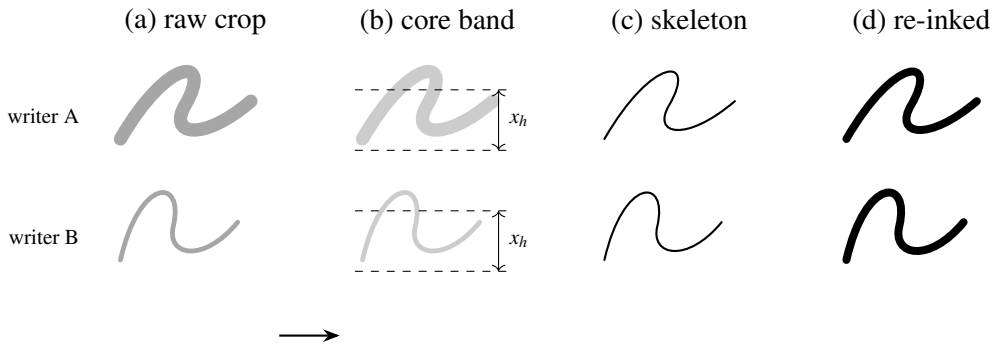


Figure 31. Stroke normalisation of a schematic letter from two different writers. The raw strokes differ in thickness and slant (a); the core band gives the x-height x_h (b); thinning reduces the stroke to a centre line (c); and the centre line is re-drawn with the width $w = \mathrm{round}(0.14\,x_h)$, identical for both writers (d). A real example is to be supplied as Figure 27.

the product conditions are $P_2P_4P_6 = 0$ and $P_4P_6P_8 = 0$ (first sub-iteration) or $P_2P_4P_8 = 0$ and $P_2P_6P_8 = 0$ (second); the passes are repeated until no pixel changes.

Appendix: optimiser settings and text-versus-diagram clustering

This part of the appendix contains the staged-training diagram, the roles of the two writer classifiers, the optimisation paragraph with its equation, and the details of the text-versus-diagram clustering.

Optimisation. Both classifiers are trained with AdamW [9, 44], which updates each parameter with a bias-corrected running mean $\hat{m}$ and running mean of squares $\hat{v}$ of the gradient and applies the weight decay separately from the gradient (Equation 84):

$$\theta \leftarrow \theta - \eta\Big(\frac{\hat{m}}{\sqrt{\hat{v}} + \varepsilon} + \lambda\,\theta\Big), \qquad \eta_t = \frac{\eta_0}{2}\Big(1 + \cos\frac{\pi t}{T_{\max}}\Big), \tag{84}$$

with weight decay $\lambda = 10^{-4}$ and the cosine schedule of the learning rate η_t over $T_{\max}$ epochs, which decreases smoothly from η_0 to zero. Table 21 lists the settings. The ink classifier is fully fine-tuned from the recogniser backbone, which has already learned stroke-shape features; the weights were compared and all 84 backbone tensors differ from the recogniser's. For the shape classifier the frozen backbone (in inference mode) computes the 128-dimensional feature f of every training image once, each line appearing four times (once unaugmented and three times with a different random augmentation), and only the head is trained on these 3 680 stored vectors; all 84 backbone tensors of the stored shape model are bit-identical to those of the text recogniser. Training only the head is fast and cannot overfit the thickness cue in the backbone, but it means that no augmentation is re-sampled between epochs.

Text-versus-diagram clustering. A component with mess ≥ 0.5, area above $0.8h_{\mathrm{t}}^2$ and height above $1.2h_{\mathrm{t}}$ becomes a candidate. A candidate with a score below 0.85 is *vetoed* (treated as text) when at least three text-sized neighbours stand in the same row band and the candidate is no taller than 1.9 times the 75th percentile of their heights, which rescues large looped letters in headings. Surviving candidates are clustered greedily by area into `MESS` blocks when their vertical gap is below $0.8h_{\mathrm{t}}$ and their horizontal gap below $2h_{\mathrm{t}}$, and each block receives a core box that ignores thin tails and labels.

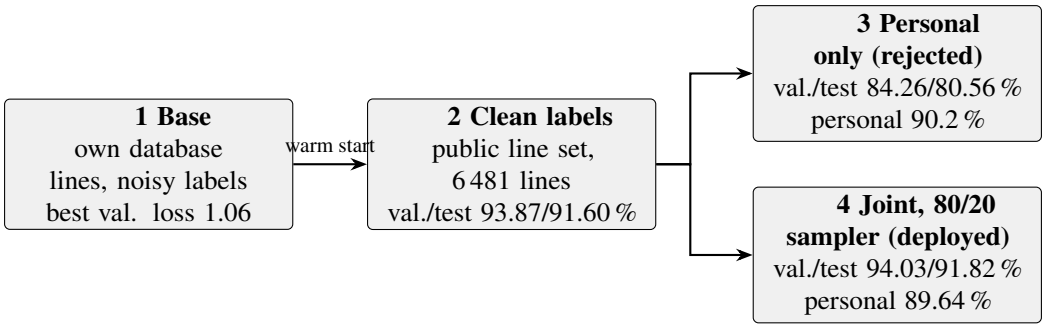


Figure 32. Staged training recipe of the text recogniser with the character accuracies (public validation and test sets, and personal validation; see Table 40 for the caveats).

Appendix: component labelling, skew estimation and blur construction

This part of the appendix contains the full text and equations of the connected-component, skew, Gaussian-blur, batch-normalisation and staged-training passages that are summarised in Sections 3.3 and 3.4.

7.0.0 Stage 7: connected components and the glyph-height scale

A *connected component* is a maximal set of ink pixels that touch each other. Components are found with a run-length two-pass algorithm with union-find, a design that follows the classical sequential labelling idea [25] but works on runs instead of single pixels. Each image row is first encoded as maximal runs $[s, e)$ of ink pixels. For every pair of adjacent rows the runs are swept with two pointers, and run i of the upper row and run j of the lower row are merged when they touch; for 8-connectivity (diagonal neighbours count) the condition (Equation 85) is

$$s_i < e_j + 1 \quad \wedge \quad s_j < e_i + 1, \tag{85}$$

and for 4-connectivity the +1 terms are dropped, where s and e are the start and end (exclusive) columns of the runs. The pointer whose run ends first is advanced. The merging uses a union-find structure with path halving [41], roots are mapped to compact labels $1 \dots n$, and the labels are painted back run by run. The cost is linear in the number of runs times an almost constant factor, which is far cheaper than a pixel-wise method for thin handwriting strokes. For every component the area, bounding box and centroid are computed by sorting the labelled pixels by label.

The *glyph height* h_{t} is estimated from the components themselves, as given in Equation 86. Heights of components with area at least 30 pixels and width below 12 times the height (which excludes rules and long underlines) are collected; h_{med} is their median, and

$$h_{\mathrm{t}} = \mathrm{median}\big\{h_c : 0.3\,h_{\mathrm{med}} < h_c < 3\,h_{\mathrm{med}}\big\}, \tag{86}$$

where h_c is the height of component c (the default is 30 pixels if no component qualifies). Because the cleaning stages change the component set, h_{t} is computed three times: after the first binarisation (for the recovery stage), after faint filtering (for edge, column and rule removal), and finally on the cleaned set (for grouping and rendering).

7.0.0 Stage 5: skew estimation and rotation

If the text lines are tilted, row-based grouping and the recogniser both degrade. The tilt of the page is estimated by the classical projection-profile principle: when the lines are horizontal, the row-sum profile of the ink has sharp peaks (line bodies) and deep troughs (gaps between lines), and any tilt smears them (Figure 22). For a candidate angle θ the ink mask, reduced by a factor 4 (any ink in a 4×4 block), is rotated by nearest-neighbour sampling, the row profile $P_\theta(y) = \sum_x R_\theta(y,x)$ is formed, and the score is its variance (Equation 87)

$$J(\theta) = \mathrm{Var}_y\big[P_\theta(y)\big], \qquad \theta^* = \arg\max_\theta J(\theta), \tag{87}$$

where R_θ is the rotated reduced mask and Var_y the variance over rows. The search uses a coarse grid $\theta \in \{-r, -r+0.5, \ldots, r\}$ of 33 angles for $r = 8°$ followed by a fine grid $\theta^* \pm 0.5°$ in steps of $0.1°$, so the angular resolution is $0.1°$ and only 44 rotations of a mask of one sixteenth of the original area are needed. Rotation about the image centre $(c_x, c_y) = ((W-1)/2, (H-1)/2)$ uses inverse mapping with bilinear interpolation (nearest neighbour for masks):

$$\begin{pmatrix} x_s \\ y_s \end{pmatrix} = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix} \begin{pmatrix} x - c_x \\ y - c_y \end{pmatrix} + \begin{pmatrix} c_x \\ c_y \end{pmatrix}, \tag{88}$$

in which (x, y) is an output pixel and (x_s, y_s) the source position that is sampled; outside samples are filled with 255 for the colour page and 0 for masks. The page is rotated only if $|\theta^*| \geq 0.15°$; stages 3 and 4 (grey conversion, correction, binarisation, speckle removal) are then recomputed on the rotated image, and a second residual pass with range $\pm 3°$ follows.
A single global angle is estimated, so page curl and keystone are not corrected; the objective was preferred to a Hough transform because ruled lines and diagrams dominate edge directions. A sign mismatch between rotation and scoring that doubled the skew in an early version was corrected.

Gaussian blur from three box blurs. A Gaussian blur with standard deviation σ is approximated by three successive box means of equal radius r, because the repeated convolution of a uniform kernel converges to a Gaussian shape by the central-limit argument. A single box of full width $w = 2r + 1$ has the variance $(w^2 - 1)/12$, and three independent passes add their variances to

$$\sigma^2_{\mathrm{real}} = \frac{w^2 - 1}{4}, \qquad w = 2r + 1, \qquad r = \max\left(1,\, \mathrm{round}\left(\tfrac{1}{2}\sqrt{\tfrac{4\sigma^2}{3} + 1}\right)\right), \tag{89}$$

where the radius r is the one chosen by the implementation for a requested nominal scale σ. An evaluation of Equation 89 shows that the box width produced by this rounding is approximately $\sqrt{4\sigma^2/3 + 1} + 1$, so that $\sigma_{\mathrm{real}} \approx \sigma/\sqrt{3} \approx 0.58\,\sigma$. For example, a nominal scale $\sigma = 60$ (the illumination blur for a page 1800 pixels wide) gives $r = 35$, $w = 71$ and $\sigma_{\mathrm{real}} = \sqrt{(71^2 - 1)/4} \approx 35.5$, and a nominal scale of 9.5 gives $r = 6$ and $\sigma_{\mathrm{real}} \approx 6.5$. The nominal σ in this report is therefore a *blur scale parameter* and not the realised standard deviation, and all constants of the block were tuned with the implementation as it stands.

Staged recipe. Training was carried out in four stages (Table 16, Figure 32), and each stage is motivated by a limitation that was found in the previous one.

1. *Base stage.* The published recipe was reproduced on lines that were segmented from database pages by the student's own tools. The data turned out to be small and noisy because of the labelling defect and segmentation errors, and the best validation loss was 1.06.

2. *Clean-label stage.* The network was warm-started from the base stage and trained on the public line-level IAM set, whose human-checked labels remove all segmentation and labelling noise.

3. *Personal fine-tuning (rejected).* Fine-tuning on the personal lines alone raised the accuracy on the personal validation lines but reduced the accuracy on the general data by about ten points (catastrophic forgetting, Table 17); freezing the convolutional layers reduced the forgetting only slightly.

4. *Joint stage (deployed).* Training continues from the clean-label stage on both data sets at once, with a weighted sampler: each public line receives the weight $w_T = (1 - f)/n_T$

and each personal line $w_P = f/n_P$, where $n_T = 6481$, $n_P \approx 280$ and $f = 0.20$, and n_T indices are drawn per epoch with replacement with probability proportional to the weights (Equation 90),

$$P(i) = \frac{w_i}{\sum_j w_j}. \tag{90}$$

In expectation 20 % of every epoch (≈ 1296 draws) is taken from the personal set, so that each personal line is seen about 4.6 times per epoch with fresh random augmentation, and 80 % from the public set. The checkpoint of the epoch with the largest $\min(\text{accuracy}_{\text{public}}, \text{accuracy}_{\text{personal}})$ is kept, so that neither domain is sacrificed.

A sweep over learning rate, LSTM width, depth, convolution width, dropout and optimiser (14 configurations of 50 epochs) was programmed, but no result file was kept, so the deployed architecture is the published one and not an optimum found by the student. Training was done offline on a general-purpose computer; the training log of the joint stage was not kept, so epoch-by-epoch curves cannot be shown, and the stored joint checkpoint contains weights only.

Batch normalisation. Deep networks train poorly if the distribution of each layer's inputs keeps changing, so every convolution is followed by batch normalisation [8]. During training, for each channel the mean μ_B and variance σ_B^2 are computed over the mini-batch and all spatial positions, and the output (Equation 91) is

$$\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}}, \qquad y = \gamma \hat{x} + \beta, \qquad \varepsilon = 10^{-5}, \tag{91}$$

where γ and β are learned scale and shift parameters of the channel. Running averages of the mean and variance are kept during training, and at test time they replace μ_B and σ_B^2, which turns the layer into the fixed per-channel affine map $y = a\,x + b$ with

$$a = \frac{\gamma}{\sqrt{\sigma_{\text{run}}^2 + \varepsilon}}, \qquad b = \beta - a\,\mu_{\text{run}}, \tag{92}$$

where μ_{run} and σ_{run}^2 are the running statistics; this is exactly the form that the inference implementation uses.

Appendix: label generation for the database pages

The labels of the database pages were produced by reading the printed prompt of each form and aligning its word list with the detected ink word runs by dynamic programming; the alignment cost for a run of measured width a and an expected width e is $c(a,e) = ((a-e)/(e+1))^2$, with a penalty of 0.5 for the merging or splitting of words.

Style-profile schema and per-writer values

This part of the appendix supports Section 3.6. Table 25 lists every key of a style-profile file with its meaning and an example value from the profile of writer 153, and Table 26 lists the scalar values of all ten stored profiles.

Key	Type	Meaning	Example (153)
`authorId`	string	Writer identifier.	`"153"`
`slantDeg`	float	Median slant of the lines, forward lean positive (Equation 38).	30.0
`xHeightPx`	float	Median pixel x-height of the kept lines.	33.0
`strokeWidthXh`	float	Ink area divided by skeleton length, in x-heights.	0.189
`ascender`, `descender`	float	Typical reach of tall and descending lower-case glyphs, in x-heights (defaults 1.7 and −0.6 if no glyph qualifies).	1.59, −1.11
`wordSpaceXh`	float	Word gap measured on the real ink, in x-heights.	1.59
`connectedness`	float	Fraction of adjacent in-word letter pairs that the pen joins (Equation 41).	0.97

Key	Type	Meaning	Example (153)
letterAdvance	dict	Median advance of each character, in x-heights (45 keys).	T: 1.438, e: 1.143
nLines	int	Number of lines that survived the alignment-confidence cut.	66
slantMeasRef, ascMeasRef, descMeasRef	float	Slant (degrees) and ascender and descender reach of the real full lines, measured with the estimators later used on synthesised renders; input of the style calibration (Equation 93).	30.0, 1.933, −1.468
inkFracRef, aspectRef, paperLevel, inkLevel, midFracRef	float	Ink fraction, width-to-height ratio and grey statistics of the real lines; stored but ignored while the constant pen width is used for rendering.	0.0620, 14.95, 246, 81, 0.038
glyphs	dict	Variant library: for each character a list of at most 13 variant dictionaries (38 characters and 280 variants for writer 153).	see below
idealGlyphs	dict	Per-stroke median prototype of a letter (21 characters), with the keys strokes, priorD, advance, lead, width, top, bot, entryY, exitY, nPool.	—
idealMedoid	dict	The real variant that is most typical of the writer (28 characters), same keys; this is the anchor used by the synthesiser.	—
legibilityLambda	float	Legibility parameter λ of Equations 42 and 44; added after the build.	0.45
Keys of one variant in glyphs[c][i]			
char	string	The character.	"e"
strokes	list	Strokes, each a list of $[x, y]$ points in x-height units, baseline $y = 0$, upright.	2 and 38 points
width, advance, lead	float	Ink width, cell width and gap before the ink, in x-heights.	0.878, 1.061, 0.0
top, bot	float	Highest and lowest point.	0.879, 0.061
entryX, entryY, exitX, exitY	float	Entry and exit points of the pen for connectors.	−0.204, 0.333, 0.767, 0.182
connL, connR	bool	The pen was joined to the left or right neighbour.	true, true
priorD	float	Cosine distance to the cross-writer letter prior (Equation 40).	0.1127
denoised	bool	Present and true if the variant is the denoised prototype.	—

Table 25. Keys of a style-profile file (JavaScript Object Notation).

Constants of the synthesiser

Table 27 lists the constants of Section 3.7 that are not given in the main text.

G-code generator configuration

Table 28 lists the machine configuration of the G-code generator of Section 3.8. The motor, driver and transmission rows are assumptions made in the program comments; the calibrated steps per millimetre replace the derived values in the interpreter.

Routes of the server application

Table 29 lists the routes of the server application of Section 3.10.

Supplementary figures and table for Sections 3.6 to 3.8

This part of the appendix collects the figures and the cost table that support Sections 3.6 to 3.8. Figure 33 shows the forced-alignment lattice, Figure 34 the Douglas–Peucker simplification, Figure 35 a part of a stored library, Figure 36 the placeholder for the builder stages on real lines, Figure 37 the placement of glyphs, Figure 38 two synthesised lines, Figure 39 the coordinate frames and Table 30 the cost of the search.

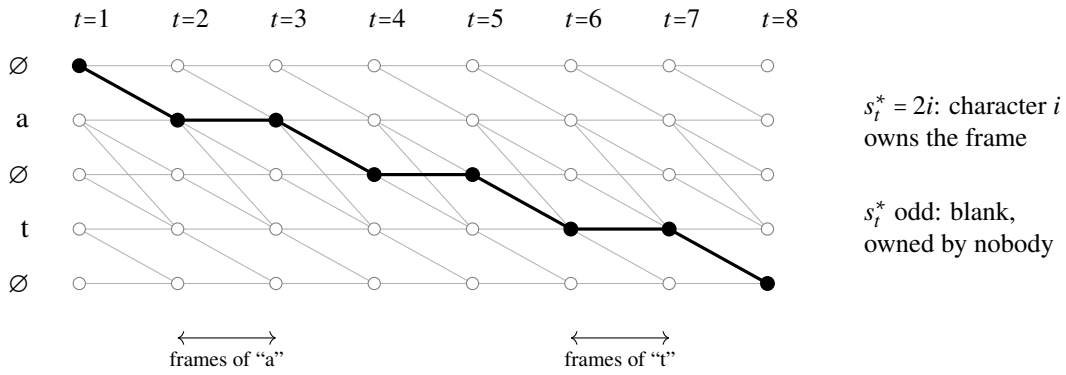


Figure 33. Forced-alignment lattice for the transcript “at” with $T = 8$ steps (cf. Figure 30). The thick path is the Viterbi path of Equation 36; only the frames on the row of a letter belong to that letter, so the spans are narrow.

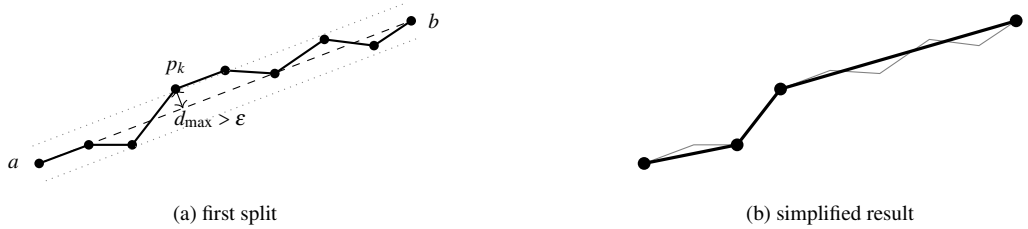


Figure 34. Douglas–Peucker simplification (schematic, tolerance exaggerated). (a) The vertex p_k with the largest distance $d_{\max}$ from the chord ab lies outside the tolerance band (dotted) and splits the chain. (b) After recursion on both halves four of the original nine vertices remain.

Hardware information and timing measurements still to be supplied

This part of the appendix supports Sections 3.9 and 3.10. Table 31 lists the hardware items with the evidence in the project files, Figure 40 is the placeholder for the photographs, and Table 32 lists the stages of the timing chains.

Design alternatives

Tables 33 and 34 list the design alternatives of Sections 3.6 and 3.7 with the reason for the chosen design.

Further figures and tables for Sections 3.8 to 3.10

This part of the appendix supports Sections 3.8 to 3.10. Figure 41 shows the start of a generated G-code program, Figure 42 the velocity profiles of Equation 49, Table 35 the pin assignment, Figure 43 the switch input circuit and Table 36 the chains of operations of the two online modes.

Style calibration of the synthesiser

Section 3.7.2 describes the calibration; its three fixed-point iterations are, with “ref” the stored real-line value and “got” the value measured on the probe,

$$\Delta_\sigma \leftarrow \Delta_\sigma + \tan\theta_{\mathrm{ref}} - \tan\theta_{\mathrm{got}}, \quad K_A \leftarrow \mathrm{clip}\Big(K_A \frac{A_{\mathrm{ref}} - 1}{A_{\mathrm{got}} - 1}, 0.5, 1.6\Big), \quad K_D \leftarrow \mathrm{clip}\Big(K_D \frac{D_{\mathrm{ref}}}{D_{\mathrm{got}}}, 0.5, 1.6\Big), \tag{93}$$

applied as $y \leftarrow 1 + (y-1)K_A$ for $y > 1$ and $y \leftarrow yK_D$ for $y < 0$ on the writer’s own glyphs; Δ_σ is added to the shear coefficient of Equation 42.

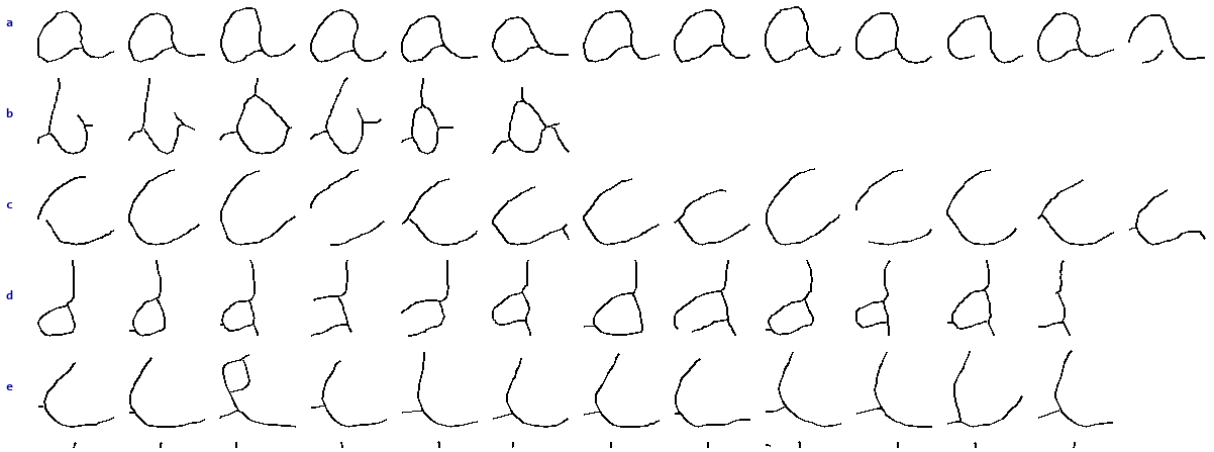


Figure 35. Stored variants of the letters a, b, c, d and e of writer 153 (a strongly joined hand) in the glyph grid of the web interface; each row is one character with up to 12 variants. The short tails at the sides are remains of ligatures.

FIGURE MUST STILL BE INSERTED BY ME

One real line of a database writer and one personal line: grey crop, ink mask with rules removed and core band, Zhang–Suen skeleton, traced chains in colours with the territory cuts as vertical lines, and one extracted glyph in the normalised frame. Source: the functions of the profile builder applied to a crop of `Data/Datasets/IAMpages10` and one of `Software/CNN/NOGIT/dylan`.

Figure 36. Stages of the style builder on real lines (placeholder).

Extended qualification-test protocols

This appendix gives the data sheets and definitions that support the qualification tests of Section 4.2.

7.0.0 Metric definitions and detailed steps (qualification tests 1 and 5)

Inter-rater agreement of the panel is quantified with Fleiss' kappa,

$$\kappa = \frac{\bar{P} - \bar{P}_e}{1 - \bar{P}_e}, \tag{94}$$

where $\bar{P}$ is the mean observed pairwise agreement and $\bar{P}_e$ the agreement expected by chance. Each feature is z-scored, $z_{wf} = (v_{wf} - \mu_f)/\sigma_f$, using the mean and standard deviation over the ten real writers, and the stylistic correlation is

$$\rho = \frac{\sum_i (a_i - \bar{a})(b_i - \bar{b})}{\sqrt{\sum_i (a_i - \bar{a})^2}\sqrt{\sum_i (b_i - \bar{b})^2}}, \tag{95}$$

where a_i and b_i are the real and synthesised z-scored entries pooled over 10 writers and 7 features (70 pairs); a per-writer value (7 pairs) and a nearest-centroid retrieval in z-space with its 10×10 confusion matrix are also computed. For the overlay test (Figure 44) the skeleton point sets A and B are compared with

$$d_{\mathrm{ch}}(A,B) = \frac{1}{2}\left[\frac{1}{|A|}\sum_{a \in A}\min_{b \in B}\|a - b\| + \frac{1}{|B|}\sum_{b \in B}\min_{a \in A}\|a - b\|\right], \tag{96}$$

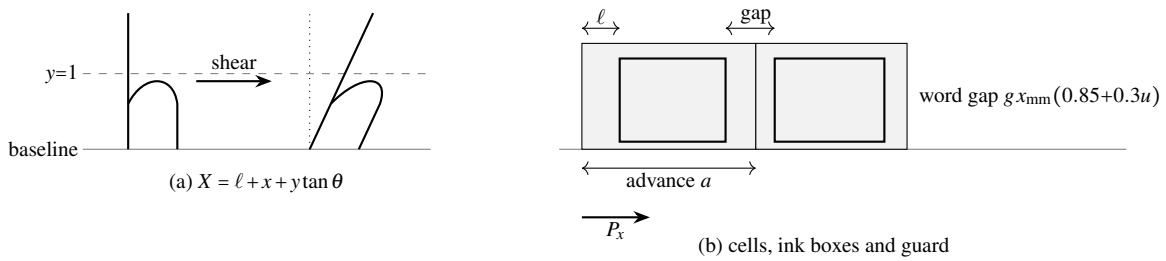


Figure 37. Glyph placement. (a) A stored upright glyph is sheared by the writer's slant (Equation 45). (b) Each glyph occupies a cell of its own advance a; the ink starts after the lead ℓ, and the collision guard keeps a minimum clear gap between consecutive ink boxes.

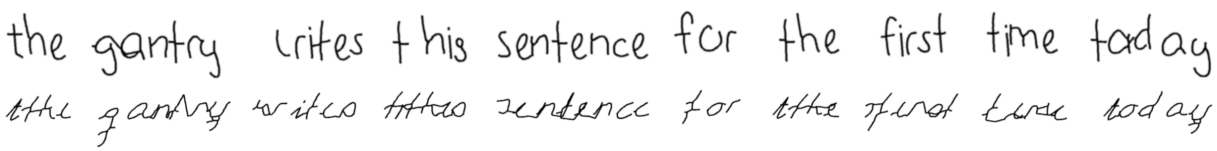


Figure 38. Synthesised line "The gantry writes this sentence for the first time today" for writer 150 (upper, printed hand; read back with three small errors) and writer 153 (lower, joined hand; read back as "the garoby writes tttas senkence for tthe furst tuse today"), one seed and candidate each, rendered from the trajectory of an end-to-end run; the N of that run was not recorded.

$$d_{\mathrm{MH}}(A,B) = \max\left[\frac{1}{|A|}\sum_{a\in A}\min_{b\in B}\|a-b\|,\ \frac{1}{|B|}\sum_{b\in B}\min_{a\in A}\|a-b\|\right], \tag{97}$$

$$\mathrm{AUC} = \Pr\left[d_{\mathrm{matched}} < d_{\mathrm{mismatched}}\right] + \tfrac{1}{2}\Pr\left[d_{\mathrm{matched}} = d_{\mathrm{mismatched}}\right], \tag{98}$$

where $\|\cdot\|$ is the Euclidean norm and distances are in units of the x-height x_{h}. Equation 96 is the mean of the two directed mean nearest-neighbour distances, Equation 97 their maximum (less sensitive to outliers than the classical Hausdorff distance), and Equation 98 is 0.5 for a distance without writer information and 1.0 for perfect separation. With 8 writers and 5 sentences there are 40 matched and $40 \times 9 = 360$ mismatched pairs.

For qualification test 1 the steps are: (1) fix the held-out pages and freeze the system; (2) produce the 150 renderings (direct and G-code replay) on the ODROID N2+ and store writer, sentence, seed and text; (3) read every rendering with the base recogniser and evaluate Equation 68 with the intervals; (4) run the panel with Table 37, computing transcription accuracy and κ; (5) run the feature extractor and compute ρ and the confusion matrix; (6) compute overlay distances, AUC and rank-1 retrieval and produce overlay figures for at least four sentences; (7) write a sentence on the gantry in a chosen style and count visibly matching characters (pass at 85%). Raters are 8 persons who are neither the student nor writers in the data set; the panel is blind and uses 30 synthesised lines (3 per writer) and 10 genuine control lines in random order, about 1350 scored characters per rater.

For qualification test 5 the absolute error and the measurement uncertainty are

$$e_i = \|\mathbf{p}_i - \mathbf{p}_i^{\mathrm{cmd}}\|, \tag{99}$$

$$u = \sqrt{u_{\mathrm{scan}}^2 + u_{\mathrm{ref}}^2 + u_{\mathrm{id}}^2}, \tag{100}$$

where $\mathbf{p}_i$ is the measured and $\mathbf{p}_i^{\mathrm{cmd}}$ the commanded centre of cross i (referenced to the machine origin 5 mm inside the minimum end-stops), u_{scan} the scanner scale error (from scanning the verified ruler), u_{ref} the registration of the sheet against the machine origin and u_{id} the error of

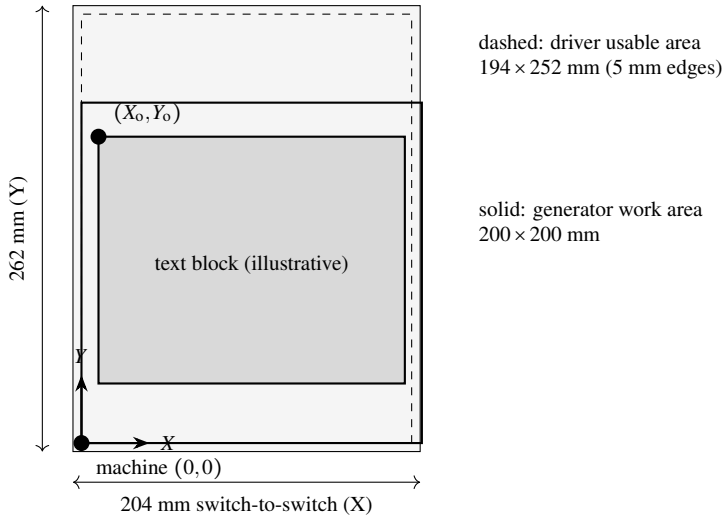


Figure 39. Coordinate frames drawn to scale. The machine frame has its origin 5 mm inside the minimum switches; the generator places the text block with its top-left corner at $(10, 180)$ mm inside a 200×200 mm work area, and the driver clamps all targets to its calibrated usable area of 194×252 mm. The extent of the text block is illustrative.

FIGURE MUST STILL BE INSERTED BY ME

Photographs of the assembled gantry (axes, end-stops, pen holder with toggle motor, paper holder) and of the scanning rig with camera; CAD renderings and a wiring diagram with component values. To be supplied by the student.

Figure 40. Photographs of the gantry and the scanning rig (placeholder).

```
; Handwriting-Robot G-code -- author yeukita
; steps/mm  X=80.000  Y=80.000 (3200 microsteps/rev)
G21   ; millimetres
G90   ; absolute
G0 F2400                      <- travel feed, emitted once
G0 X10.059 Y179.271           <- travel to the first stroke
M3   ; pen DOWN (one 90deg pulse of the one-way gear motor)
G4 P0.350   ; wait for the 90deg rotation to finish
G1 F900                       <- draw feed
G1 X10.084 Y178.920           <- one line per vertex (0.35 mm apart)
...
M5   ; pen UP (one 90deg pulse of the one-way gear motor)
G4 P0.350
...                           <- next stroke; at the end: park, M2
```

Figure 41. Start of a generated program (test word of writer yeukita: 14 strokes, 239 draw moves, vertices 0.348 mm apart on average); the arrows are annotations.

locating a cross centre (half a pixel); the expanded uncertainty is $2u$. The steps are: (1) calibrate and verify the direction pin on both axes and record the calibrated steps per millimetre; (2) fix

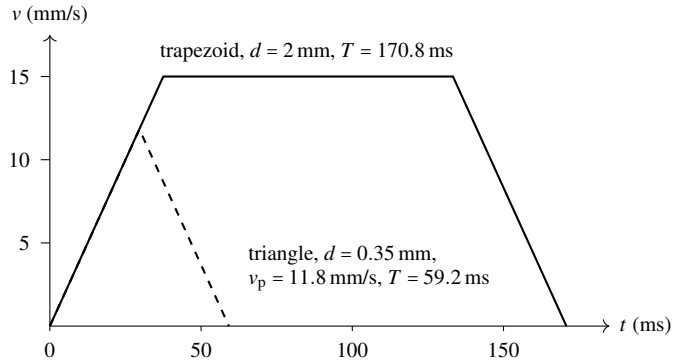


Figure 42. Velocity profiles of Equation 49 for $a = 400\,\text{mm/s}^2$ and $v = 15\,\text{mm/s}$: a 2 mm move is trapezoidal (solid); the usual 0.35 mm segment is triangular (dashed) and never reaches the cruise speed.

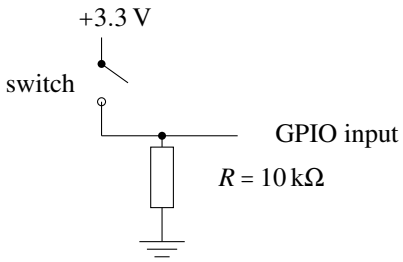


Figure 43. Switch input circuit as documented in the program comments; the closing current is $3.3\,\text{V}/10\,\text{k}\Omega = 0.33\,\text{mA}$. The origin of the 3.3 V (header pin) is inferred and [MUST STILL BE COMPLETED BY ME: confirm this item].

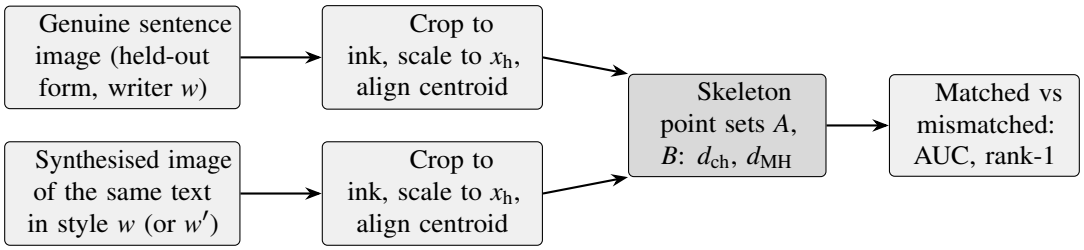


Figure 44. Overlay test of qualification test 1: both images are normalised in the same way and matched writer pairs are compared with mismatched pairs.

the sheet against the stops, draw the grid, scan it with the ruler; (3) locate the cross centres with sub-pixel image analysis and compute e_i; (4) repeat 5 times and write the same word twice on one sheet for the overlay; (5) estimate u. Repeatability is the radius of scatter of the 5 repeats of one cross about its centroid.

7.0.0 Human legibility panel (qualification test 1)

Table 37 is the worksheet that each of the 8 raters completes; it contains no information on whether a line is synthesised or genuine.

7.0.0 Style-feature definitions (qualification test 1)

The features of Table 38 are computed by one deterministic extractor with identical thresholds on every image, after cropping to the ink and binarising with the same rule.

7.0.0 Hardware overlay measurement sheet (qualification test 5)

Table 39 is the sheet on which the measurements of the grid test are recorded for each repetition.

Appendix: design-stage measurements of the text recogniser

Table 40 lists the accuracies of the deployed network that served as design evidence. They are *not* the final qualification results, which belong to Section 4, and every figure carries the caveat of its data set (Table 15).

Three conclusions follow. First, joint training kept the general accuracy while adding the personal writers (the test accuracy rose by 0.22 points over the stage with clean labels only), so the weighted sampler is justified; fine-tuning on the personal lines alone was rejected because the general accuracy fell by about ten points. Second, the word error rate is about four times the character error rate, so roughly one word in four contains a character error and only about one line in six is entirely correct; greedy decoding without a lexicon leaves spelling errors uncorrected. Third, the gap between the operating figure of 98.61 % on correctly cut crops and the 61.21 % through the complete page pipeline is the largest discrepancy in this report, and it shows where the remaining end-to-end loss most probably lies. That comparison is not clean. The two figures used different recogniser versions and different page sets, and the pages used for the full-page measurement were later included in the fine-tuning data, so an unknown part of the 37-point difference is recognition improvement rather than segmentation or label-alignment error. What can be stated is that the crops in the operating figure were hand-checked to be correct whereas the full-page path has no such guarantee, so the difference bounds the combined effect of segmentation and label alignment from above and cannot be attributed to recognition, measured at 98.61 % given correct crops. Splitting the two causes requires re-running the full page pipeline with the current recogniser on pages never used for training, the protocol proposed in Section 5.

Appendix: design-stage measurements of the synthesiser

Table 41 lists the measurements that motivated the design (six novel sentences, two seeds, ten writers: 12 lines per writer, so one misjudged line moves a writer's rate by 8.3 percentage points). *Every figure after the search was introduced is the opinion of the very networks that*

selected the output: the search loads the same weight files as the measurement, so a selection effect is added to genuine quality, and λ, the anchor threshold and N were tuned on the same sentences. The values show that the pipeline can satisfy these two judges, not that human readers or an independent recogniser agree; the non-circular reference is the recogniser on *real* held-out handwriting of the same writers, 89.4 % character and 70.8 % word accuracy (Table 15; possible training overlap). Section 4 reports the final evaluation.
Four remarks follow. First, on the matching comparison of identical words the writers' own hands are read at 89.0 % and the synthesis of those words at 81.6 % character accuracy, so synthesis costs 7.4 percentage points. Second, the same comparison divides word accuracy by more than 1.5 (70.5 % against 46.7 %), locating the loss in spacing and word boundaries rather than letterforms. Third, at the final settings the two most joined writers stay well below the 85 % target: writers 151 and 552 reach 53 % and 56 % character accuracy through G-code, against about 84 % for the other eleven. This is the clearest limitation of the exemplar approach on strongly cursive hands. Fourth, the writer-identification figures on synthesised text (80.1 % in the official run and 89.7 % in the coverage run) come from a classifier that also selects the candidates, so a best-of-N search necessarily raises the score by which it is judged; they are reported as what the system computes at run time, not as an independent measure of style. Figure 15 shows the per-writer breakdown and Figure 38 of the appendix one candidate each for a print and a joined hand.

Appendix: design decisions, alternatives and trade-offs

The two tables below list every software and hardware design decision of the concept, the alternatives considered for each and the reason the preferred option was chosen. They support Section 2 and are reported here so that the main report stays within its page limit.

Appendix: supporting figures for qualification test 1

The two figures below support the style sub-test of qualification test 1. They are reported here rather than in Section 4 because the headline figures of that test are already given there, and these two add detail rather than a further result.

Appendix: supporting material for the text recogniser

This part of the appendix contains the supporting detail of Section 3.4 that the main argument of that section does not depend on: the architecture diagram, the symbol set, the canvas geometry, the parameter and operation counts, the implementation conventions of the CTC loss, the layer inventory of the from-scratch module, the protocol of the gradient check and the code of the convolution.

The symbol set and the label filter. The 74 symbols of Equation 20 are the 26 lower-case and 26 upper-case letters, the ten digits and twelve symbols (space and . , ; : ’ " ! ? ( ) -). Characters outside this set are removed from every label, lines that become empty are dropped, and lines of more than 120 characters, which indicate merged lines, are dropped.

Canvas geometry and the rejected plain stretch. Typical text lines are about 14:1 in width to height and the canvas is 10:1, so for most lines the width binds in Equation 13 and the line occupies fewer than 64 rows. A plain stretch to the canvas was used initially but distorted letter proportions: in one example it applied a horizontal scale of 0.48 against a vertical scale of 0.35, which is why the uniform scale of Equation 13 replaced it.

Grouped form of the LSTM pre-activations. In the implementation of Equation 18 the four pre-activations come from one product, $[i; f; g; o] = W_{ih}x_t + b_{ih} + W_{hh}h_{t-1} + b_{hh}$ with $W_{ih} \in \mathbb{R}^{1024\times I}$ and $W_{hh} \in \mathbb{R}^{1024\times 256}$.

Parameter count and operation count. The counts of Table 14 follow from the layer shapes. A convolution with batch normalisation has $C_{\text{out}}(9C_{\text{in}} + 1) + 2C_{\text{out}}$ trainable parameters, and one

LSTM direction has

$$N_{\mathrm{LSTM}} = 4n(I+n) + 2\cdot 4n, \tag{101}$$

with I the input size, $n = 256$ the hidden size and the last term the two bias vectors; the first layer gives $N = 4\cdot 256\cdot 384 + 2048 = 395\,264$ and the second 788 480 per direction. The LSTM layers hold 70.5 % of the 3 358 763 parameters, stage 3 24.2 %, stage 2 3.9 %, the output layer 1.1 % and stage 1 0.3 %. The number of multiply–accumulate operations (MACs) for one line is

$$\begin{aligned} N_{\mathrm{MAC}} = &\sum_{\ell} 9 C_{\mathrm{in},\ell} C_{\mathrm{out},\ell} H_\ell W_\ell \\ &+ 2T\left[4n(128+n) + 4n(512+n)\right] + T\cdot 512\cdot 75 \approx 4.17\times 10^9, \end{aligned} \tag{102}$$

with $H_\ell W_\ell$ the feature-map size at convolutional layer ℓ and $T = 160$ the number of time steps: 3.79×10^9 for the convolutions, 0.38×10^9 for the BiLSTM and 6×10^6 for the output layer. Convolutions therefore dominate the arithmetic (91 %), while the LSTM is inherently sequential, with $160\times 2\times 2 = 640$ small matrix–vector products.

CTC implementation conventions and the label cap. Two conventions of the implementation of Equations 21 to 24 matter for the losses quoted in Section 3.4: each line's loss is divided by its target length, so reported losses are per-character negative log-likelihoods, and lines with no valid path are given zero loss. Every line of a batch uses the full $T = 160$ steps including the padding. A label cap of 120 characters, $\lfloor 160\cdot 0.75 \rfloor$, keeps every label far below T, which CTC requires: it needs $T \geq U$ plus the number of adjacent repeated letters of the target.

Learning-rate schedule of the recogniser. A plateau schedule halves the learning rate: after a number of validation checks (the patience) without improvement of the validation loss, $\eta \leftarrow \max(0.5\,\eta, \eta_{\min})$, with η the learning rate of Equation 26 and $\eta_{\min}$ a floor below which it is not reduced.

Layers implemented from first principles. The module of Section 3.4.7 implements, from the defining equations and with no machine-learning library, the forward and the backward pass of the convolution of Equation 15, batch normalisation in its training form (batch statistics) and its folded inference form (Equation 92), the rectified linear unit, max-pooling and the two adaptive pools of Equations 17 and 30, a fully connected (linear) layer, dropout, the multilayer bidirectional LSTM of Equations 18 and 19, and the CTC forward–backward recursion of Equations 22 to 24. It also implements the RMSprop update of Equation 26, the AdamW update of the writer classifiers (Equation 84), the plateau and cosine learning-rate schedules and the gradient-norm clip of Equation 27, so no part of training depends on an external library. One shared training program calls it to build and train either the recogniser or a writer classifier, since the two share the convolutional backbone (Section 3.5).

Protocol of the gradient check. Each hand-derived backward pass was compared with the numerical estimate

$$\frac{\partial \mathcal{L}}{\partial \theta_i} \approx \frac{\mathcal{L}(\theta + \varepsilon e_i) - \mathcal{L}(\theta - \varepsilon e_i)}{2\varepsilon}, \qquad \varepsilon = 10^{-5}, \tag{103}$$

where e_i is the unit vector of parameter i and $\mathcal{L}$ a scalar loss on a small random input. A configuration is one (layer variant, tensor) pair; both the input gradient and the gradient with respect to every learnable parameter of that layer are checked, so one layer gives several configurations. A configuration is accepted when

$$\frac{\max_i |g_i^{\mathrm{a}} - g_i^{\mathrm{n}}|}{\max\left(1,\ \max_i |g_i^{\mathrm{a}}| + \max_i |g_i^{\mathrm{n}}|\right)} < 10^{-6}, \tag{104}$$

with g^{a} the analytic and g^{n} the numerical gradient. The scalar loss used in Equation 103 is $\mathcal{L} = \sum_j o_j R_j$ for a fixed random R, so $\partial\mathcal{L}/\partial o = R$ exactly and no second approximation enters

the comparison. The check is a standalone program, `GradCheckNumpyNN.py`, part of the code submission; its full output is saved beside it. All arithmetic is in double precision, $\varepsilon = 10^{-5}$, and up to 40 randomly chosen elements of each tensor are probed (2044 elements in total). On the stricter per-element measure $|g^{\mathrm{a}} - g^{\mathrm{n}}|/(|g^{\mathrm{a}}| + |g^{\mathrm{n}}|)$ the worst case over all 2044 elements is 2.0×10^{-7}, on a recurrent weight of the two-layer bidirectional LSTM whose gradient is itself near zero, where the central difference has no significant digits left. Max-pooling and the rectified linear unit were checked with inputs from a permuted linear spacing, so no element lies within ε of the kink and no perturbation can change which element is the maximum; dropout was checked with its mask fixed; and batch normalisation in its training form, the form whose backward pass is used.

Warm-starting and resumption of a training run. Rather than train to convergence from a random initial condition at the speed reported in Section 3.4, the training program can load every tensor of an existing checkpoint whose shape matches (a warm start), so a further run only continues improving an already-competent network. It also writes the current epoch number and the best validation score into the checkpoint after every epoch, so a run interrupted by a crash or a time limit resumes from the last completed epoch.

Appendix: supporting material for writer identification

This part of the appendix contains the architecture diagram of the two writer classifiers, the threshold constants of the stroke-normalisation algorithm, the magnitudes of the augmentation and the deployment details of Section 3.5.

Thresholds of the stroke-normalisation steps. In the binarisation step of Section 3.5.3 the crop is thresholded with the global Otsu threshold (Equation 6), ink being the pixels below it. Ruled lines and underlines are removed with the run-length idea of Section 3.3: with h the crop height, an ink pixel is a rule pixel if

$$H_{\mathrm{run}} \geq \max(24, \lfloor 1.1h \rfloor) \quad \wedge \quad V_{\mathrm{run}} \leq \max(3, \lfloor 0.12h \rfloor), \tag{105}$$

where H_{run} and V_{run} are the horizontal and vertical ink-run lengths through the pixel; rule pixels crossed by a vertical stroke have a large V_{run} and are kept. Hairline breaks are then closed with a square structuring element of side $k_c = \mathrm{clip}(\mathrm{round}(0.045h), 2, 4)$, and 8-connected components of fewer than 6 pixels are removed. If no ink remains the sample is dropped. In the re-inking step the skeleton is dilated with the $k \times k$ square of Equation 34, computed from a box sum, a pixel being set if the sum exceeds 0.5:

$$\mathrm{ink}_{\mathrm{norm}} = \mathrm{Dilate}_{k \times k}(\mathrm{skeleton}). \tag{106}$$

Because a square kernel is used, a stroke has the width k along the axes and about $k\sqrt{2}$ along diagonals; this is the same for every writer.

Magnitude of the augmentation against the measured writer slants. A shear of ±0.6 is about ±31°, against measured writer slants between about −7.5° and +42°.

Deployment code path and checkpoint format of the classifiers. Unlike the earlier design, in which an inference-only program had to be checked against a separate training program, the training and the deployed code of the classifiers are the same Python functions: the evaluation pass the training script runs after every epoch to measure validation accuracy (eval-mode batch normalisation, no dropout) and the pass the web server runs on a photographed page call identical backbone and head functions. Both training programs share the recogniser's checkpoint format: every tensor goes into a single plain array archive (no object serialisation of any kind), keyed by the same dotted names as the layer functions (for example `author_head.0.weight`); because the archive cannot store a Python dictionary, the thirteen writer names are written as one extra array of strings and rebuilt into a name-to-index mapping when opened.

Appendix: supporting material for the style-profile builder

This part of the appendix holds detail relocated from Section 3.6: the book-keeping of the forced alignment with the frame-to-pixel conversion and the rule that drops weak lines, the binarisation and core-band recipe, the skeleton-tracing thresholds, the Douglas–Peucker distance and recursion, the resampling and smoothing constants of kept variants, and the quality-gate cascade with its thresholds. The design itself, its justifications, its measured outcomes and its stated limitations are in Section 3.6, which refers to each paragraph below by name; this part records only supporting detail.

Alignment book-keeping, frame-to-pixel conversion and line selection. The Viterbi path of Equation 36 ends on the last character or the final blank ($s_T^* = S$ if $\delta_T(S) \geq \delta_T(S-1)$, else $S-1$) and is traced backwards through the stored back-pointers (Figure 33; cost $O(TS)$). Because the recogniser input is left-aligned with the uniform scale of Equation 13, the frame boundary a lies at the original pixel

$$x_{\mathrm{px}}(a) = \frac{640\,a}{T\,s}, \qquad s = \min\Big(\frac{64}{h}, \frac{640}{w}\Big), \tag{107}$$

for a crop of $h \times w$ pixels (about 6 pixels per frame for a 980-pixel line). The alignment confidence is the mean log-probability along the path,

$$\mathrm{conf} = \frac{1}{T}\sum_{t=1}^{T} \ell_t\big(y'_{s_t^*}\big), \tag{108}$$

and every line below the 25th percentile of the writer's lines is dropped (if at least eight exist), e.g. 22 of 88 lines for writer 153.

Binarisation and core-band measurement. Each line is binarised like the shape classifier's input (Otsu threshold, rule removal by Equation 105, closing, speck removal; Section 3.5.3); the baseline and x-height come from the core band of Equation 33 (median x_h 20 to 46 pixels over the ten writers).

Skeleton tracing, spur pruning and junction merging. Walks from ends and junctions follow degree-2 pixels of the 8-neighbourhood graph, and chains under 2.5 pixels with a free end are pruned as spurs. Chain ends within 1.5 pixels are merged repeatedly in order of the smallest continuation angle

$$\varphi = \arccos\big(-\mathbf{d}_i \cdot \mathbf{d}_j\big) < 75^\circ, \tag{109}$$

where $\mathbf{d}_i, \mathbf{d}_j$ are the outward unit directions of the two ends ($\varphi = 0$ is a straight continuation).

Douglas–Peucker distance and recursion. Of a chain with first and last vertex a, b, the vertex p_i farthest from the line through a and b has the distance

$$d_i = \frac{\big|(b_x - a_x)(p_{i,y} - a_y) - (b_y - a_y)(p_{i,x} - a_x)\big|}{\|b - a\| + 10^{-9}}, \tag{110}$$

and if $\max_i d_i > \varepsilon$ the chain is split there and both halves are simplified recursively, otherwise only a and b are kept (Figure 34). The tolerance used is $\varepsilon = 0.6$ pixels, as stated in Section 3.6 together with the implementation note on closed loops.

Resampling, smoothing and connector trimming. On a side confirmed connected the outward-running low connector is trimmed only if at least 18 % of the glyph width remains. Kept variants are resampled at 0.055 x-heights arc length and smoothed with two passes of the binomial filter

$$q_i \leftarrow \tfrac{1}{4}q_{i-1} + \tfrac{1}{2}q_i + \tfrac{1}{4}q_{i+1} \tag{111}$$

(end points fixed), because the pixel staircase (median turn angle 8.9°, 90th percentile 59.5°) would otherwise be drawn as hand tremor.

Quality-gate cascade and derived glyphs. The gates are applied per character in order to the raw instances of a writer (about 2 000 to 6 200 per writer, of which roughly 10 % survive): (1) *geometry* (size limits, at most six strokes, a class prior on height, no swallowed neighbour, a pen length of at least 1.05 x-heights); (2) *outliers* (width and height within fixed ratios of the 40th-percentile width and the median height); (3) *cross-writer prior*, the gate of Equation 40, which keeps the 70 % nearest and is stated with its justification in Section 3.6; (4) *consensus*: the nearest 75 % to the L1-medoid of the descriptors are kept, removing glyphs contaminated by a neighbour; (5) *typicality*: the best 12 by closeness to the reference width and height are kept. Two derived glyphs are then stored: the *medoid*, the real variant closest to all others, and a *denoised prototype*, the per-stroke median curve of the modal stroke-count group, prepended (so a library can hold 13 variants) only if clearly cleaner.

Appendix: supporting material for the synthesiser

This part of the appendix holds detail relocated from Section 3.7: the construction of the anchor glyph, the variant draw probabilities, the jitter model, the placement and line-breaking constants, the steps of the best-of-N search with its flow chart, the order-statistics argument for the gain in N, and the cost model with the measured timings. The design itself, the justifications of the design choices and the limitations of the block, including the circularity of the scoring, remain in Section 3.7, which refers to each paragraph below by name.

Construction of the anchor glyph. The anchor is the writer's own medoid if its priorD $\le$ 0.30, otherwise a hand-designed single-stroke skeleton (joined cursive for $c \ge 0.5$, upright print otherwise) re-proportioned to the writer's reach, $y \mapsto 1 + (y-1)\max(0.35, (A-1)/0.75)$ for $y > 1$ and $y \mapsto y\max(0.35, |D|/0.55)$ for $y < 0$.

Variant draw probabilities without the entry-height factor. Without the exponential factor of Equation 43 the probabilities of the five eligible ranks are 0.494, 0.206, 0.130, 0.095, 0.075.

Glyph jitter and baseline drift. Each glyph receives the jitter

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = R(\vartheta)\,\mathrm{diag}(s_x, s_y)\begin{pmatrix} x \\ y \end{pmatrix} + \begin{pmatrix} d_x \\ d_y \end{pmatrix}, \quad \begin{array}{ll} d_x \sim \mathcal{N}(0, (0.022J)^2), & d_y \sim \mathcal{N}(0, (0.0132J)^2), \\ \vartheta \sim \mathcal{N}(0, (2.2^\circ J)^2), & s_{x,y} \sim 1 + \mathcal{N}(0, (0.05J)^2), \end{array} \tag{112}$$

with the amplitude J of Section 3.7 and the translations in x-heights; the baseline drifts as a slow first-order walk whose constants are in Table 27.

Advance, word spacing and line breaking. A guard shifts glyph and pen right if the glyph would come closer than $0.05\,x_{\mathrm{mm}}$ to the previous ink. After a word the pen advances by $g\,x_{\mathrm{mm}}(0.85 + 0.3u)$, $u \sim U(0,1)$, and a word that would cross the line width (185 mm in the server) starts a new line at the pitch L. Every polyline is resampled at 0.35 mm arc length for even plotter motion.

Steps of the joint best-of-N search. The steps of Figure 50 are: (1) for $k = 0, \ldots, N-1$, synthesise a candidate with seed base $+ 977k$ and $J = 0.5$, render it at a constant pen width of 0.085 x-heights (18 pixels per millimetre), compute S (Equation 47), keep it if $S >$ best (initially -1), and stop early at best ≥ 0.99; (2) *repair* (at most three rounds, skipped if best ≥ 0.995): forced-align the intended text to the recogniser output on the best image (Equation 36) and score each non-space character i by its mean own-class log-probability over its frames,

$$m_i = \frac{1}{b_i - a_i} \sum_{t=a_i+1}^{b_i} \ell_t(y_i), \tag{113}$$

with $m_i = -10^9$ if the path never visits it, sorted ascending; (3) pin the two worst unpinned characters to the anchor, re-synthesise the *whole* line with seed base, render and score it; (4) accept only if the joint score strictly exceeds best; Section 3.7 gives the reason for the strict joint

test, and the implementation notes on the repair pass are stated there.

Order statistics of the best-of-N gain. If candidate scores were independent with distribution function F, then

$$\Pr\left[\max_k S_k \le \tau\right] = F(\tau)^N, \tag{114}$$

so the gain grows roughly logarithmically in N. Section 3.7 states why this is only approximate for the delivered search.

Values of N, cost of a candidate and measured timings. N is 6 as the function default, 14 for Table 41, 20 as the web default and 30 in the official evaluation script. A candidate costs one synthesis, one rendering, one recogniser pass and one classifier pass,

$$T \approx N\left(T_{\mathrm{syn}} + T_{\mathrm{ren}} + T_{\mathrm{text}} + T_{\mathrm{wid}}\right) + (\text{up to three repair candidates}), \tag{115}$$

and each network pass is about 4×10^9 multiply–accumulate operations (Equation 102, Table 19). The laptop timings per stage are in Table 30.

Appendix: supporting material for the gantry and motion control

This part of the appendix supports Section 3.9. The design equations, the component table and the two circuit figures remain in that section; what follows is the supporting detail that the section now points to. Table 35 gives the pin assignment, Table 31 the hardware information still to be supplied and Figure 40 the placeholder for the photographs.

Alternative mechanical arrangement. Putting the second motor on the moving bridge keeps the transmission simple, because each axis is one motor and one belt, but the lower axis then has to accelerate the upper motor's mass as well as the carriage. The alternative, a fixed second motor with a longer belt routed through the bridge, removes that mass but adds idlers and a belt path that changes length as the bridge moves. The simpler arrangement was chosen because the writing speed turned out to be limited by the search in Section 3.7 and by the pen toggle, not by how fast the axes can be accelerated.

Calibration procedure, fault conditions and duration. Figure 51 is the flow chart of the direction-agnostic calibration of Section 3.9.9. An already triggered switch, an unexpected switch of the other axis or more than 200 000 steps (2500 mm) without a trigger aborts the calibration and jobs are refused; X is calibrated before Y, the gantry parks at $(0, 0)$, and the two traverses alone need at least $(204 + 262)/7.8 \approx 60$ s (derived, not measured).

Clearing the latched emergency stop. On a trip the program retracts off every triggered switch, stopping early if another switch trips, and aborts the job (Section 3.9.10). The latched state clears only by an explicit command, which refuses while a switch is still triggered.

The pen-toggle routine. The routine behind `M5` (up) and `M3` (down) refuses to run if an end-stop is tripped, returns at once if the switch already shows the target (a toggle motor must not run when no change is needed, or the state inverts), and otherwise runs the motor, polling the switch every 2 ms until it shows the target, an end-stop trips or 3 s elapse (for "up" the motor runs 40 ms longer so that the cam seats); if the switch never confirms, a warning is printed and the job continues.

What the motion program does not model. The program does not model acceleration, torque margin or stall speed, belt stretch and backlash, the timing between DIR and the first STEP pulse and the minimum STEP widths of the driver, or pen tilt and compliance; missed steps cannot be detected, since there is no feedback.

Appendix: supporting material for the software architecture

This part of the appendix supports Section 3.10. Table 29 lists the routes of the server, Table 36 the chain of operations of each mode and Table 32 the stages of the timing chains with the

measurements still missing; the two timing equations and the architecture figure remain in Section 3.10.

Start-up and caching. A background thread prepares each writer's glyph-grid image and two sample crops (slow on the first run, stored afterwards); the two requests that need them answer "still preparing" until it finishes, so other pages are not blocked.

Job storage and determinism. Photographs and jobs (the G-code and its preview) are stored under random identifiers, and every identifier received from a browser is reduced to alphanumeric characters before use as a file name, preventing path traversal. Because the seed is fixed by default, the same text, writer and N give the same trajectory.

Deployment. The server listens on all interfaces on port 5000 and is intended for the single-board computer; the pin interface is imported lazily, so the program also starts on a laptop, where a motion request is answered with "not connected".

Ergonomic choices. The live preview is a toggle used only while framing the page, the quality field (N, 1 to 60, default 20) trades synthesis time against quality, and the read-back of every preview allows checking before the pen moves.

Data set	Split rule	Caveat on the validation figure
Lines segmented by the student's own block from scanned database pages, with labels generated by alignment (first stage only)	The last labelled page of every writer is held out.	Same writers in training and hold-out, so the figure is writer-dependent; the labels are project-generated and noisy.
Public line-level IAM transcription set: 6 482 training, 976 validation, 2 915 test lines (6 481, 976 and 2 915 after label cleaning)	Official training, validation and test partitions.	The validation set drives learning-rate decay and checkpoint choice, so its accuracy is mildly optimistic; the test set is used for no decision. The partitions are believed, but not verified, to be writer-independent, and the warm start from the first stage may have seen some of the same forms.
13 photographed pages of the two personal writers (8 and 5 pages), 355 transcription lines of which one is a diagram marker. **[MUST STILL BE COMPLETED BY ME: This row describes the two-writer personal set. The project now has five personal writers, 30 photographed pages and 669 transcription lines, and Section 4 reports the recogniser's accuracy on all 669 of them while stating that those writers appear in its fine-tuning data. Confirm which personal set the deployed recogniser was actually fine-tuned on, and correct this row and the one below to match. I could not establish this from the saved checkpoints.]**	Per page, 15 % of the lines (at least one) are drawn at random with a fixed seed as validation; about 52 validation lines in total.	Validation lines come from the same pages, pen, paper, lighting and vocabulary as training lines; the figure measures "same writer, same notebook, new lines" and is optimistic. It also takes part in checkpoint selection.
Ten-author hold-out (eight database writers and two of the personal writers)	Used only as an evaluation set.	The held-out database lines may have been seen in the line-level training data, and their labels are project-generated, so the figure is not a writer-independent measure.

Table 15. Data sets of the text recogniser and their leakage assessment.

Stage	Data	η_0	Batch	Epochs	Clip	Schedule and selection
Base	own lines	10^{-3}	16	≤ 200	none	halve after 5 stalls; stop after 15; up to 3 restarts from the best weights; best validation loss
Clean labels	public set	10^{-3}	16	≤ 200	none	as above, up to 8 restarts; best public validation loss
Personal	personal lines	3×10^{-4}	8	60	5	halve after 5 stalls; best personal validation loss
Joint	both, $f = 0.20$	5×10^{-4}	16	30	5	halve after 4 stalls on the sum of both validation losses; best min of the two accuracies

Table 16. Hyper-parameters of the four training stages. All stages use RMSprop (Equation 26) with weight decay 10^{-5}; the minimum learning rate was 10^{-5} or 10^{-6}.

Training	Public val./test (%)	Personal val. (%)	Remark
Clean-label stage only	93.87 / 91.60	–	before personal data
Personal only, all layers	84.26 / 80.56	90.2	forgetting
Personal only, frozen convolutions	85.87 / 82.62	88.5	forgetting
Joint (deployed)	94.03 / 91.82	89.64	both domains kept

Table 17. The catastrophic-forgetting experiment: character accuracies of the three ways of adding personal data.

Alternative	For	Against
Character segmentation and a per-character classifier (small convolutional network)	Simple network; the proposal's plan.	Needs a segmenter that works on joined letters; classification errors cannot be repaired by context; needs character-level labels that the available data lack.
Line-level CNN-BiLSTM-CTC (chosen)	No segmentation below the line; alignment-free training from line transcriptions; context from both sides; output is a string; 3.36 million parameters can be run by a from-scratch implementation.	Needs line segmentation; greedy decoding leaves spelling errors; CTC assumes independent steps.
Lexicon or language-model decoding	Repairs near-miss words; gives the lowest error in the published design.	Needs a corpus and a decoder; the technical vocabulary and symbols of the pages are largely out of vocabulary. Future work.
Deeper backbones with skip connections	Easier training of very deep networks.	The 12-layer network with batch normalisation trained well; larger models cost memory and time on the single-board computer.
Attention-based encoder–decoder or transformer	Strong accuracy on large corpora.	Far more parameters, large pre-training data, slow autoregressive decoding that can skip or invent text, and hard to implement from scratch for the board.
Word-level recognition	Uses the lexicon directly.	Needs word segmentation and a closed vocabulary; poor for names, numbers and symbols.

Table 18. Alternatives to the line-level recogniser.

Block	Output shape	Trainable parameters
Stage 1 (2 conv units, 32 channels)	$32 \times 64 \times 640$	9 696
Max-pool 2×2	$32 \times 32 \times 320$	0
Stage 2 (4 conv units, 64 channels)	$64 \times 32 \times 320$	129 792
Max-pool 2×2	$64 \times 16 \times 160$	0
Stage 3 (6 conv units, 128 channels)	$128 \times 16 \times 160$	813 312
Height max-pool and width mean-pool	128	0
FC 128→96, ReLU, dropout	96	12 384
FC 96→10	10	970
Total (backbone 952 800, head 13 354)		**966 154**

Table 19. Layer shapes and parameters of the writer classifiers. Batch normalisation adds 2 176 non-trainable running statistics (968 330 stored numbers in total), and a forward pass needs about 3.79×10^9 multiply–accumulate operations.

Writer	Training lines	Validation lines	Validation lines are
150, 151, 152, 153	82, 78, 79, 88	9, 8, 8, 9	the held-out page
384, 551, 552, 588	72, 78, 63, 78	11, 8, 10, 8	the held-out page
Personal writer 1	118	20	random lines of training pages
Personal writer 2	184	32	random lines of training pages
Total	**920**	**123**	

Table 20. Lines of the shape classifier. The ink classifier uses the same split but its counts may differ slightly, because no lines are lost to the stroke normalisation.

	Ink classifier	Shape classifier
Optimiser, weight decay	AdamW, 10^{-4}	AdamW, 10^{-4}
Initial learning rate	10^{-3}	3×10^{-3}
Schedule	cosine over the epochs	cosine over 500 epochs
Epochs	20 (default)	500
Batch	16 images	64 feature vectors
Gradient clipping	global norm 5	none
Trainable parameters	966 154	13 354
Checkpoint rule	best validation accuracy	best validation accuracy

Table 21. Training settings of the two writer classifiers. The best epoch and the number of epochs that were actually run for the stored weights were not recorded.

Alternative	For	Against
Hand-crafted features (slant, x-height, spacing, connectedness) with nearest-profile classification	Interpretable; the style-profile builder already measures these features.	Needs feature selection; weak when two hands are similar in coarse statistics (writer 150 and a personal writer); no classification accuracy of such a method was measured.
Contour-direction and allograph statistics [27]	Text-independent and strong for large writer sets.	Hand-built features; less direct as a judge of constant-width output.
Siamese or metric learning	Open-set: new writers without retraining.	Needs pair mining and threshold calibration; the project needs only a closed set of ten.
Deep fragment or patch networks [28]	Scale to hundreds of writers.	Only ten writers and about 1 000 lines are available.
One joint network with a text head and a writer head	A single backbone.	Tried first: writer accuracy reached 98.9 to 100 % by epoch 20 while the text accuracy stalled at 28 to 29 %, so separate single-task networks were adopted.
Ink classifier with random pen-width augmentation	No second network.	Reduces but does not remove the thickness cue; the normalisation removes it by construction and matches the transform of the plotter. No ablation was run.

Table 22. Alternatives to the two-classifier design.

Proposal unit	Implemented block	Difference and reason
FU 1.1, 1.2	USB camera over an A4 page, read by the operating-system driver.	As planned; the “serial bitstream converter” is the standard camera interface.
FU 2.1	Page conditioning (Section 3.3).	Much larger than planned: lighting, skew, ruled paper, neighbouring pages and diagrams are handled.
FU 2.2	Line segmentation (Section 3.3).	Lines instead of characters, because cursive characters have no reliable boundaries.
FU 2.3	Text recogniser and writer classifier (Sections 3.4 and 3.5).	Two separate networks instead of one character classifier.
FU 2.4	Offline training programs.	Not on the single-board computer, because training needs far more memory and time than inference.
FU 2.5, 2.6	Style-profile builder and permanent profile files (Section 3.6).	Offline; stores letter paths per writer instead of one trajectory per recognised character.
FU 2.7	Synthesiser (Section 3.7).	Constructs new text from stored letter variants and adds a best-of-N search.
FU 2.8	G-code generator and interpreter with pulse generation (Sections 3.8 and 3.9).	Split into two stages with a standard interchange format, which allows the trajectory to be checked without the gantry.
FU 3.1	Stepper driver modules on 3.3 V pins.	No separate voltage-shifter block is documented. **[MUST STILL BE COMPLETED BY ME: confirm.]**
FU 3.2	Two stepper motors on one gantry.	As planned.
FU 3.3, 3.4	Pen holder and toggle pen-lift motor with position switch.	Pen lift is not driven through the stepper driver, as the proposal assumed, but by a one-direction gear motor with feedback switch.
FU 4	Power supply.	Not documented in the project files.
(new)	End-stop switches and web server.	Added for protection of the mechanism and to replace the proposed display and input interface.

Table 23. Relation of the implemented blocks to the functional units of the project proposal.

	Ink-based classifier	**Shape (stroke-normalised) classifier**
Input	Raw grey-scale line crop.	Line crop after stroke normalisation (constant pen width).
Used for	Real photographed pages in the classification mode.	Judging synthesised and G-code-rendered output, inside the best-of-N search of Section 3.7.
What was trained	All 966 154 parameters, started from the weights of the text recogniser backbone.	Only the 13 354 parameters of the head; the convolutional backbone is the frozen text-recogniser backbone.

Table 24. Roles of the two writer classifiers.

Writer	**Lines**	**Slant (°)**	x_h **(px)**	**Stroke width**	A	D	**Gap**	**Conn.**	λ	**Chars**	**Var.**
150	61	−4.5	46	0.097	1.58	−0.72	1.33	0.347	0.90	40	295
151	58	33.0	22	0.271	2.68	−1.09	1.59	0.981	1.00	44	285
152	59	19.5	46	0.103	1.48	−0.82	1.47	0.735	0.45	37	271
153	66	30.0	33	0.189	1.59	−1.11	1.59	0.970	0.45	38	280
384	54	4.5	43	0.107	1.95	−0.71	1.37	0.472	0.30	36	256
551	58	42.0	20	0.225	1.85	−0.68	2.68	0.583	1.00	38	278
552	47	−10.5	31	0.149	1.64	−0.56	1.53	0.621	0.00	38	279
588	58	0.0	29	0.161	1.72	−0.67	2.07	0.680	0.30	40	257
dylan	88	−7.5	27	0.130	1.68	−0.61	1.73	0.213	1.00	37	295
yeukita	138	0.0	27	0.145	1.46	−0.58	0.92	0.432	0.15	56	376

Table 26. Scalar values of the ten stored style profiles: kept lines, slant, pixel x-height, stroke width (x-heights), ascender A and descender D and word gap (x-heights), connectedness, legibility parameter λ, number of characters and number of variants. Writers 150 to 588 are database writers, the last two are the personal writers.

Constant	Value	Role
Eligible variants per letter	5	Highest-ranked variants that can be drawn (Equation 43).
Clean and garbage prior distances	0.12 and 0.55	Limits of Equation 44.
Maximum prior distance of an anchor medoid	0.30	Otherwise the generic skeleton is used.
Connected-hand threshold for the cursive anchor	0.5	A value above 1 disables the cursive anchor (an earlier configuration used 2.0); lowering it to 0.3 lowered the evaluation score and was reverted.
Slant limit of anchor glyphs	30° / 22°	Cursive / print skeleton.
Jitter share of anchor glyphs	0.15	Anchor glyphs wobble less.
Collision gap	0.05 x_{mm}	Minimum clear space between consecutive ink boxes.
Output resampling and connector step	0.35 mm	Arc-length step of every pen-down polyline.
Rendering for the judges	0.085 x, blur 0.30	Constant pen width in x-heights and Gaussian blur as a fraction of the stroke width; 18 pixels per millimetre; padding $0.18\,x_{\mathrm{mm}}$.
Search stopping scores	0.99 / 0.995	Early stop of the candidate loop / no repair above this score.
Repair rounds and pins per round	3 and 2	Joint-aware repair pass.
Seed step between candidates	977	Candidate k uses base $+\,977k$.
Calibration probe	seed 17, $J = 0$	Sentence “the quick brown fox jumps over lazy dogs and by half past eight it kept flying quietly”.

Perturbation (Equation 112)	**Standard deviation**	**At $J = 0.5$, $x_{\mathrm{mm}} = 4$ mm**
Translation in x / y	$0.022J$ / $0.0132J$ x-heights	0.044 mm / 0.026 mm
Rotation	$2.2^\circ J$	1.1°
Scale in x and y (independent)	$0.05J$	2.5 %
Baseline drift (AR(1), clipped)	$0.05J \cdot 0.35$ per step, ± 0.25	± 0.25 x-heights
Connector sag noise	$0.015\,x_{\mathrm{mm}}$	0.06 mm
Word-gap factor	uniform 0.85 to 1.15	—

Table 27. Constants of the synthesiser and the jitter model.

Field	Value	Meaning
Full steps per revolution	200	Assumed 1.8° stepper motor.
Microstepping	16	Assumed driver setting.
Travel per revolution X, Y	40.0 mm	Assumed belt of 2 mm pitch and a 20-tooth pulley.
Steps per millimetre (derived)	80.000	$200 \cdot 16/40$.
Work area	0 to 200 mm in X and Y	Limits of the clamp.
Origin and park position	(10, 180) mm	Top-left corner of the text block.
Draw feed	900 mm/min	Pen down (15 mm/s).
Travel feed	2400 mm/min	Pen up (40 mm/s); emitted once.
Acceleration	400 mm/s^2	Used only by the time model.
Minimum segment	0.12 mm	Shorter moves are collapsed.
Pen pulse and settling time	260 ms and 90 ms	Assumed time for a 90° rotation and dwell after a toggle.
Pen codes	`M5`, `M3`	Pen up, pen down.
Pen state at power-on	up	Assumed.

Table 28. Configuration of the G-code generator.

Method	Path	Behaviour
GET	`/api/authors`	List of the loaded writers with slant and connectedness.
POST	`/api/camera/capture`	Capture a still (from the open preview if there is one), store it, return an image identifier.
POST	`/api/camera/preview/start` and `/stop`	Open or release the camera for live framing.
GET	`/api/camera/preview/stream`	Live motion-JPEG stream; refused unless the preview is active.
GET	`/api/images/<id>`	Return a stored photograph.
POST	`/api/classify`	Photograph (file or identifier), optional expected text and writer; returns the recognised text, the character accuracy, the writer with its confidence, the per-writer mean probabilities and the number of lines; error 422 if no line is found.
POST	`/api/generate`	Text, writer and optional N (default 20); returns the job identifier, G-code and preview addresses, line count, pen pulses, read-back text and accuracy and the synthesis time.
GET	`/api/gantry/status`	Connected, calibrated and the calibration; tries to connect as a side effect.
POST	`/api/gantry/calibrate`	Run the calibration (blocking).
POST	`/api/write/<job>`	Run the stored G-code on the gantry (blocking); reports not connected, not calibrated, aborted by an end-stop, or completed.
POST	`/api/gcode/upload`	Store an uploaded G-code file as a job and return its preview rendered from the file; nothing is sent to the gantry.
GET	`/api/gcode/<job>` and `/api/images/job_<job>`	Raw G-code text and preview image of a job.
GET	`/api/model_info`	Model architectures, sizes and stored accuracy figures.
GET	`/api/glyphs/<writer>`	Glyph-grid image of a writer's library (status 202 while it is being prepared).
GET	`/api/samples/<writer>` and `/api/samples/<writer>/<file>`	Sample crops of real lines with their transcriptions.

Table 29. Routes of the server application (18 routes; the optional access key is checked before every route).

Quantity (56-character sentence)	Writer 150	Writer 153	Writer 551
Render size (pixels)	3585×225	5508×207	7876×277
Synthesis of one candidate (ms)	11	13	16
Rendering (ms)	20	27	56
Text recogniser (s)	0.50	0.49	0.74
Writer classifier incl. stroke normalisation (s)	1.03	1.65	3.74
Complete search, $N = 14$, with repair attempts (s)	23	37	137

Table 30. Indicative cost of the search on the development laptop (single profiling run, unknown background load, jitter 0.5); times on the single-board computer were not recorded.

Item	Evidence, and content to be supplied
Computer	ODROID N2+ with 3.3 V pins (Table 35).
Motors, drivers, transmission	Two motors with STEP/DIR driver modules; the program assumes 200 full steps per revolution, an A4988 at 1/16 microstepping and 40 mm per revolution (2 mm belt, 20-tooth pulley), and replaces the last by calibration. **[MUST STILL BE COMPLETED BY ME: motor model, holding torque, rated current; driver model, current-limit setting, microstepping jumpers, minimum STEP pulse width; belt or screw type, pulley teeth, guides; whether any level shifter lies between the 3.3 V pins and the drivers.]**
Travel	Measured switch-to-switch distances 204 mm (X) and 262 mm (Y). **[MUST STILL BE COMPLETED BY ME: the calibrated steps per millimetre printed for both axes.]**
End-stops	Four switches, active-high at 3.3 V, each with a pull-down of about 10 kΩ (Figure 43). **[MUST STILL BE COMPLETED BY ME: switch type and mounting.]**
Pen mechanism	One-direction gear motor whose cam toggles the pen every 90°, with a pen switch (high = up) (Section 3.9.10). **[MUST STILL BE COMPLETED BY ME: how the motor is switched (transistor, flyback diode), supply voltage, pen model and holder, lift height.]**
Power supply (FU 4)	**[MUST STILL BE COMPLETED BY ME: voltages and current ratings of the motor, computer and pen-motor supplies, grounding scheme, camera supply.]** The program comments state that the motors have no flyback diodes and share a ground return, which makes the switch inputs noisy (Section 3.9.10).
Camera (FU 1)	Generic USB video-class camera. **[MUST STILL BE COMPLETED BY ME: model, resolution, mounting height, field of view, lamp.]**

Table 31. Hardware items: evidence in the project files and information still to be supplied.

Stage	Known information and value to be supplied
Capture	Framing by the user; five discarded frames without preview. **[MUST STILL BE COMPLETED BY ME: time from pressing capture to the stored image.]**
Conditioning and segmentation	Whole page on the board. **[MUST STILL BE COMPLETED BY ME: time per page on the board.]**
Recognition and writer classification	Two passes of about 4×10^9 multiply–accumulate operations per line. **[MUST STILL BE COMPLETED BY ME: time per line on the board and total for the R4 test paragraph.]**
Synthesis	Returned in the server response; laptop values in Table 30. **[MUST STILL BE COMPLETED BY ME: time on the board for the N used in the demonstration.]**
G-code and preview	Milliseconds (not measured).
Gantry motion	Model 2.4 to 2.8 s per character (Section 3.9.7). **[MUST STILL BE COMPLETED BY ME: stopwatch time of a sentence of known length, from send to the final pen lift.]**

Table 32. Stages of the timing chains of the two modes and the measurements still missing.

Decision	Alternative	Reason for the chosen design
Character boundaries	Fixed widths, projection minima, hand labelling.	Joined letters have no gaps and hand labelling of about 90 pages is impractical; the recogniser knows the text, so alignment is automatic.
Thin whole line, then cut	Thin each slice.	A slice edge creates a spurious spine stored as a bar.
Several variants per letter	One averaged letter.	Averaging blurs letters whose strokes are arranged differently; variants keep the natural variation, medoid and prototype remain options.
Prior as a filter	Import the median shape into every writer.	Importing would erase the writer; filtering keeps the shapes the writer's own.
Glyphs from scanned pages	Generator learned from on-line pen data.	No on-line data of the ten writers exist (47 to 138 usable lines each), and the pen path must be recovered from the image in any case (Section 3.7).

Table 33. Design alternatives for the style extraction.

Alternative	Assessment
Recurrent sequence generator with attention and mixture-density output [29]	Natural cursive, but needs on-line pen data (hundreds to thousands of lines per writer); only 47 to 138 scanned lines per writer exist. A from-scratch version trained on skeleton-derived strokes learned slant, roundness and connectedness but no legible letters. Rejected.
Adversarial or diffusion image generators [30, 45]	Low data need through pre-training, but the output is a raster: it needs vectorisation with the same thinning artefacts, size, slant and spacing cannot be controlled, and a trial took 35 to 45 s (one line, laptop processor) to 2–3 min against 3 to 8 s for glyph synthesis. Rejected and parked.
Glyph assembly from one writer's sample [31]	Related approach; the present design adds the recogniser-aligned profile and the judge-driven search.
Glyph library with search (chosen)	Needs about nine pages per writer and no training; pen strokes are the native output; size, slant, joins and gap are explicit numbers; failures trace to one glyph; every glyph has a fallback. Weakness: cursive hands, whose joins are lost by cutting.

Table 34. Alternatives to the glyph-library synthesiser (timings are trial runs on a laptop without accelerator).

Function	Offset	Header pin	Direction	Initial level
X motor DIR / STEP	64 / 68	7 / 11	output	high / low
Y motor DIR / STEP	81 / 69	12 / 13	output	high / low
X_MIN / X_MAX switch	72 / 65	15 / 16	input	—
Y_MIN / Y_MAX switch	66 / 67	18 / 22	input	—
Pen-position switch	70	33	input	—
Pen motor drive	71	35	output	low

Table 35. Pin assignment of the gantry driver on `/dev/gpiochip0`, requested in one call; the header pin numbers are those of the program comments and were not checked against the board pinout.

Mode	Page	Chain of operations
Classification	Read	Capture (live preview) or upload → conditioning and line segmentation → per line the text recogniser and the ink-based writer classifier → mean writer probabilities → text, writer, confidence and number of lines; with expected text and writer typed, also the character accuracy and a match mark.
Reproduction	Write	Calibrate once per session → text, writer and N → best-of-N synthesis → G-code, preview, read-back text and accuracy, pen pulses and synthesis time → "send to gantry". An uploaded G-code file uses the same preview and send path.
Training	(none)	Offline programs: segmentation of photographs, training of the networks and the profile builder (Section 3.6); the ten writers are pre-built files and the interface has no training function.

Table 36. Mapping of the proposal's modes to the web interface. A third page shows model architectures, sizes and stored accuracies, and for each writer sample crops and the glyph grid.

Line no.	Transcription written by the rater (without hint of the content)	Illegible characters (positions)	Confidence (1–5)
1			
2			
⋮			
40			

Table 37. Worksheet for the blind human panel: 30 synthesised lines and 10 genuine control lines in random order.

Feature	Definition
Slant	Shear angle (degrees) that maximises the variance of the column ink projection of the line.
Stroke width	Median stroke width from the distance transform at the skeleton, divided by x_{h}.
Ink density	Fraction of ink pixels inside the core band between the baseline and the x-height line.
Word gap	Median horizontal gap between words (gaps larger than $0.5\,x_{\mathrm{h}}$) divided by x_{h}.
Connectedness	Number of connected ink components divided by the number of characters of the text.
Character width	Line width divided by the number of characters, divided by x_{h}.
Ascender/descender ratio	(Height of the ascender zone plus depth of the descender zone) divided by x_{h}.

Table 38. Hand-crafted style features for the correlation and retrieval of qualification test 1 (definitions to be implemented by the student exactly as listed).

Repeat	Cross	x^{cmd} (mm)	y^{cmd} (mm)	x meas. (mm)	y meas. (mm)	e_i (mm)
1	1					
1	2					
⋮						

Table 39. Data sheet for the absolute positional error of qualification test 5; one row per cross and repetition, together with the scanner scale check and the registration check.

Data set	Result	Status of the figure
Public set, validation (976 lines)	94.03 % character accuracy; WER 21.1 %	Used for model selection, therefore mildly optimistic.
Public set, test (2 915 lines)	91.82 % character accuracy; WER 26.7%; 15.9 % of lines exactly right	Used for no decision; the cleanest figure for the general benchmark, with the partition caveats of Table 15.
Personal crops, 669 correctly cut lines (current 13-writer system)	**98.61 % character, 94.21 % word**	Measured on crops the segmenter produced and that were confirmed to match the label count, so segmentation failures are excluded; the writers also appear in the fine-tuning data. This is the operating figure of Section 4, not an end-to-end page figure.
Personal crops, per writer	Owen 99.15 %, Dylan 99.03 %, Abhinav 98.94 %, Yeukita 98.30 %, Thiya 96.46 % character accuracy	75 to 206 crops each; Thiya is the weakest hand for recognition.
Ten personal photographed pages, full page pipeline including segmentation and label alignment	61.21 % (CER 38.79 %, 3 617 of 9 324 characters)	Six unique pages (four listed twice); measured with an earlier recogniser, before the personal pages were used for fine-tuning. It is the only figure in the project that exercises the complete page-to-text chain.
Recognition speed, 669 crops	8.86 ms per character, 353 ms per line	Against the 25 ms per character of R4 this is a margin of about 2.8 times, on the development laptop.

Table 40. Accuracy of the text recogniser. The public-set figures and the full-page figure predate the final training and are retained because they are the only ones measured on an independent partition or through the complete chain; the 669-crop figure is the current operating point.

Stage	Writer ID, direct / G-code (%)	Char. acc., direct / G-code (%)
Plain synthesis after label and glyph fixes	79.2 / 80.0	75.9 / 76.3
+ per-writer λ	83.3 / 86.7	77.2 / 77.8
Text-only best-of-N with plain repair	76.7 / 73.3	83.8 / 82.7
Joint best-of-N, $N = 6$	92.5 / 92.5	79.9 / 79.6
Joint best-of-N, $N = 14$	97.5 / 95.8	81.6 / 81.4
Final system, 13 writers, 3 sentences, $N = 8$	**92.3 / 89.7**	**79.2 / 79.2**
Final system, 13 writers, 6 sentences × 2 seeds, $N = 30$	**80.1 / 78.8**	**82.2 / 82.5**
Matching comparison: the writers' own handwriting of the same words	not applicable	**89.0 / —**
Matching comparison: the system's synthesis of those same words	not applicable	**81.6 / —**

Table 41. Measurements that motivated the design (upper rows) and the final system (lower rows). "G-code" means converted to G-code and read back (Section 3.8). The upper rows and the writer-identification columns are judged by the networks that drive the search and were tuned on the same sentences, so they are not independent evidence; the two matching-comparison rows are independent of the search and are the figures to trust.

Decision	Alternatives	Comparison	Preferred and reason
Unit of recognition	(a) Cut characters, classify each. (b) Cut lines, read with a sequence recogniser.	(a) Needs a segmenter for joined letters and character-level labels (absent); cannot repair a wrong cut. (b) No boundaries below the line; uses context; larger network.	(b): cursive cannot be segmented reliably, labels exist per line, and segmenter errors would bound system accuracy.
Networks for text and writer	(a) One network, two outputs. (b) Separate networks.	(a) One computation, but the tasks compete. (b) Each trainable and measurable alone.	(b): an early joint network learned the writer quickly while text accuracy stayed very low (training-script documentation; log not kept).
Synthesis of writing	(a) Neural generator on pen trajectories. (b) Pretrained image generators. (c) Library of the writer's own glyphs.	(a) Needs recorded pen trajectories (unavailable). (b) Raster output, no control of size or slant, slow without a graphics processor. (c) About nine pages per writer, strokes with controllable parameters; cursive links partly lost.	(c): a from-scratch generator trained on stroke data from the photographs learned slant and roundness but not legible letters; image generators took tens of seconds per line on a processor-only computer.
Selection of output	(a) Use the first result. (b) Draw N candidates, keep the best by recogniser and writer classifier jointly.	(a) Fastest, but random variation gives good and bad results. (b) Time grows with N.	(b): with $N = 14$ writer identification of the synthetic output rose from 79.2 % to 97.5 % and character accuracy from 75.9 % to 81.6 %; the judges also select, so this is not independent evidence.
Location of processing	(a) Offload to a personal computer. (b) All inference on the SBC.	(a) Faster, but contradicts the proposal. (b) Limits memory, compute and methods.	(b), as required; method cost was considered throughout (Section 1.1.7).

Table 42. Software design decisions, alternatives and reasons for the preferred solution.

Decision	Alternatives	Comparison	Preferred and reason
Page capture	(a) Fixed camera. (b) Flatbed scanner.	(a) Instant capture, standard interface, live framing; lighting and tilt handled in software (FC1 applies). (b) Even lighting; larger separate device with a scanning delay.	(a): suits the compact rig and the capture budget of R4; cost is the extensive conditioning of Section 3.3.
Robot type	(a) Cartesian gantry. (b) Serial arm. (c) Parallel robot.	(a) Identity kinematics, flat page, axis errors do not add up. (b) Inverse kinematics, joint errors add up. (c) Stiff, but small workspace, complex kinematics.	(a): position follows from step counts; the 0.5 mm limit of R5 is simplest to meet and verify [34].
Motor control	(a) Separate microcontroller receives moves. (b) SBC generates STEP/DIR pulses.	(a) Offloads timing, but an issued move cannot be stopped without a round trip. (b) Less deterministic timing; limit switches read before every step.	(b): stopping within one step on a limit-switch trip was valued above deterministic timing.

Table 43. Hardware design decisions, alternatives and reasons for the preferred solution.

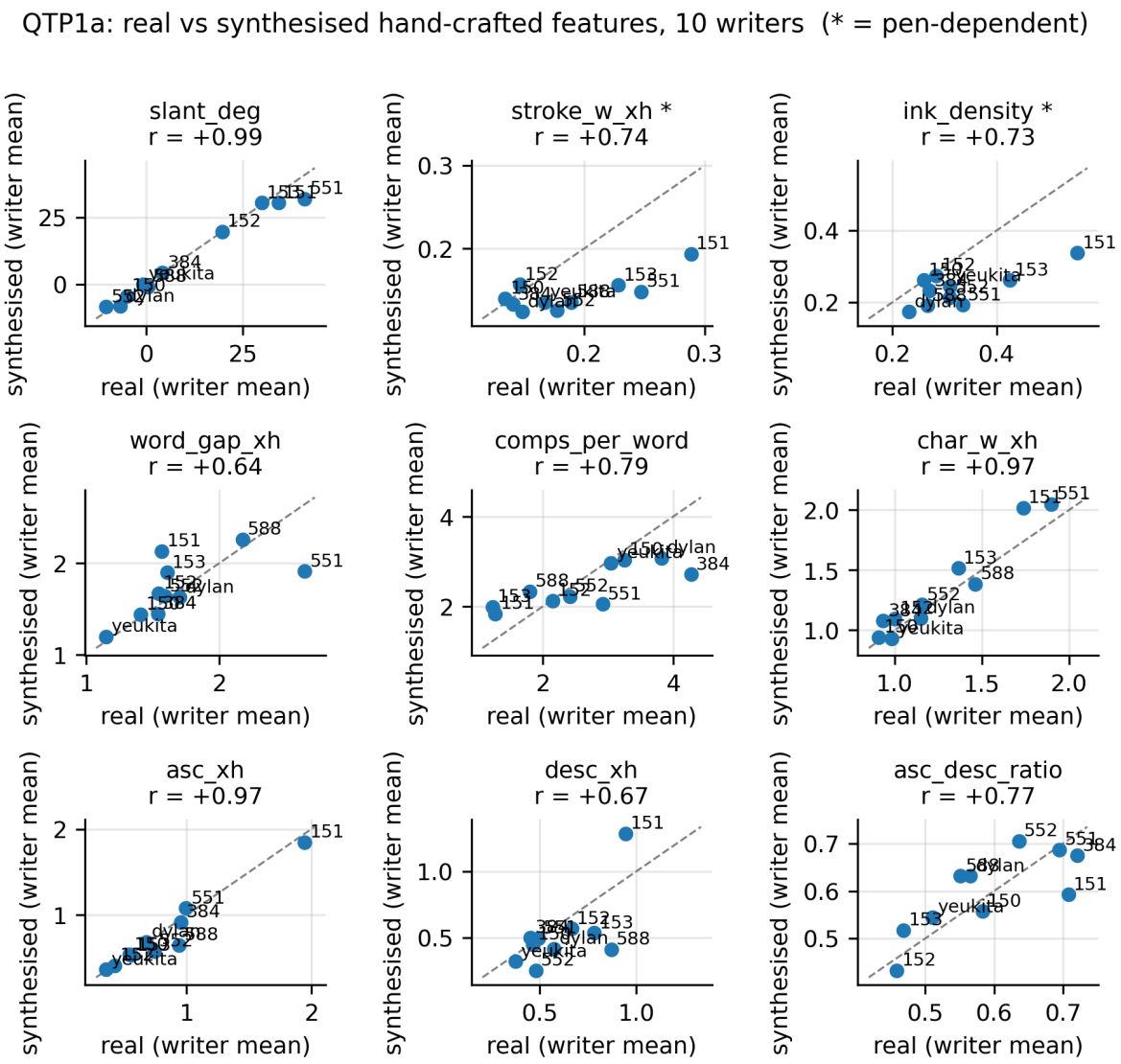


Figure 45. Style-correlation test on nine geometric features extracted by one deterministic routine from held-out real pages and from synthesised lines, for the ten writers. Each panel is one feature; the axes are the writer means. Slant, ascender and character width track the real hand closely, whereas descender and the ascender-to-descender ratio do not, which is the same defect that the word-accuracy gap identifies.

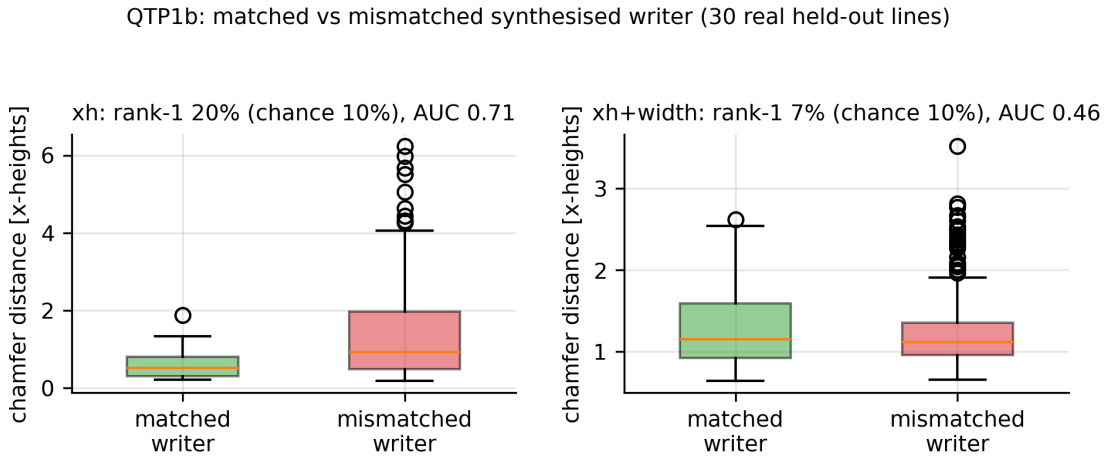


Figure 46. Distributions of the matched and mismatched overlay distances. The two distributions overlap but their means are separated by a factor of about 2.2 on the chamfer measure, giving an AUC of 0.71: the synthesis carries real writer information, but it is far from indistinguishable from the hand it imitates.

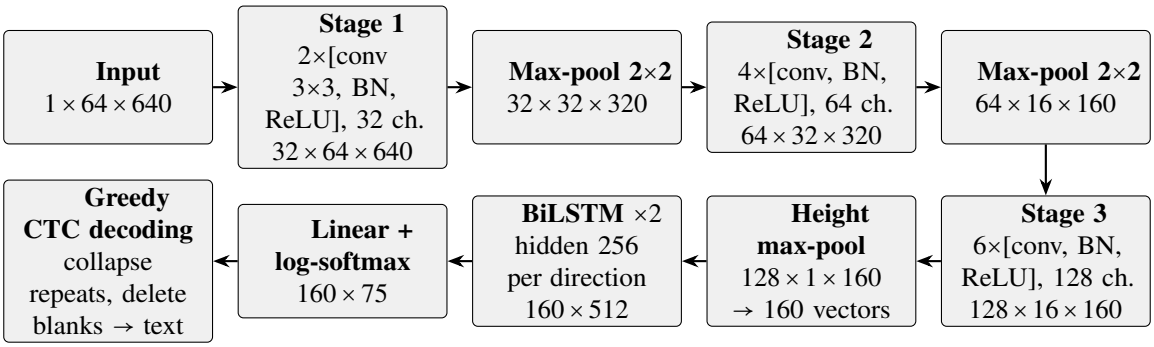


Figure 47. Architecture of the text recogniser. Tensor shapes are channels × height × width for the convolutional part and time steps × features for the recurrent part; BN denotes batch normalisation.

```
def Conv2dForward(x, w, b, padding=1):
    cols = Im2Col(x, kernelSize=3, padding=padding)      # (B, Cin*9, H*W)
    wFlat = w.reshape(w.shape[0], -1)                    # (Cout, Cin*9)
    out = (wFlat @ cols).reshape(B, Cout, H, W) + b[:, None, None]
    return out, (x, w, cols)                             # cache for backward

def Conv2dBackward(dOut, cache):
    x, w, cols = cache
    dOutFlat = dOut.reshape(B, Cout, -1)
    dw = (dOutFlat @ cols.transpose(0, 2, 1)).sum(axis=0).reshape(w.shape)
    db = dOut.sum(axis=(0, 2, 3))
    dCols = w.reshape(Cout, -1).T @ dOutFlat
    dx = Col2Im(dCols, x.shape, kernelSize=3, padding=padding)
    return dx, dw, db
```

Figure 48. Forward and backward pass of the two-dimensional convolution, simplified from the student's implementation. The batch dimension B is handled by a single broadcasted matrix product (`@`) rather than a Python loop over samples, which was measured to be about 35 % faster than an equivalent per-sample loop for a realistic training batch.

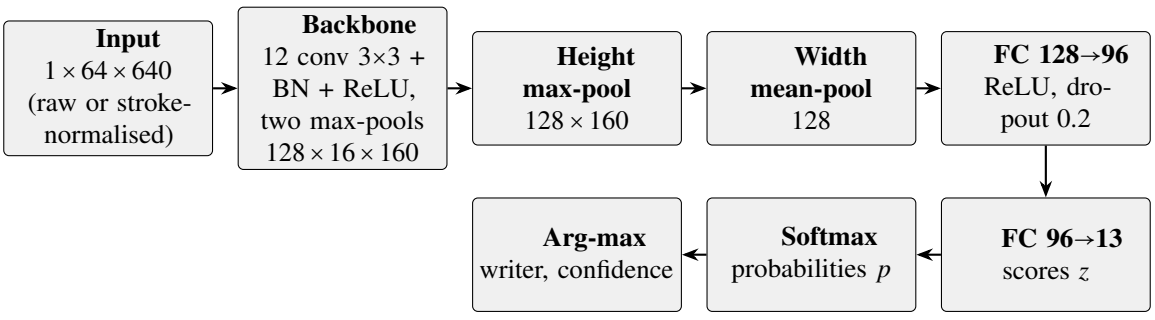


Figure 49. Architecture of the two writer classifiers. The backbone is that of the text recogniser; the head has 13 645 parameters. FC denotes a fully connected (linear) layer.

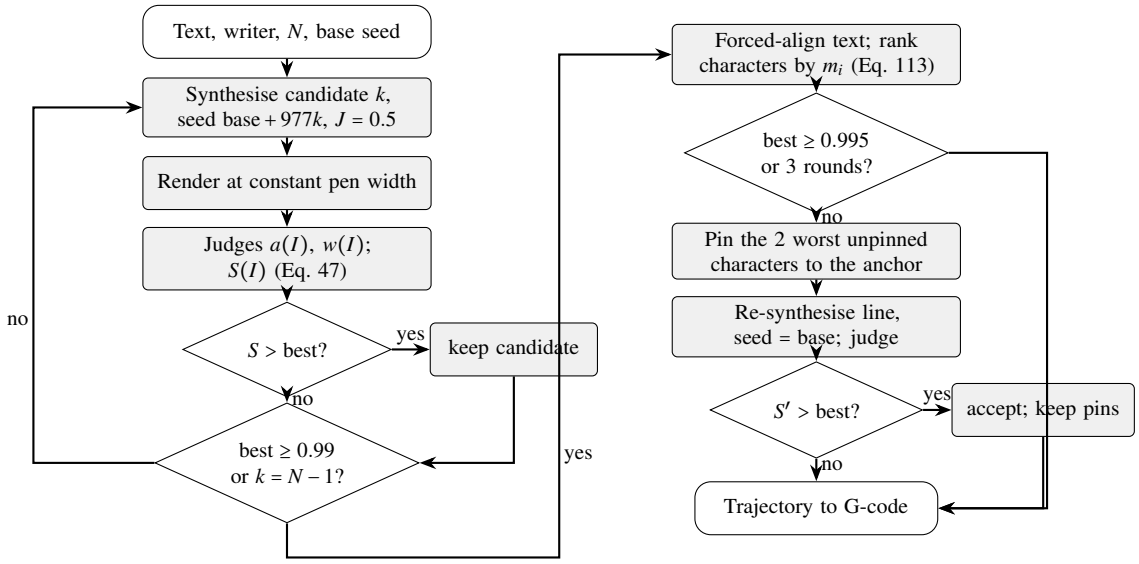


Figure 50. Flow chart of the joint best-of-N search: the search over N candidates (left) and the repair pass (right). A repair round ends after acceptance or rejection and the next round starts from the best candidate; when its test is satisfied the repair loop exits to the output.

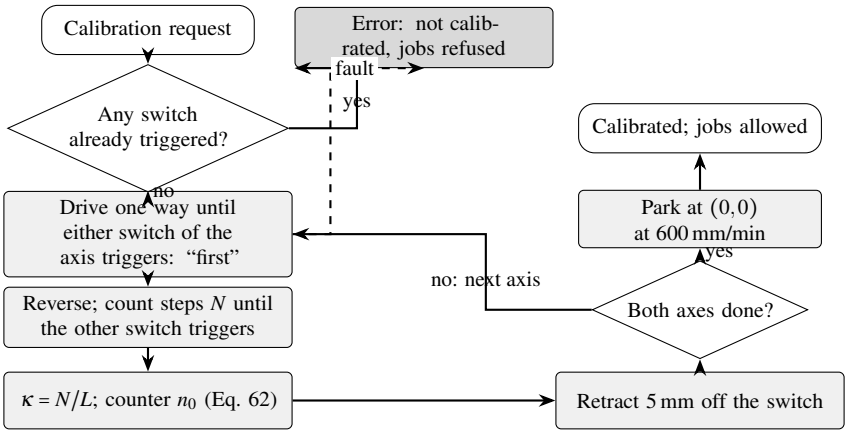


Figure 51. Flow chart of the calibration; the left column runs for each axis in turn (X, then Y). A triggered switch at the start, an unexpected switch or more than 200 000 steps without a trigger is a fault.